# COMPUTERS, CONTROLLERS, AND DATA LINKS

CHAPTER 14

In this chapter and the next we deal with the interesting subjects of computers and controllers (with the common alternative names of *microcomputers* and *microcontrollers*). These are huge subjects, and we do not attempt a complete treatment. Rather, our primary interests, in the context of electronic circuit design, are (a) how to interface external electronics to an existing computer, and (b) how to use a microcontroller as an "embedded" piece of electronics within a custom electronic device or instrument.

In this chapter, then, we look at computers and data *buses* (the mechanism for data interfacing), to understand how they work and how to create a data interface. Along the way we review, briefly, some basics of computers – their architecture, processor and memory hardware, bus timing, and instruction set – and we illustrate interfacing with hardware and code for simple input and output "ports" attached to a classic parallel data bus.[1] We end with a review of popular serial data buses: FireWire, USB, CAN-bus, Ethernet, and wireless (WiFi, Bluetooth, Zigbee).

In Chapter 15 we'll deal with embedded microcontrollers, the inexpensive and wonderful chips that let you put the intelligence of a computer into pretty much anything electronic.

### Some terminology

Mainframes, CPUs, minicomputers, microcomputers, microprocessors, microcontrollers – what are all these names, anyway? History is partly to blame for this proliferation of confusing terms, because names like "microcomputer" were created (with understandable excitement) to describe a computer in which the central processor (the central processing unit, CPU) was integrated onto a single chip (the "microprocessor"), rather than constructed from a board full of smaller logic chips (as it was in a "minicomputer" or "mainframe" of the time).

This is no longer a big deal, and the important distinction is between *computers*, which are used primarily for computational processing of data, versus embedded *controllers*, which are incorporated within other electronic devices that are not themselves computers. Examples of the former are PCs, laptop and notebook computers, and the larger "mainframe" computers. Examples of controllers are the chips that run your electric toothbrush, bathroom scale, television and stereo sets, and the like.[2] In general, the microprocessors used in computers are optimized for computational performance, tend to be expensive, and require support complex "chipsets" in order to function. By contrast, the microcontrollers intended for embedded design trade off computational performance in favor of standalone capability, with as many peripheral functions as possible integrated onto the same chip; they tend to be inexpensive, low power, and completely self-sufficient. Figure 14.1 dramatically illustrates the progress in miniaturizing computer processors for embedded applications[3].

Generally speaking, with the computers we talk about in this chapter, the design of the computer itself (including the integration of memory, disks, and I/O, as well as system programming and utility program development) has been taken care of by the manufacturer (and suppliers of compatible hardware and software). The user need worry only about special-purpose interfaces and the job of user programming. By contrast, in an embedded controller application, the circuit design and choice of hardware, along with all programming, are done by the designer. Computer manufacturers are generally committed to providing system and utility software as part of a complete computing system (often including peripherals), whereas the manufacturers of microcontrollers (semiconductor companies) generally see the design and marketing of newer and better chips as their central tasks.

[1] We've chosen the PC104/ISA bus, both because of its simplicity, and its stability as an entrenched standard in the PC104 embedded PC world.

[2] Sometimes the line is a bit blurry: the processor inside a cable television "set-top" box or Blu-ray™ player, for example, is doing heavy processing of digital video (decryption, decompression, format conversion), yet the box is not a "computer."

[3] And a decade later the same $5 gets you a 100 MHz ARM microcontroller with 1 MByte of flash memory, 80 kBytes of SRAM, 47 parallel I/O ports, 11 channels of 10-bit ADC, a 10-bit DAC, six 16-bit timers, three $I^2C$ and four SPI serial ports, four PWM controllers, four UARTs, and three USARTs. What's not to like about Moore's Law!

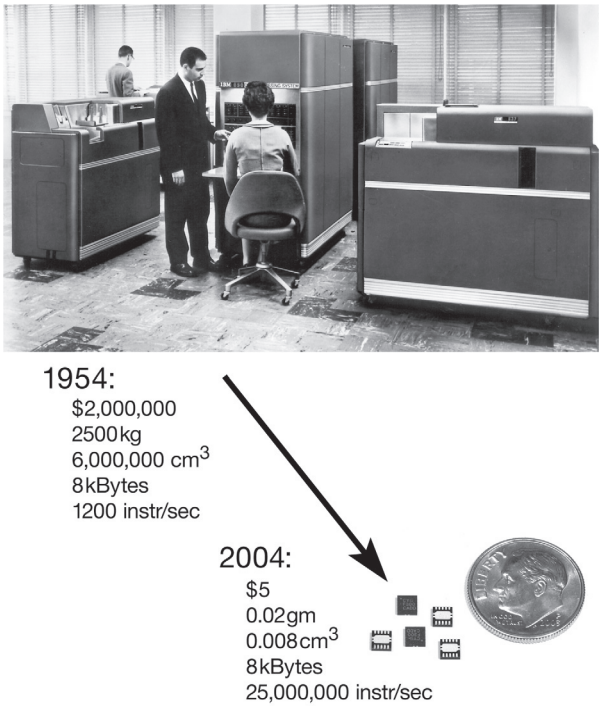



**Figure 14.1.** From mainframe to microcontroller: 50 years of computer progress (mainframe photograph courtesy IBM Corporation).

## *14.1 Computer architecture: CPU and data bus*

In spite of rapid change in the field of computers and microprocessors, there are important basic ideas, many of which carry over to digital electronics generally. These are most easily understood from a look at a traditional bus-oriented architecture, shown in the block diagram of Figure 14.2.

The idea is to link all the connected devices via a shared set of lines – a data "bus" – rather than a rat's nest of direct connections. It uses far fewer wires; and, because traditionally the CPU was responsible for most transactions anyway, it was not necessary to have separate data paths to achieve good performance.[4] So, for example, if the CPU wants to store a byte at some address in random access memory (RAM), it would put the address and the data onto a set of bus lines (called ADR and DATA), then it would assert a WRITE signal; the RAM recognizes the address, and accepts the byte. If the CPU instead wants to fetch a byte, it would put the address only, then it would assert a READ signal; the RAM, recognizing a request for data from one of its addresses, responds by putting the corresponding byte on the DATA lines. Likewise for transfers to or from other bus-attached devices (each of which listens for its assigned range of addresses).

To understand what's going on, let's review the components in Figure 14.2, taking it from left to right.

### 14.1.1 CPU

The central processing unit, or CPU, is the heart of the machine. Computers do their computation in the CPU on chunks of data organized as computer *words*. Word size can range from 4 bits to 64 bits or more, with 32- and 64-bit word sizes being the most popular in contemporary computers. A *byte* is 8 bits (half a byte, or 4 bits, is sometimes called a "nybble"). A portion of the CPU called the *instruction decoder* interprets the successive instructions (fetched from memory), figuring out what should be done in each case. The CPU has an *arithmetic logic unit*, or ALU, that can perform the instructed operations, such as add, complement, compare, shift, move, etc., on quantities contained in *registers* (and sometimes in *memory*). The *program counter* keeps track of the current location in the executing program. It normally increments after each instruction, but it can take on a new value after a "jump" (branch) or a "call" instruction. The *bus control circuitry* handles communication with memory and I/O. Most computers also have a *stack pointer register* (more on that later) and a few *flags* (carry, zero, sign) that get tested for conditional branching. All high-performance processors also include *cache* memory, which holds values (both data and instructions) recently fetched from memory for quicker access.

Contemporary high-performance microcomputers exploit "parallel processing," in which several interconnected CPUs (each with multiple ALU paths) achieve greater computational power; high-density chips integrate this into a "multicore" architecture. However, to keep our discussion simple, and recognizing our interest in *interfacing* to a computer (rather than designing one), we consider only the

[4] This kind of single "multidrop" parallel bus was widely used throughout the minicomputer era (e.g., the DEC "Unibus") and well into the microcomputer era (in the form of the IBM "industry standard architecture" bus, or ISA, which is still used in the PC104 embedded-PC standard). But buses have evolved into more complex creatures, and contemporary PCs have multiple buses and connecting bridges, with names like "Front Side Bus," PCI and PCIe, "Northbridge," and "Southbridge," designed to optimize high-speed communication (such as memory transfers) without being slowed down by low-speed communication (such as peripherals). Also, as we'll see later, in spite of the elegance and economy of shared lines, a "point-to-point" (rat's nest) connection scheme is actually much better than a shared multidrop bus for high-speed communication.

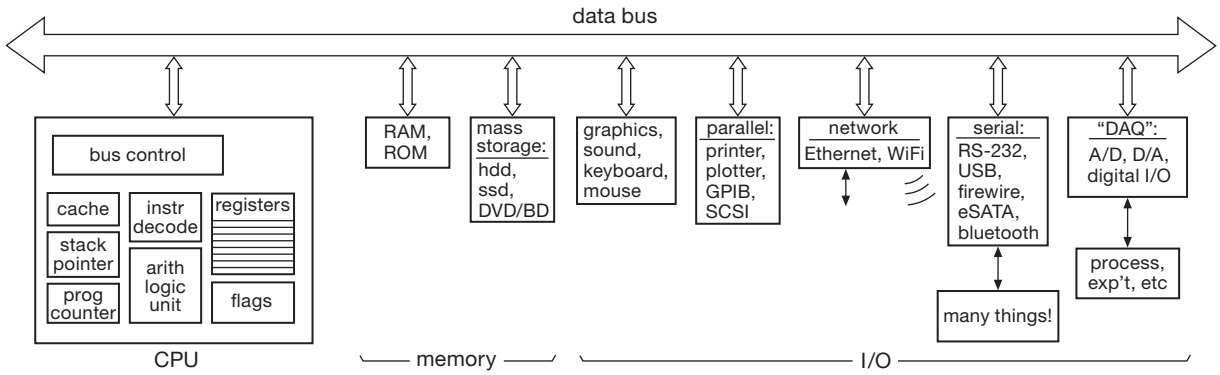



**Figure 14.2.** Traditional bus-oriented computer.

single-CPU machine, executing instructions serially: it's a world of relentless "read, do, write" operations.

### 14.1.2 Memory

All computers have some fast "random-access" memory,[5] called RAM (it used to be called "core," because tiny magnetic cores held the data, one bit per core). In an inexpensive computer this would typically amount to several gigabytes, whereas an embedded micro*controller* typically incorporates much less, as little as 4K–64K.[6] This memory can typically be read and written in about 100 ns. The high-density RAM modules usually used in computers (called DRAM[7]) are *volatile*, which means that its information evaporates when power is removed (maybe it should be called "forgettory"!). All computers therefore include some nonvolatile memory, usually "flash ROM" (read-only memory), to bootstrap the computer, i.e., get it started from a state of total amnesia when power is first turned on. (That nonvolatile memory additionally holds the entire program, in the case of an embedded controller.) Much more about memory in §14.4.

To get or store information in memory, the CPU "addresses" the desired word. Most computers address memory by bytes, beginning at byte 0 and going sequentially through to the last byte in memory. Since most computer words are several bytes long, you are usually storing or fetching a group of bytes at a time; this is usually expedited by having a data bus that is several bytes wide.

Both programs and data are kept in memory during program execution. The CPU fetches instructions from memory, figures out what they mean, and does the appropriate things, often involving data stored somewhere else in memory. General-purpose computers usually store programs and data in the same memory ("von Neumann architecture"), and in fact the computer doesn't even know one from the other. (Amusing things start to happen if a program goes awry and you "execute" data!) Embedded controllers, on the other hand, often use a "Harvard architecture," with separate data and program memory, the latter in non-volatile (usually flash or EEPROM) storage for obvious reasons.

Because computer programs spend most of their time looping through a relatively short sequence of instructions, you can enhance performance by providing a small, but fast, *cache* (pronounced "cash") memory, in which you routinely store copies of the most recently used memory locations. A cached CPU checks its local cache first, before fetching from (slower) main memory; when looping through familiar territory, you often achieve a cache "hit" rate of 95% or better, dramatically improving execution speed.

### 14.1.3 Mass memory

Computers intended for program development or computation, as opposed to embedded controllers, use nonvolatile mass storage devices such as hard disks (HDDs or

[5] So named because you can access data at any address in equal time, as distinguished from something like a shift register or FIFO.

[6] When used to describe memory sizes, K doesn't mean 1000, but rather 1024, or $2^{10}$; thus, 16K bytes (16 KB) is actually 16,384 bytes. We employ the lowercase symbol k to mean 1000.

[7] For *dynamic* RAM: a form of memory in which data bits are stored, briefly, as a voltage state on tiny on-chip capacitors, whose charge must be periodically "refreshed." By contrast, *static* RAM ("SRAM") holds each bit as a flip-flop state, requiring no refresh. DRAM is more compact, requiring one transistor per bit ("1T"), versus 6T for SRAM. Both SRAM and DRAM are volatile, as contrasted with flash ROM or EEPROM ("E-squared").

hard-disk drives: magnetic), solid-state disks (SSDs, or solid-state drives: nonvolatile semiconductor "flash" memory), and removable optical disks (CD, DVD, Blu-ray); the latter come in both read-only and rewritable forms, whereas HDDs and SSDs are always rewritable. We've learned, through past editions of this book, that statements about the impressive performance of "modern" electronics invariably look silly within just a few years.[8] So we simply point out, as a historical datapoint, that we're in the era of terabyte hard disks, 16 GB flash ROMs, and 1 GB RAM chips (factors of $10^3$–$10^4$ larger than reported in our previous edition).

### 14.1.4 Graphics, network, parallel, and serial ports

These are standard items in most computers, needed for communication with the user, with the network, and with external hardware. We presume the reader to be generally familiar with them. Worth noting, however, is the trend away from parallel interfaces (like internal IDE and SCSI, and external "LPT" printer ports), in favor of fast serial interfaces (SATA, USB, FireWire, Ethernet), the latter providing comparable (or better) performance, along with simplified wiring and reduced electrical noise. We'll talk about parallel-port and serial-port communications later, beginning in §14.5.

### 14.1.5 Real-time I/O

For experiment or process control and data logging, or for exotic applications such as speech or music synthesis, you need A/D and D/A devices that can communicate with the computer in "real time," i.e., while things are happening. The possibilities are almost endless here, although a general-purpose set of multiplexed A/D converters (ADCs), a few fast D/A converters (DACs), and some digital "ports" (serial or parallel) for exchange of digital data will permit many interesting applications.[9] Such general-purpose peripherals are commercially available for most popular computer buses, both internal (e.g., PCI and PCIe) and external (USB, FireWire, Ethernet), as exemplified by the extensive offerings from National Instruments. There is increasing usage of external ports, with the advantage of simplicity, flexibility (e.g., use of laptop computers and smartphones, with no access to internal buses), and reduced digital-noise contamination. If you want something fancier, such as improved performance (higher speed, more channels) or special-purpose functions (tone generation, frequency synthesis, time-interval generation, etc.), you may have to build it yourself. This is where a knowledge of bus interfacing and programming techniques, which is always helpful, becomes essential.

### 14.1.6 Data bus

Communication between the CPU and memory or peripherals takes place on a *bus*, a set of shared lines for exchange of digital words. The use of a shared bus vastly simplifies interconnections, since otherwise you would need multiwire cables connecting every pair of communicating devices.[10] With a little care in bus design and implementation, everything works fine.

Buses have evolved over time, reaching such levels of speed and complexity that interfacing now requires considerable expertise. For this reason we are going to adopt an older and simpler bus, called PC104/ISA, which makes interfacing a simple job. The ISA bus was originated in the IBM PC, and as the PC104 bus it is alive and well, and supported by more than 75 manufacturers. A colleague, when asked its likely lifetime, replied "PC104 will bury us all."[11]

In general, a data bus contains a set of DATA lines (generally the same number as bits in a word – 8 for small processors and controllers, 16 to 64 for more sophisticated microcomputers), some ADDRESS lines for determining who should "talk" or "listen" on the line, and a bunch of CONTROL lines that specify what action is going on (data going to or from the CPU, interrupt handling, direct memory access transfers, etc.). All the DATA lines, as well as a number of others, are *bidirectional* – they're driven by three-state devices, or in some cases by open-collector gates with resistor pullups somewhere (usually at the end of the bus, where they also serve as terminators to minimize reflections; see §12.10.1) and Appendix H); pullups may be necessary with three-state drivers also, if the bus is physically long.

[8] In our first edition (1980) we were excited about several-megabyte hard disks, 2 KB EPROMs, and 8 KB RAM; a decade later we were even more excited by several-hundred-megabyte hard disks, 128 KB EPROMs, and 512 KB RAM.

[9] And don't overlook the humble "sound card" (often integrated onto the motherboard), for lab use in noncritical audio-rate applications where dc coupling is not needed. The best of these can do 192 kb/s multichannel sampling at 16-bit resolution.

[10] This idea is repeated within the CPU itself, where internal data buses are used for communication between the arithmetic logic units (ALUs) and the registers (for example). And, in the case of a microcontroller, where "peripherals" like A/D converters and USB ports are integrated onto the same chip, the work of an external bus is carried out with on-chip buses.

[11] We think he meant that it will *outlive* us, not *kill* us. But we're not sure.

Three-state or open-collector devices are used so that devices connected to the bus can disable their bus drivers, because in normal operation only one device is asserting data onto the bus at any time. Each computer has a well-defined protocol for determining who asserts data, and when. If it didn't, total chaos would result, with everyone shouting at once. (Engineers cannot resist personalizing electronic circuits, in spite of advice like "Don't anthropomorphize computers – they don't like it.")

We'll return to the bus in detail, with 8-bit PC104 interface examples. First, though, we need to look at the CPU's instruction set.

## *14.2 A computer instruction set*

### 14.2.1 Assembly language and machine language

To understand bus signals and computer interfacing, you've got to understand what the CPU does when it executes various instructions. At this point, therefore, we would like to introduce the instruction set that goes with the Intel x86-family processors used in PC104 computers. Unfortunately, the instruction sets of most real-world microprocessors tend to be rich with complexities and extra features (including the ghosts of processors past), and the Intel x86 series is no exception. However, since our purpose is only to illustrate bus signals and interfacing (not fancy programming), we'll take a shortcut by laying out a subset of x86 instructions. By leaving out the "extra" instructions we'll wind up with a compact set of instructions that is both understandable and complete enough to do most any programming task. We'll then use it to show some examples of interfacing and programming. These examples will help convey the idea of programming at the "machine-language" level, something quite different from programming in a high-level language like C or C++.

First, a word on "machine language" and "assembly language." As we mentioned earlier, the computer's CPU is designed to interpret certain words as instructions and carry out the appointed tasks. This "machine language" consists of a set of binary instructions, each of which may occupy one or more bytes. Incrementing (increasing by one) the contents of a CPU register would be a single-byte instruction, for example, whereas loading a register with the contents of a memory location would usually require at least two bytes, perhaps as many as five (the first would specify the operation and register destination, and four more would be necessary to specify an arbitrary memory location in a large machine). It is a sad fact of life that different computers have different machine languages, and there is no standard whatsoever.

Programming directly in machine language is extremely tedious, because you wind up dealing with columns of binary numbers, each bit of which has to be bit-perfect, so to speak. Instead, you can represent each instruction in its *assembly-language* readable-text form, using easily remembered mnemonics for the instructions, and symbolic names of your own choosing for memory locations and variables. This assembly-language program, consisting of a plain text file, is then massaged by a program called an *assembler* to produce as its output a finished program in machine-language *object code* that the computer can execute. Each line of assembly code corresponds to one machine-language instruction, which translates to a few machine-language bytes (1 to 15 bytes, for the x86). The computer cannot execute the assembly-language (text) instructions directly. To make these ideas concrete, let's look at our subset of the x86 assembly language and do a few examples.

### 14.2.2 Simplified "x86" instruction set

The x86-family processors (Intel, AMD, VIA) have a rich, and somewhat idiosyncratic, instruction set; part of its complexity stems from the designers' objective to maintain compatibility with the original 8080 8-bit processor. Later and more sophisticated processors in the x86 family (e.g., Pentium, Core 2, i3/i5/i7, Xeon) have many additional instructions but can still execute the original x86 instruction set. We've gone through those instructions with a machete, keeping just 10 arithmetic operations and 11 others (Table 14.1).

#### A. A quick tour

Some explanations: the first six arithmetic instructions in Table 14.1 operate on pairs of numbers ("2-operand" instructions), which we've abbreviated as *b,a*, and which can be any of the 5 pairs listed in the table footnotes; *m* means the contents of a memory location, *r* means the contents of a CPU register (there are 8 in the original 8086), and *imm* means an *immediate* argument, which is a number stored in the next 1 to 4 bytes of memory following the instruction. Thus, for example, the instructions

```
MOV   count,CX
ADD   small,02H
AND   AX,007FH
```

**Table 14.1 Simplified x86 Instruction Set**

| Instruction | | What you call it | What it does |
|---|---|---|---|
| *arithmetic* | | | |
| **MOV** | *b,a* | move | *a* → *b*; *a* unchanged |
| **ADD** | *b,a* | add | *a* + *b* → *b*; *a* unchanged |
| **SUB** | *b,a* | subtract | *b* – *a* → *b*; *a* unchanged |
| **AND** | *b,a* | and | *a* AND *b* → *b* bitwise; *a* unchanged |
| **OR** | *b,a* | or | *a* OR *b* → *b* bitwise; *a* unchanged |
| **CMP** | *b,a* | compare | set flags as if *b* – *a*; *a*,*b* unchanged |
| **INC** | *rm* | increment | *rm* + 1 → *rm* |
| **DEC** | *rm* | decrement | *rm* – 1 → *rm* |
| **NOT** | *rm* | not | 1's complement of *rm* → *rm* |
| **NEG** | *rm* | negate | negative (2's comp) of *rm* → *rm* |
| *stack* | | | |
| **PUSH** | *rm* | push | push *rm* onto stack (2 bytes) |
| **POP** | *rm* | pop | pop 2 bytes from stack to *rm* |
| *control* | | | |
| **JMP** | *label* | jump | jump to instr *label* |
| **J***cc* | *label* | jump conditional | jump to instr *label* if *cc* true |
| **CALL** | *label* | call | push next adr, jump to instr *label* |
| **RET** | | return | pop stack, jump to that adr |
| **IRET** | | return from int | pop stack, restore flags, return |
| **STI** | | set interrupt | enable interrupts |
| **CLI** | | clear interrupt | disable interrupts |
| *input/output* | | | |
| **IN** | AX,*port* | input | *port* → AX (or AL) |
| **OUT** | *port*,AX | output | AX (or AL) → *port* |

**Notes:** *b,a*: any of *m,r r,m r,r m,imm r,imm*
*rm*: register or memory, via various addressing modes
*cc*: any of **Z NZ G GE LE L C NC S NS**
*label*: via various addressing modes
*port*: byte (via imm) or word (via DX)

have arguments of the form *m,r*, *m,imm*, and *r,imm*, respectively. The first copies the contents of register CX to a memory location that we've named "count"; the second adds 2 to the contents of another memory location called "small"; the third clears the top 9 bits of 16-bit register AX while preserving the bottom 7 bits unchanged (a so-called *masking* operation). Note Intel's argument convention: the first argument is replaced with or modified by the second argument.

The last four arithmetic operations in the table take only a single operand, which can be the contents of either a register or memory. Here are two examples:

```
INC   count
NEG   AL
```

The first adds 1 to the contents of memory-location "count," and the second changes the sign of the contents of register AL.

### B. A detour: addressing

Before continuing, a word on registers and memory addressing. The original 8086 claimed to have eight "general-purpose" registers, but after reading the fine print you'll realize that most of them also have special uses (Figure 14.3). Four of them (A–D) can be used either as single 16-bit registers (e.g., AX; think of "X" as "extended") or as a pair of byte registers (AH, AL; "high" and "low" halves).[12] The BX and BP registers can hold addresses, as can the SI and DI registers, and tend to be used for addressing (see below). Special looping instructions (which we omitted from our short list) use register C, and multiply/divide and I/O instructions make analogous use of registers A and D.

Data used in instructions can be immediate constants, values held in a register, or values in memory. You

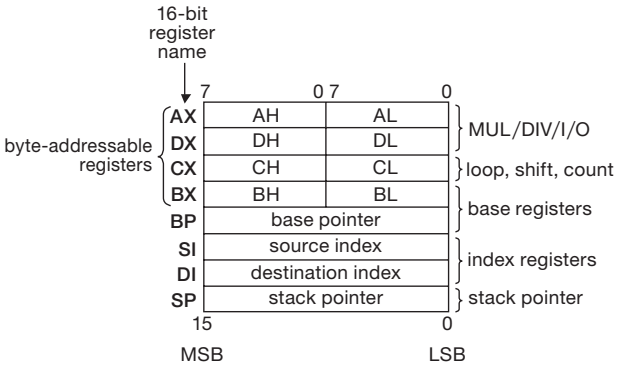



**Figure 14.3.** 8086 general-purpose registers.

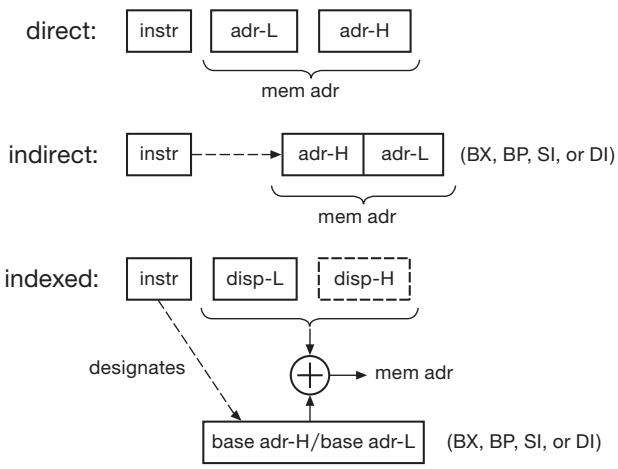



**Figure 14.4.** Some addressing modes.

[12] Later x86 CPUs extended these to 32 bits and to additional 64-bit registers.

specify immediates by value, and registers by name, as in the examples above. To address memory, the x86 provides six addressing modes, three of which are described by the diagrams in Figure 14.4. In assembly language you can just name the variable *directly*, in which case its address gets assembled as a pair of bytes immediately following the instruction; you can put the variable's address in an addressing register (BX, BP, SI, or DI), then use an instruction that specifies addressing *indirectly* through the register; or you can combine these actions, adding an immediate *displacement* to the value in a designated addressing register to get the variable's address. The indirect mode is faster (assuming the address has already been loaded into an addressing register) and much better if you want to do something to a whole set of numbers (a *string* or *array*). Here are a few addressing examples:

```
MOV   count,100H     (direct,immediate)
MOV   [BX],100H      (indirect,immediate)
MOV   [BX+1000H],AX  (indexed,register)
```

The last two assume you've already put an address into BX. The last instruction copies the contents of AX to a memory location 4K (1000 hex) higher than where BX points in memory; we'll give an example shortly, showing how you could use this to copy an array.

There's one other complexity of x86 memory addressing that we've swept under the rug: the "address" generated by any of these addressing modes is not actually the final address, as should be obvious from the fact that the address register BX has only 16 bits (which can address only 64K bytes of memory). In fact, it's called an *offset*; to get an actual address, you add to the offset a 20-bit *base* formed by shifting left 4 bits the contents of a 16-bit *segment register* (there are four such registers). In other words, the x86 lets you access groups of 64K bytes of memory at a time, with the location of those "segments" within a total memory size of 1MB set by the contents of the segment registers. The use of 16-bit addressing in the 8086 was basically a big mistake, inherited from earlier generations of microprocessors. Newer processors (80386 onward, and recent designs in other CPU families) are done right, with 32-bit or 64-bit addressing throughout.[13] Rather than complicate our examples, we'll simply ignore segments entirely; in real life you would, of course, have to worry about them.

[13] And with an additional level of indirection (virtual memory implemented with page tables) on the way to the actual physical memory. But you didn't really want to know about that, right?

### C. Instruction-set tour (continued)

The *stack* instructions PUSH and POP come next. A stack is a portion of memory, organized in a special way: when you put data onto the stack (a *push*), it goes into the next available spot ("top" of the stack); and when you retrieve data (a *pop*), it is taken from the top, i.e., it is the item last pushed onto the stack. Thus a stack is a consecutive list of data, stored last-in, first-out (LIFO). It may help to think of a lunchroom tray dispenser.

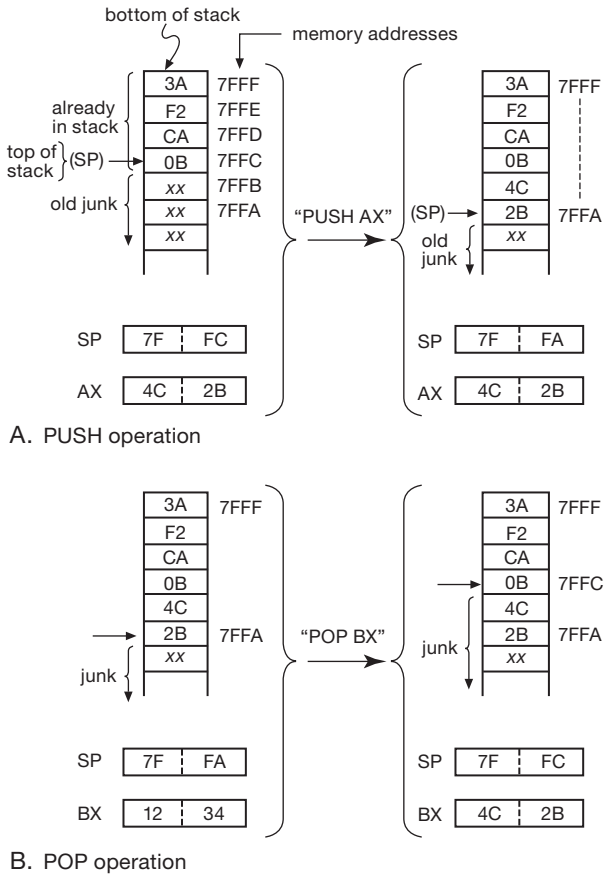



**Figure 14.5.** Stack operation: A. effect of PUSH (illustrated with register AX); B. effect of POP (illustrated with register BX.

Figure 14.5 shows how it works. The stack lives in ordinary RAM, with the CPU's *stack pointer* (SP) keeping track of the location of the current "top" of the stack. The 8086 stack holds 16-bit words and grows *down* in memory as you push data onto it. The SP is automatically decremented by 2 before each PUSH, and incremented by 2 after each POP. Thus, in the example, the 16-bit data in register AX is copied onto the top of the stack by the

**Program 14.1**

```
        MOV  BX,1000H       ;put array address in BX
        MOV  CL,100         ;initialize loop counter
LOOP:   MOV  AX,[BX]        ;copy array element to AX
        MOV [BX+400H],AX    ;then to new array
        ADD  BX,2           ;increment array pointer
        DEC  CL             ;decrement counter
        JNZ  LOOP           ;loop if count not zero
NEXT:   (next statement)    ;exit here when done
```

instruction PUSH AX; the SP is left pointing at the last byte pushed. POP reverses the process, as shown. As we will see, the stack plays a central role in subroutine calls and interrupts.

JMP causes the CPU to depart from its usual habit of executing instructions in sequential order, detouring instead to the instruction that you jump to. Conditional jumps (there are ten possibilities, indicated generically as *Jcc*) test the flag register (which lives in the CPU and whose bits are set according to the result of the most recent arithmetic operation), then either jump (if the condition is true) or execute the next instruction in sequence (if the condition is not true). Program 14.1 shows an example. It copies 100 words from the array beginning at 1000 hex to a new array beginning 1kB (400H) higher.

Note the explicit loading of the pointer (to address register BX) and the loop count (to CL). The actual array of words had to move through a register (we chose AX) because the 8086 does not permit memory-to-memory operations (see the instruction set notes). At the end of the 100th pass through the loop, CL = 0, and the jump nonzero (JNZ) instruction no longer jumps. This example will work, but in practice you would probably use one of the x86's faster *string move* instructions. Also, it's good programming practice to use symbolic names for sizes and arrays, rather than constants like 400H and 1000H.

The CALL statement is a subroutine (or "procedure" or "function") call. It's like a jump, except that the return address (the address of the instruction that would have come next, except for the intervening CALL) is pushed onto the stack. At the end of the subroutine you execute a RET statement, which pops the stack so the program can find its way home (Figure 14.6). The three statements STI, CLI, and IRET have to do with interrupts, which we'll illustrate with a circuit example later in the chapter. Finally, the I/O instructions IN and OUT move a word or byte between the A register and the addressed port; more on this shortly.

### 14.2.3 A programming example

As the array-copy example above suggests, assembly language tends to verbosity, with a lot of little steps needed to do a basically simple thing. Here's another example. Suppose you want to increment a number, *N*, if it equals another number, *M*. This will typically be a tiny step in a larger program, and in higher level languages it will be a single instruction:

```
if (n==m) n++;          (C, C++, Java)
if n==m:                (Python)
  n+=1
IF (N.EQ.M) N=N+1       (FORTRAN)
if n=m then n:=n+1;     (Pascal), etc
```

In x86 assembly language, it looks like Program 14.2. The assembler program will convert this set of mnemonics to machine language, generally translating each line of assembler *source code* to one *object code* instruction (occupying several machine-language bytes), and the resultant machine-language code will get loaded into successive locations in memory before being executed. Note that it is necessary to tell the assembler to assign some storage space for variables. This you do with the assembler *pseudo-op* "DW" (define word) (called a *pseudo*-op because it doesn't produce any executable code). Unique symbolic labels (e.g., NEXT) can be used to tag instructions; this is usually done only if there is a jump to that location (JNZ NEXT). Giving some locations understandable (to you!) names and adding comments (separated by a semicolon) make the job of programming easier; it also means that you have a chance of understanding what you've written a few weeks later.

Programming in assembly language can still be a nuisance, but it is sometimes preferable to write short routines in it, callable from a higher-level language, to code tight loops, or to handle unusual I/O. Assembly-language programs often run faster than programs compiled from a

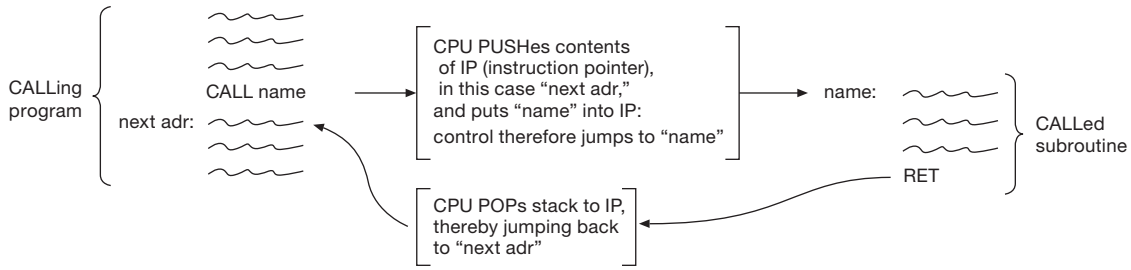



**Figure 14.6.** CALL operation.

higher language, so they are sometimes used where speed is crucial (e.g., the innermost loop of a long numerical calculation). To a great extent the development of the powerful C and C++ programming languages has minimized the occasions when you must use assembly code. In any case, you can't really understand computer interfacing without understanding the nature of assembly-language I/O. The correspondence between mnemonic assembly language and executable machine language is explored further in the student manual[14] in the context of microcontroller programming.

### *14.3 Bus signals and interfacing*

A typical microcomputer data bus has 50–100 signal lines, devoted to the transfer of data, addresses, and control signals. The PC104/ISA bus is typical of a small machine, with 53 signal lines and 8 power and ground lines. Rather than throw them at you all at once, we will approach the subject by building up the bus, beginning with the signal lines necessary for the simplest kind of data interchange (programmed I/O) and adding additional signal lines as they become necessary. We will give some useful interface examples as we go along, to keep things comprehensible and interesting.

The PC104 bus signals are carried from board to board on a 64-pin (2 rows of 32 pins each) stack-through connector: it is a socket on the top side, and a plug on the bottom. Figure 14.7 shows a PC104 stacked pair, with CPU above a fast high-resolution ADC peripheral board.

#### 14.3.1 Fundamental bus signals: data, address, strobe

To move data on a shared (party-line) bus, you have to be able to specify the data, the recipient, and the moment when data is valid. Thus a minimum bus must have DATA lines (for the data to be transferred), ADDRESS lines (to identify the I/O device or memory address), and some STROBE lines (which tell when data is being transferred). There are usually as many DATA lines as bits in the computer word, so a whole word can be transferred at once. In the 8-bit PC104,[15] however, there are only eight DATA lines (D0–D7); you can move a byte in one transfer, but to move a 16-bit word you have to do two transfers. The number of ADDRESS lines determines the number of addressable devices: if the bus is used for both I/O and memory (the usual situation) there will be 16 to 32 ADDRESS lines

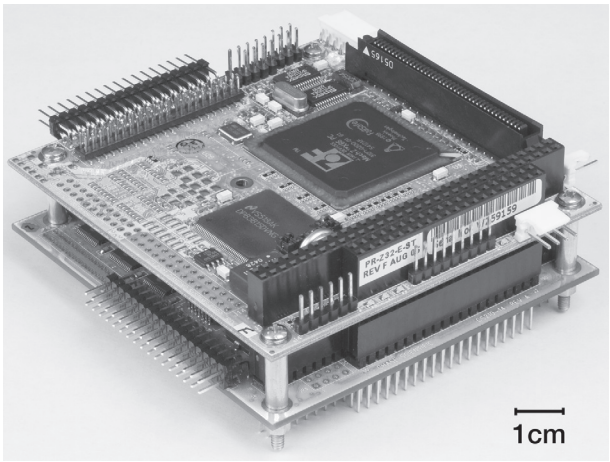



**Figure 14.7.** A PC104 CPU motherboard (top), with attached fast ADC (bottom). The Diamond Systems CPU includes serial and parallel I/O, USB, video, keyboard, mouse, hard-disk and floppy interfaces, and network ports; the Chase Scientific peripheral includes a pair of 14-bit 10 Msps ADCs, along with digital ports and memory. The PC104 stack-through bus can be seen along the southeast edge: 2×32 pins for the 8-bit bus, plus 2×20 pins for the 16-bit extension – voilà, 104 pins in all.

[14] Hayes and Horowitz: *Learning the Art of Electronics – a Hands-On Course*, Cambridge University Press, 2015.

[15] The PC104/ISA specification allows both 8-bit and 16-bit buses; we will use the 8-bit version, for simplicity.

**Program 14.2**

```
n  DW 0     ;n (a "word") lives here, and
m  DW 0     ;m lives here, both initialized to 0

        MOV  AX,n   ;get n
        CMP  AX,m   ;compare
        JNZ  NEXT   ;unequal, do nothing
        INC  m      ;equal, increment m
NEXT:   (next statement)
          o
          o
          o
```

(corresponding to a 64 KB to 4 GB address space); a bus used for I/O only might have 8 to 16 ADDRESS bits (256 to 64K I/O devices). PC104/ISA talks to both memory and I/O on its bus, and has 20 ADDRESS lines (A0–A19), corresponding to a 1 MB address space.

Finally, data transfer itself is synchronized by pulses on additional "strobing" bus lines. There are two ways in which this can be done: by having separate READ and WRITE lines, with a pulse on one or the other synchronizing data transfer; or by having one DATA STROBE line (DS) and one READ/WRITE′ line (R/W′), with a pulse on DS synchronizing data transfer in a direction specified by the level on the READ/WRITE′ line. PC104/ISA uses the first scheme,[16] with (active-LOW) READ/WRITE lines called IOR′, IOW′, MEMR′, and MEMW′; there are four because the PC distinguishes between memory and I/O addresses, with individual pairs of READ/WRITE strobes for each.

These bus signals – DATA, ADDRESS, and the four strobes – would normally be all you need to do the simplest kind of data transfers. However, on the PC104 bus you need one more, called ADDRESS ENABLE (AEN), to distinguish normal I/O transfers from what's called "direct memory access" (DMA). We'll get to DMA in §14.3.10; for now, all you need to know is that AEN is LOW for normal I/O, and HIGH for DMA. We now have 33 bus signals: D0–D7, A0–A19, IOR′, IOW′, MEMR′, MEMW′, and AEN. Let's see how they work.

### 14.3.2 Programmed I/O: data out

The simplest method of data exchange on a computer bus is known as "programmed I/O," meaning that data is transferred via an IN or OUT statement in the program (the directions for IN and OUT are among the few things on which all computer manufacturers agree: IN always means *toward* the CPU, and OUT always means *from* the CPU). The whole process of data OUT (and memory write) is extremely simple and logical (Figure 14.8). The ADDRESS of the recipient and the DATA to be sent are put onto the respective bus lines by the CPU. A write strobe (IOW′) is asserted (LOW) by the CPU to signal the recipient that data is good. On the PC104/ISA 8-bit bus, the address is guaranteed valid beginning ∼100 ns before IOW′, and the data are guaranteed valid ∼500 ns before the end of IOW′ (and for another 25 ns thereafter). To play the game, the peripheral looks at the ADDRESS and DATA lines. When it sees its own address, it latches the information on the DATA lines, using the trailing edge of the IOW′ pulse as a clocking signal. That's all there is to it.

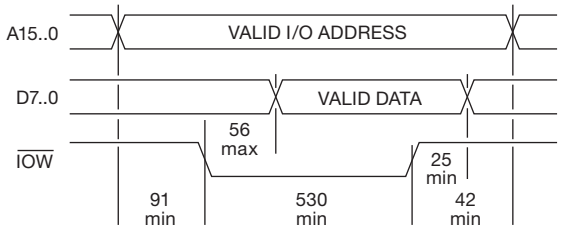



**Figure 14.8.** I/O WRITE cycle. Timing is in units of nanoseconds. Note that timing diagrams like these are rarely drawn to scale.

#### A. Example: byte-wide register

Figure 14.9 shows this simple logic: the box labeled "adr decode" produces a HIGH "adr match" output when the address lines A15..0 contain the peripheral's assigned address; this enables the gate to produce an output pulse when IOW′ goes LOW (signaling a write to the peripheral), the trailing edge of which clocks the outgoing data into the byte-wide D-register. You can summarize this compactly

[16] Because historically Intel processors used that scheme; Motorola chose R/W′ and DS.

by saying "the data is clocked into the register by the write pulse, qualified by the decoded address." Note that we've used the *trailing* edge of IOW′ to clock the D-register: that is because the data is not yet valid on the leading edge, but is guaranteed valid by the trailing edge – in fact, the data has a generous setup time of ≥474 ns (and a hold time of ≥25 ns) relative to that edge. We've shown also the analogous gating for the alternative "Motorola" bus signal pair of direction and timing (R/W′ and DS′), which replaces the "Intel" pair of IOR′ and IOW′.

The assembly code for this interface is outrageously simple. If we want to send out the byte that's already in register AL, this is all it takes:

```
OUT 3F8h, AL ;(send to port adr = 3F8 hex)
```

The processor springs to action, with the prescribed bus hardware response of Figure 14.8: first it asserts the specific address 3F8 (hex)[17] on address lines A9..0, then it asserts IOW′ and puts the byte in AL onto the data lines D7..0, and finally it disasserts IOW′, then the data and address. Then it obediently fetches and executes the next instruction.

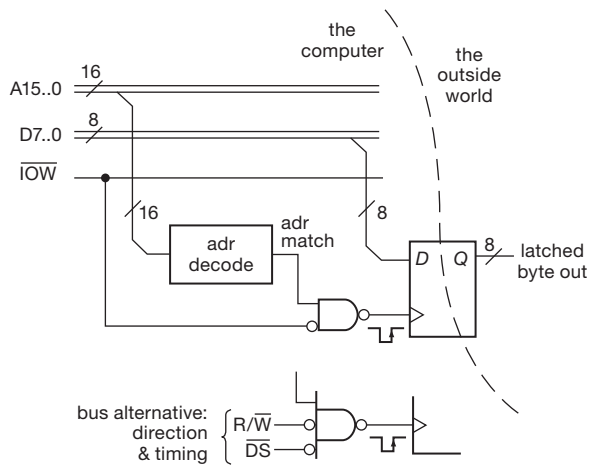



**Figure 14.9.** Programmed data out: a byte asserted by the CPU onto the data lines is clocked to an external device by the write pulse, if the address lines hold the peripheral's assigned address. For simplicity we've ignored the PC104's quirky AEN signal (see Figure 14.10).

### B. Example: 16-bit XY vector graphic display

A more complete (and interesting) example is shown in Figure 14.10, where we've interfaced a pair of high-resolution (16-bit) D/A converters to the PC104 8-bit bus, for example to drive a high-resolution vector graphic display screen. This might be used in conjunction with a high-resolution (up to 64k×64k!) *XY* oscillographic display device that accepts analog *x* and *y* input voltages and that plots the corresponding point by unblanking the trace at each point when a *z*-input logic level is asserted.

The Analog Devices AD660 DACs have a pair of internal 8-bit registers that you load by asserting an enable (HBE′ and LBE′ for high and low byte) while clocking the WR′ line; after that you transfer the pair to the internal 16-bit register that holds the value being converted to an output voltage (Figure 14.11). This scheme prevents false outputs: it lets you transfer the 16-bit values one byte after another, and then transfer them all, simultaneously, to the actual DAC output. It does, however, add a bit of complexity, when compared with the simple byte-wide register output port of Figure 14.9.

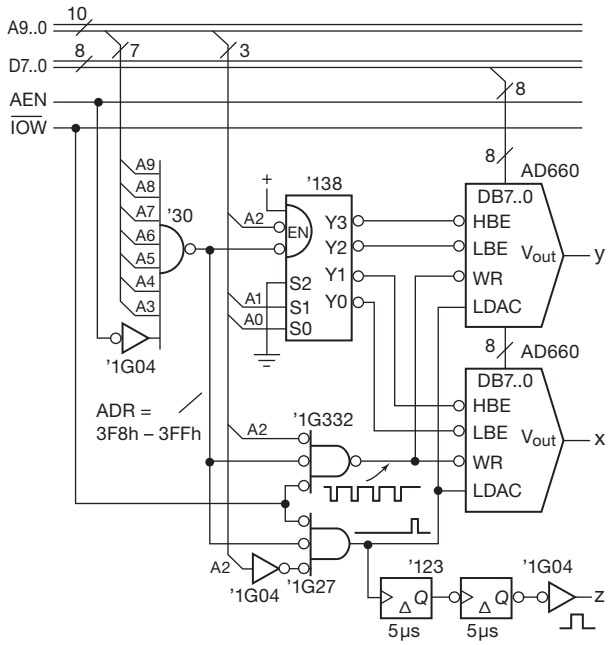



**Figure 14.10.** Dual-channel 16-bit DAC interface on the PC104 8-bit bus.

For this interface we use 8 successive bytes (addresses 3F8h–3FFh, or binary 11 1111 1xxx), choosing "base address" 3F8h (binary 11 1111 1000). The 8-input NAND detects this address range, qualified by the AEN bus signal asserted LOW.[18] Its output enables the '138 1-of-8 decoder (§10.3.3D), which responds to the two low-order address lines (A1 and A0) to enable the successive HBE′ and LBE′

[17] We trust the reader will forgive our unlucky address choice: 3F8h is the x86's default serial port.

[18] AEN is asserted HIGH to signal a *DMA transfer* (§14.3.10); so for ordinary programmed I/O it must be LOW.

during byte-wide writes to addresses 3F8h, 3F9h, 3FAh, and 3FBh. After loading, these bytes are transferred to the business end of the DAC by a write to address 3FCh, which causes the lower gate to generate an LDAC (load DAC) pulse when it sees the IOW′. Note that this last "write" needs no data; the circuit ignores D7..0, and just uses the IOW′, qualified by address 3FCh.[19] The LDAC pulse also generates a delayed "*z*-axis" unblanking pulse, providing time for the DACs and vector display screen to settle before displaying each $x, y$ point.

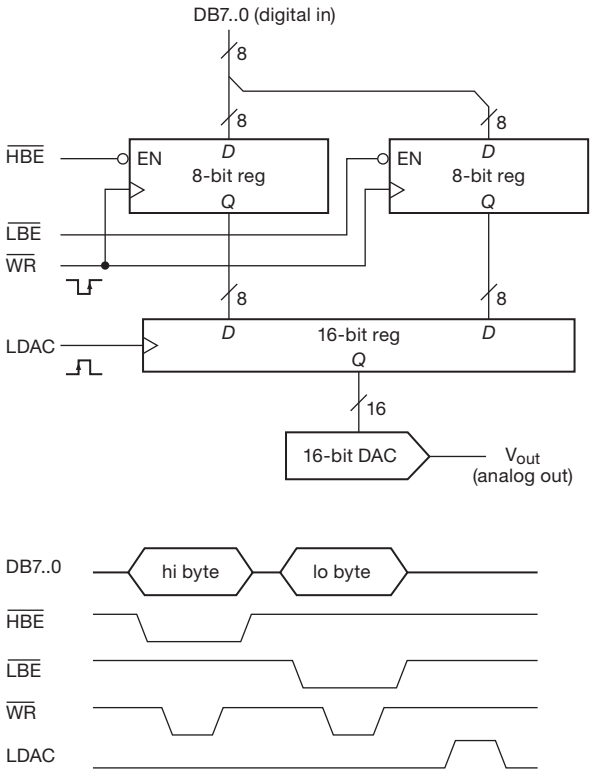
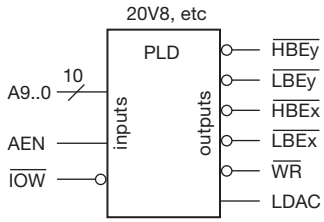



**Figure 14.11.** Block diagram and timing sequence of the double-buffered AD660 16-bit ADC with byte-wide parallel data input.

In practice you would probably combine all the logic, including address decoding, into a programmable logic device (PLD, Chapter 11), as in Figure 14.12; you might additionally include jumpers to set the base address.




```
DACSEL = a9 & a8 & a7 & a6 & a5 & a4 & a3 & !aen;
wr_bar = DACSEL & iow_bar & !a2;
ldac = DACSEL & iow_bar & a2;
lbex_bar = DACSEL & !a2 & !a1 & !a0;
hbex_bar = DACSEL & !a2 & !a1 & a0;
lbey_bar = DACSEL & !a2 & a1 & !a0;
hbey_bar = DACSEL & !a2 & a1 & a0;
```

**Figure 14.12.** The logic of Figure 14.10 fits easily into a Lilliputian PLD like the 20V8. The code is written with logical assertion, assuming that active-LOW polarities are so defined in the HDL header file.

### 14.3.3 Programming the *XY* vector display

The programming to run this interface is straightforward. Program 14.3 shows what you do. The addresses of the first $x$ and $y$, and the number of points to be plotted, have to be available to the program. The display program will probably be a subroutine, with those parameters passed as arguments in the subroutine call. The program puts the addresses of the $x$ and $y$ arrays (i.e., the address of the first $x$ and $y$) into address pointer registers SI and DI, and the number of points to be plotted into CX. It then enters a loop in which successive $x, y$ pairs are sent to I/O ports 3F8h and 3FAh. The $x$ and $y$ 2-byte pointers are advanced each time around, and the counter is decremented and tested for zero, which indicates that the last point has been displayed; the pointers and counter are then reinitialized, and the process begins again. The $x$ and $y$ values are each 2-byte (16-bit) integers; the code fetches them each with a single MOV (implicit in the use of the 16-bit internal AX register), and sends each out as two successive write-cycle transfers on the 8-bit bus. The x86 processors store multibyte quantities in successive memory locations in the byte order smallest-to-largest,[20] beginning always on an even-numbered byte. That is why we assigned the LBE and HBE pair addresses as shown.

A couple of important points: once started, this

[19] Actually, qualified by any address in the range 3FCh–3FFh, because the 3-input gate ignores A1 and A0, in what we call "lazy address decoding."

[20] Known as "little-endian," to be distinguished from the alternative scheme of "big-endian" whose byte order is largest-to-smallest in successive memory locations; see §14.8. (OK, it's a corny pun – but, hey, we didn't invent it, so don't shoot the messenger.)

program displays the $x, y$ array forever. In real life the program would probably check the keyboard to see if the operator wants the plot terminated. Alternatively, the display could be terminated after a specified time had elapsed, or by an "interrupt," which we will discuss shortly. With this sort of *refreshed* display there usually isn't time to do much computing while displaying. A display device refreshed from its own memory takes that burden off the computer, and this is generally a better method. Nevertheless, if the objective is to make a precision plot for photographic hard copy, this program and interface would do the job.[21]

### 14.3.4 Programmed I/O: data in

The other direction (data *in*) of programmed I/O is similar. The interface looks at the ADDRESS lines as before. If it sees its own address (and AEN is LOW), it puts data onto the DATA lines coincident with the IOR′ pulse (Figure 14.13). Figure 14.14 shows the correspondingly simple hardware. This interface lets the processor read a byte latched in the '374 *D*-type register, which conveniently has three-state outputs.[22] Because the clock input and data inputs of the register are accessible to an external device, the register could hold just about any sort of digital information (the output of a digital instrument, A/D converter, etc.). The line labeled "ADR=200h" comes from an address decoder (e.g., a PLD, or discrete gates, that produce a HIGH when A9..0 contain binary 10 0000 0000); from now on we'll omit such minor implementation details.

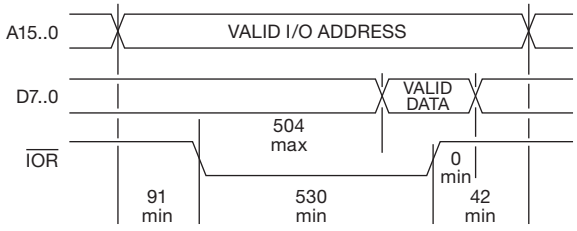



**Figure 14.13.** I/O READ cycle. Timing is in units of nanoseconds.

When an "IN AL,200H" instruction is executed, the CPU asserts $200_H$ (hexadecimal 200, sometimes written 0x200) on A9..0, waits a while, then asserts IOR′ for 530 ns. The CPU latches what it sees on the DATA lines (D7..0) at the trailing edge of IOR′, then disasserts A9..0. The peripheral's responsibility is to get the data onto D7..0 at least 26 ns before the end of IOR′; that's pretty relaxed timing, since it has known that data is being requested from it for at least 504 ns. With typical gate-logic propagation times of 10 ns, 500 ns looks like forever (an indication of its birthdate in 1981, an era of more relaxed timing).

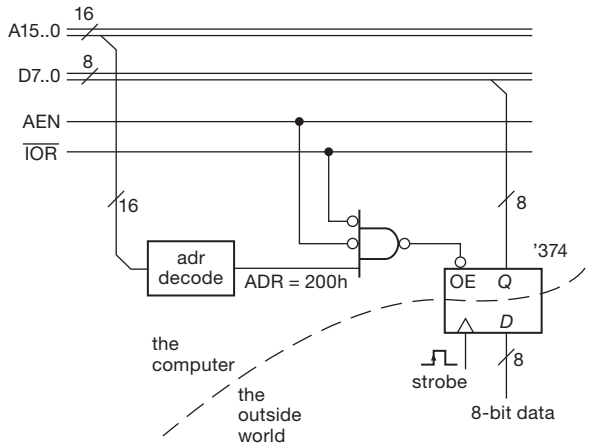



**Figure 14.14.** Programmed data in: an external device asserts its data on the data lines during the IOR′ pulse if the ADDRESS lines hold the peripheral's assigned address. The '374 is an octal *D*-register with three-state outputs.

#### A. Bus signals: bidirectional versus one-way

From the two examples we've done so far, you can see that some bus lines are *bidirectional*, for example the DATA lines: they are asserted by the CPU during write, but asserted by the peripheral during read. Both CPU and peripheral use three-state drivers for these lines. Others, like IOW′, IOR′, and the ADDRESS lines, are always driven by the CPU with standard two-state (active pullup) driver chips. It is typical of data buses to have both kinds of lines, using bidirectional lines for data that goes both ways and one-way lines for signals that are always generated by the CPU (or, more accurately, generated by the associated bus control logic). There is always a precise protocol, like our rules for asserting or reading according to IOW′, IOR′, and ADDRESS, to prevent "bus contention" on these shared lines.

Of the PC104 bus signals so far, only the DATA lines are bidirectional; the ADDRESS lines, AEN, and READ/WRITE

[21] In a more sophisticated implementation you might generate *software* $z$-axis pulses: you would assign a port address (say 3FDh) with a flip-flop to which you would write a HIGH bit followed, after a timed delay, by a LOW bit; the pulse width and delay would be created with CPU timers. Alternatively, you could decorate our circuit with a programmable hardware $z$-axis pulse generator, using a port address with a byte-wide register so that software could instruct the hardware to set the delay and width.

[22] It is for this very reason – this common use of shared (bused) data lines – that many chips include three-state outputs, controlled by an output enable (OE′) control pin.

**Program 14.3**

```
          ;routine to drive 16-bit xy DAC port
INIT:   MOV  SI,xpoint   ;initialize x pointer
        MOV  DI,ypoint   ;initialize y pointer
        MOV  CX,npoint   ;initialize counter

RASTER: MOV  AX,[SI]  ;get x word (2 bytes)
        OUT  3F8H,AX  ;send out (2 byte xfers)
        ADD  SI,2     ;advance x word pointer
        MOV  AX,[DI]  ;get y word (2 bytes)
        OUT  3FAH,AX  ;send out (2 byte xfers)
        ADD  DI,2     ;advance y word pointer
        OUT  3FCH,AL  ;load x and y to DAC
        DEC  CX       ;decrement counter
        JNZ  RASTER   ;not done, send more
        JMP  INIT     ;done, start over}
```

strobes are one-way from the CPU. (Lest we give the wrong impression, we should point out that more complex computer systems permit other riders on the bus to become bus "masters." Obviously in such a system nearly all bus signals must be shared and bidirectional. The PC104/ISA bus is unusually simple.)

### 14.3.5 Programmed I/O: status registers

In our last example, the computer can read a byte from the interface any time it wants to. That's nice, but how does it know when there's something worth reading? In some situations you may want the computer to read data at equally spaced intervals, as determined by its "real-time clock." Perhaps the computer instructs an ADC to begin conversions at regular intervals (via an OUT command), then reads the result a few microseconds later (via an IN command). That might suffice in a data-logging application. However, it is often the case that the external device has a mind of its own, and it would be nice if it could communicate what's happening to the computer without having to wait around.

A classic example is an alphanumeric input keyboard. You don't want characters to get lost; the computer has to get every character, and without much delay. With a fast device like a disk or high-speed serial interface the situation is even more serious; data must be moved at rates up to many megabytes per second without delay. There are actually three ways to handle this general problem: status registers, interrupts, and direct memory access. Let's begin with the simplest method – status registers – illustrated by the keyboard interface in Figure 14.15.

In this example, the raw "ASCII keyboard" is a simple device: when a key is struck, it puts the corresponding byte onto its output data lines (KQ7..0) and generates a short output pulse on STB, as shown.[23] We use the STB (for "strobe") to clock that byte-wide character code into an octal *D*-type register. We rig up the standard programmed data-in circuit, as shown, using the three-state outputs of the octal *D*-register to drive the DATA lines directly. The input labeled KBDATA_SEL′ comes from the usual address decoding circuit of the sort used in the previous examples, and it goes LOW when the particular address chosen for

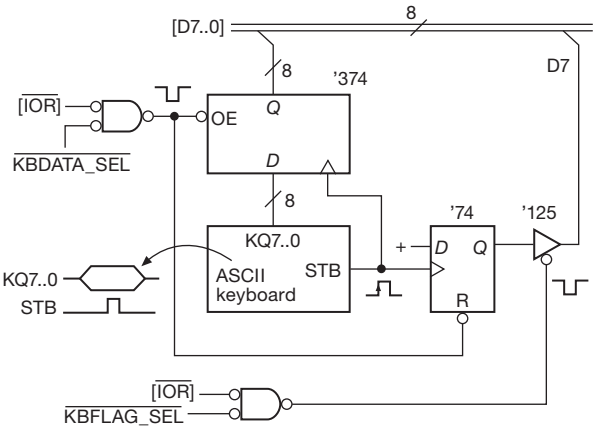



**Figure 14.15.** Keyboard interface with status bit. Brackets indicate PC104 bus signals.

[23] This keyboard lacks the intelligence of contemporary units, which generally include an onboard processor to convert the key codes to a bit-serial format, typically delivered via the USB hardware interface. Its very lack of intelligence, however, permits us to display our own.

this interface appears on the ADDRESS lines of the bus (in combination with AEN disasserted LOW).

What's new in this example is the flip-flop, which gets set when a character is struck, and cleared when a character is read by the computer. It's a 1-bit *status* register: HIGH if there's a new character available, LOW otherwise. The computer can query the status bit by doing a data IN from the other address of this device, decoded (with gates, decoders, or whatever) as KBFLAG_SEL′. You need only one bit to convey the status information, so the interface drives only the most-significant bit (MSB) on the bus (D7), in this case with a '125 three-state buffer. (*Never* drive a bidirectional line with an active-pullup (two-state) output! *Never!*) The line coming into the side of the buffer symbol is the three-state output enable, asserted when LOW, as indicated by the negation bubble.

This circuit could be implemented with standard logic, as indicated. Alternatively, it (and the address decoding logic) would fit easily in a small PLD (for example an XC2C32, XC9536, ATF2500, Mach4032, or even a lowly 22V10).

### A. Program example: keyboard terminal

The computer now has a way to find out when new data is ready. Program 14.4 shows how. This is a routine to get characters from the keyboard, whose data port address is KBDATA (it's good programming style to define the actual numerical port addresses – which correspond to what the hardware decodes as KBDATA_SEL, etc. – in some statements near the beginning of the program, as shown); each character is "echoed" on the computer's display device (port address = OUTBYTE). When it has gotten a whole line, it transfers control to a line-handling routine, which might do just about anything, based on what the line says. When it's ready for another line, it types an asterisk "prompt." This sort of protocol should make sense to you if you've had some experience with computers.

The program begins by initializing the character buffer pointer (BP), by moving the *address* of the buffer that we just allocated to the address register BP. Note we can't just say "`MOV BP,charbuf`" because that would load the *contents* of `charbuf`, not its address; in x86 assembly language you use the word "offset" in front of a memory label to signify its address. The program then reads the keyboard status bit via an IN instruction, ANDs it with 80h to keep only the status bit (this is called "masking"), and tests for zero. Zero means the bit isn't set, so the program loops. When a nonzero status bit is detected, it reads the keyboard data port (which clears the status flag flip-flop), stores it consecutively in the line buffer, increments the pointer (BP), and calls the routine that echoes the character to the screen. Finally, it checks to see if the line was terminated by a carriage return (CR): if it wasn't, it goes back and loops on the keyboard status flag again; if it was a CR, it transfers control to the line handler, after which it prints an asterisk and begins the entire process anew.

A subroutine has been used to display a character, since even that simple operation requires some flag checking and masking. The routine first saves the byte into AH, then reads and masks the screen's busy flag. A nonzero result means the screen is busy, so it keeps checking; otherwise it restores the character to AL, sends it to the screen's data port, and returns.

Some notes on the program. (a) We could have omitted the keyboard flag-masking step, because the MSB (where we put the flag bit in our hardware) is the sign bit; thus we could have used the instruction `JNS KFCHK`. However, this trick works only for testing the MSB and thus is somewhat specialized. (b) In keeping with good programming practice, the carriage return symbol (0Dh) and asterisk probably should be defined constants, similar to KBMASK. (c) The line handler probably should be a subroutine, also. (d) Characters will be lost if the line handler takes too long; this leads us to the more elegant approach of *interrupts*, which we'll take up shortly. (e) Keyboard and terminal handlers are used so often that microprocessor operating systems provide built-in handlers, accessed through "software interrupts" (we'll see them later); thus our program may not even be needed.

### B. Status bits generalized

This keyboard example illustrates status bit protocol; but it's so simple that you may come away with the wrong idea. In an actual peripheral interface of some complexity, there will usually be several flags to signal various conditions. For example, in an Ethernet interface you will normally have individual status bits indicating a successful packet transmission, or (if you're not so lucky) any of various mishaps that may have occurred. As a specific example, Microchip's ENC28J60 Ethernet Controller IC has a 56-bit transmit status register; bit 20 indicates a "Transmit CRC Error," described as "The attached CRC in the packet did not match the internally generated CRC."[24]

For peripherals of modest complexity, the usual procedure is to put all the status bits into one byte or word, so that a data IN command from the status register gets all bits at once. Typically you would have a bit indicating any of a set

[24] These things can get humorously arcane: bit 27 ("Transmit Excessive Defer") signifies "Packet was deferred in excess of 24,287 bit times."

**Program 14.4**

```
          ;keyboard handler -- uses flags
KBDATA  equ ***H        ;put kbd data port adr here
KBFLAG  equ ***H        ;a different port for kbd flag
KBMASK  equ 80H         ;kbd flag mask
OUTBYTE equ ***H        ;put disp port adr here
OUTFLAG equ ***H        ;another for disp port flag
OUTMASK equ ***H        ;disp port busy mask

charbuf DB 100 dup(0)  ;allocates buffer of 100 bytes

INIT:   MOV  BP,offset charbuf   ;init char buf pntr
KFCHK:  IN   AL,KBFLAG  ;read kbd flag
        AND  AL,KBMASK  ;mask unused bits
        JZ   KFCHK      ;flag not set -- no new data
        IN   AL,KBDATA  ;flag set -- get new kbd byte
        MOV  [BP],AL    ;store it in line buffer
        INC  BP         ;and advance pointer
        CALL TYPEIT     ;echo last char to display
        CMP  AL,0DH     ;was it carriage return (0Dh)?
        JNZ  KFCHK      ;if not, get next char
LINE:     o             ;if so, do something with line
          o             ;keep at it
          o             ;don't quit now
          o             ;done at last!
        MOV  AL,'*'
        CALL TYPEIT     ;type a "prompt" -- asterisk
        JMP  INIT       ;get another line

                        ;routine to type character
                        ;types and preserves AL
TYPEIT: MOV  AH,AL      ;save the char in AH
PCHK:   IN   AL,OUTFLAG ;check printer busy?
        AND  AL,OUTMASK ;printer flag mask
        JNZ  PCHK       ;if busy check again
        MOV  AL,AH      ;restore char to AL
        OUT  OUTBYTE,AL ;type it
        RET             ;and return
```

of error conditions as the MSB of the status word, so a simple check of sign tells if there are *any* errors; if there are, you test specific bits of the word (by ANDing with masks) to find out what's wrong. Furthermore, in a complex interface you probably wouldn't have the status bits reset "automatically," as we did with our single bit; instead, a data OUT statement might be used, each bit of which clears a specific flag.

**Exercise 14.1.** With our keyboard interface there is no way for the computer to know if it missed a character. Modify the circuit so there are two status bits: CHAR_READY (that's what we have already) and LOST_DATA. The LOST_DATA flag should be readable as D6 on the same status port as CHAR_READY; it is 1 if a key was struck before the previous character was fetched by the computer, 0 otherwise.

**Exercise 14.2.** Add a section of code to Program 14.4 to check for lost data. It should call a subroutine called LOST if it detects lost data; otherwise the program should work as before.

### 14.3.6 Programmed I/O: command registers

To summarize, a status bit (or a collection of bits: a status register) reports a condition to the CPU (when asked). Going in the opposite direction, the CPU can send out a bit (or

collection of bits) to a peripheral, to tell it to do something. This is called a *command* bit (or command register). A simple example might be a bit that tells an *xy* positioning stage to move to the coordinates that have been deposited in a pair of data registers of the peripheral by a pair of previous programmed OUT statements. Or, taking the example of our Ethernet controller, the CPU deposits to a pair of registers in the IC the beginning and ending addresses of the data packet to be transmitted; then it sets a particular command bit (bit 3, to be exact) in the chip's "ECON1" register, which opens the floodgates by commanding the chip to send the packet out on the Ethernet port. The chip does as it has been told (in this case using DMA for highest speed; see §14.3.10), and reports back to the CPU (when asked; or more aggressively by using an *interrupt*; see next subsection) via the status registers.

### 14.3.7 Interrupts

The use of status flags just illustrated is one of three ways for a peripheral device to "tell" the computer when some action needs to be taken. Although it will suffice in many simple situations, it has the serious drawback that the peripheral cannot announce that some action needs to be taken – it has to wait to be asked by the CPU, via a data IN command from its status register. Devices that need quick action (such as disks or latency-sensitive real-time I/O) would have to have their status flags queried often, and with several such devices in a computer system the CPU would soon find itself spending most of its time checking status flags, as in the last example.

Furthermore, even with continual status flag checking you can still get in trouble: in the last example, for instance, the CPU will have no trouble keeping up with someone typing at the keyboard when it is in the main (flag-checking) loop. But what if it spends 1/10 second in the line-handling portion? Or what if the interfaced device is a slow one, making the program wait for its busy flag to clear?

What is needed is a mechanism for a peripheral to *interrupt* the normal action of the CPU when something needs to be done. The CPU can then check the status register to find out what the trouble is, take care of what needs to be done, and go back to its normal business.

To add interrupt capability to a computer, it is necessary to add a few new bus signals: at least one shared line for peripherals to signal an interrupt, and (usually) a pair of lines by which the CPU can determine who interrupted. As luck would have it, the PC104/ISA is not a very instructive example, because it does not implement a full interrupt capability. What it lacks in power, though, it more than makes up for in simplicity; implementing hardware interrupts in a PC104 peripheral interface is like falling off a log.

Here's how it works: the PC104 bus has a set of six *interrupt request* lines, called IRQ3–IRQ7 and IRQ9. They are active-HIGH inputs to the CPU's support circuitry. To make an interrupt, you simply bring one of the lines HIGH. If interrupts are enabled in general (along with the particular IRQ you assert), the CPU will break off after its next instruction, then (after saving its flags and current location onto the stack) jump to an "interrupt-handler" program somewhere in memory. You write the handler to do what you want (e.g., get keyboard data), and you can put the handler anywhere you wish, because the CPU figures out where to jump by looking for the handler's 4-byte address in a special ("vector") location in low memory. That location depends on which IRQ you've asserted; for the x86 it is given in hex by 20+4*n*, where *n* is the interrupt level. For example, the CPU would respond to an interrupt on IRQ2 by jumping to the (4-byte) address stored in locations 28h through 2Bh (it's just like indirect addressing, except that the address is found in memory rather than in a register); of course, you would have cleverly arranged for the starting address of your handler to be there. At the end of your handler you execute an IRET instruction, which causes the CPU to restore the preexisting flag register and jump back to wherever it was when the interrupt happened.

#### A. Example: keyboard interface with interrupts

Let's illustrate by adding interrupts (Figure 14.16) to our keyboard interface circuit of Figure 14.15. We've kept the flag bit ("character ready") and programmed I/O circuitry essentially as before, except that we've ORed the flag clear with a new bus line, RESET, a bus signal that is momentarily asserted HIGH when the computer is turned on. This signal is generally used to force your flip-flops and other sequential logic into a known state at power-up. Obviously it should reset a flag that indicates a valid byte is ready to be claimed (and that, in our new interface, will further cause an interrupt).

The new interrupt circuitry consists of a three-state driver to assert IRQ3 when a character is ready. That's all the new hardware you need. Although not strictly necessary, we've added the capability to disable the interrupt driver buffer by sending a data byte with D0 LOW to the KBFLAG port address. This would be used if you wanted to plug in another peripheral with interrupts at the same IRQ level, allowing only one peripheral to use its interrupts at any given time (later we'll have further explanation on this awkward point).

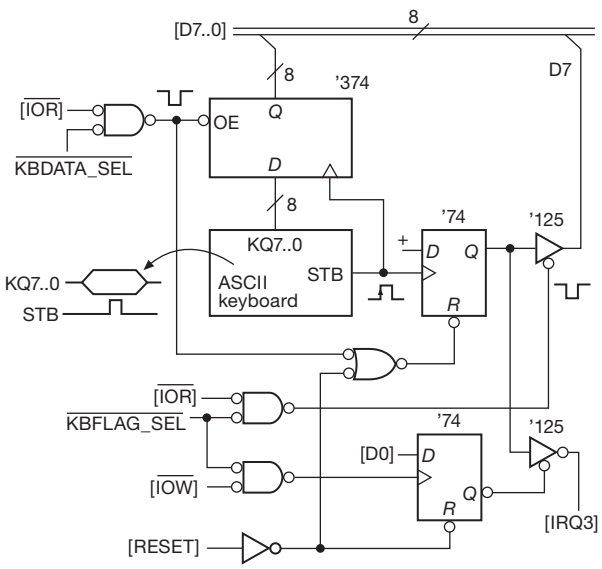



**Figure 14.16.** Keyboard interface with status bit and interrupt.

### 14.3.8 Interrupt handling

The PC104 bus signals, derived from the original IBM PC, make interrupt handling particularly easy, though limited in flexibility compared with the general (and more sophisticated) method described below (§14.3.9). The bus signals require, in addition to the CPU itself, some interrupt control circuitry on the motherboard.[25] This "support-chip" circuitry does most of the hard work, which consists of prioritizing, masking, and asserting vectors (we'll describe these after finishing the example). The CPU, for its part, recognizes the interrupt and responds by saving the instruction pointer and flag register, disabling further interrupts, then making a jump via the corresponding address stored in the low-memory vector area. Your handler program does the rest, namely: (a) saves (pushes) any registers you'll be using (remember that the interrupted program can't prepare for the interrupt, since it can happen anywhere in the running program – it's a bolt out of the blue); (b) figures out what needs to be done, by reading status registers if necessary; (c) does it; (d) restores the saved registers from the stack; (e) tells the interrupt control circuitry that you're done (by sending an "end-of-interrupt" byte 20h to its register at I/O address 20h); and, finally, (f) executes a return from interrupt instruction `IRET`; this causes the CPU to restore the old flag register that it saved on the stack and to jump (via the old instruction pointer it saved on the stack) back to the program that was interrupted. Somewhere in the program, you must have (g) loaded the handler's address into the vector location corresponding to the IRQ level used by the hardware and told the interrupt control hardware to enable interrupts at that level.

Program 14.5 shows the code for the keyboard with interrupt. Here's the overall scheme: the main program sets things up, then loops on a flag (in memory, not hardware) that the interrupt handler sets when it recognizes a carriage return; when the main program sees that flag set, it goes off and does something with the line, then returns to the flag-checking loop. The handler, entered at each interrupt, puts a character into the line buffer, sets the flag if it was a carriage return, then returns.

Let's look at the program in some detail. After defining port addresses and the all-critical vector location for IRQ3, it allocates 100 bytes (initially filled with zeros) for the character buffer. The actual program execution begins by putting the buffer address in address register SI, zero in the end-of-line flag, and the address of the handler (which begins with `KBINT`) in location 2Ch. To enable level-3 interrupts in the interrupt controller, we clear bit 2 of its existing mask (`IN`, `AND`, `OUT`); then we enable CPU interrupts and send a 1 to `KBFLAG`, which enables the three-state driver. Now we're running. The program then loops, with interrupts secretly happening right under the main program's nose, until it mysteriously finds the "buflg" set. It resets the pointer and flag immediately (in case another interrupt occurs soon), then gobbles up the line. It would be well advised either to move quickly or to copy the line to another buffer, since another interrupt (with a new byte to go in the buffer) could come along in a few milliseconds; in that time you can execute a few hundred thousand instructions, however, more than enough to copy the line.

The interrupt handler is a separate little piece of code, with no entry from the main program. It gets entered upon a level-3 interrupt, via its address that we initially loaded into 2Ch. It knows exactly what it has to do, and it does it without complaining: it saves `AX` (because it plans to clobber it), reads the character from the keyboard data port, puts it in the buffer, increments the pointer, echoes the character to the screen, sets the flag if it was a carriage return, sends end-of-interrupt to the interrupt controller, restores `AX`, and returns.

If you look back at our list of handler tasks above, you'll see that we omitted just one step, namely, reading status flags to figure out which of several actions needed to be done. That's unnecessary here, though, because there's only one reason to interrupt, namely that a new keyboard character needs to be read. (The programmer obviously has

[25] Which was done by an 8259 interrupt-controller IC on the original PC motherboard.

**Program 14.5**

```
          ;keyboard handler -- uses interrupts
KBVECT equ word pntr 002CH   ;INT3 vector
KBDATA equ ***H        ;put kbd data port adr here
KBFLAG equ ***H        ;put kbd flag port adr here
buflg    DB   0        ;allocates "end-of-line" flag
bufpos   DW   0        ;allocates buffer pointer
charbuf  DB 100 dup(0) ;allocates 100-byte char buf

        CLI ;disable interrupts
        MOV  bufpos,offset charbuf   ;initialize buf pntr
        MOV  buflg,0   ;clear end-of-line flag
        MOV  KBVECT,offset KBINT ;hndlr adr->vec area
        STI            ;enable interrupts
        MOV  AL,1
        OUT  KBFLAG,AL ;enable hardware 3-state drvr
PROMPT: MOV  AL,'*'
        CALL TYPE      ;type prompt "*"
        IN   AL,21H    ;existing int ctrl mask
        AND  AL,0F7H   ;clear bit 3 to enable INT3
        OUT  21H,AL    ;and send back to intctrl OCW1
LNCHK:  MOV  AL,buflg
        OR   AL,AL     ;needed to set zero flag
        JZ   LNCHK     ;loop til end-of-line flag set
LINE:   MOV  bufpos,offset charbuf   ;reset pointer
        MOV  buflg,0   ;clear line flag
          o            ;do something with line
          o
          o
        JMP  PROMPT     ;and wait for another line

; *** this ends the main program. ***
; *** the code below is completely independent ***
          ;keyboard interrupt handler
          ;an INT3 lands you here, via vect we loaded
KBINT:  STI            ;enable interrupts
        PUSH AX        ;save AX register, used here
        PUSH SI        ;save SI, possible other users
        MOV  SI,bufpos ;and copy buffer pointer there
        IN   AL,KBDATA ;get data byte from keyboard
        MOV  [SI],AL   ;put it in line buffer
        INC  SI        ;and advance pointer
        MOV  bufpos,SI ;and copy to bufpos
        CALL TYPE      ;echo to screen
        CMP  AL,0DH    ;check for carriage return
        JNZ  HOME      ;not a CR -- return
        MOV  buflg,0FFH  ;CR -- set end-of-line flag
        IN   AL,21H    ;existing int ctrl mask
        OR   AL,08H    ;set bit 3 to disable INT3
        OUT  21H,AL    ;and send back to intctrl OCW1
HOME:   MOV  AL,20H
        OUT  20H,AL    ;end-of-int signal to int ctlr
        POP SI         ;restore SI
        POP  AX        ;restore old AX
        IRET           ;and return
```

to understand under what conditions the hardware makes an interrupt, and what is required to service the interrupt.)

A few notes on this program: first, even though we're using interrupts, the program seems as dumb as before – it loops continually on the end-of-line flag. However, it could be doing other things, if there were things to do. In fact, it does just that beginning at statement LINE, where it processes the finished line; during that time, interrupts make sure that new characters are put into the buffer, whereas they would have been lost in our previous example without interrupts.

This brings up a second point, namely, even with interrupts we're still in trouble if the program is doing things with the previous line when the next line has been completely entered. Of course, *on the average* the program simply has to keep up with keyboard entry; but you could have a situation in which the line user occasionally spends a lot of time, and you need to buffer more than one line temporarily. One solution to this is to make a copy to a second buffer or to alternate between two buffers. An elegant alternative is to organize input as a queue, implemented as a "ring buffer" (or "circular buffer"), in which a pair of pointers keeps track of where the next input character goes, and from where the next character is removed. The interrupt handler advances the input pointer, and the line user advances the output pointer. Such a ring buffer might typically be 256 bytes long, permitting the line handler to get behind by a few lines.

A third point concerns the interrupt handler itself. It's usually best to keep it short and simple, perhaps setting flags to signal the need for complicated operations in the main program. If the handler does become long-winded, you risk losing data from other interrupting devices because interrupts are disabled (at that level and below) when the CPU jumps into the handler. The solution in this case is to re-enable interrupts *within* your handler with an STI instruction, after doing the critical things that have to be done first. Then, if an interrupt occurs, your interrupt handler will itself be interrupted. Because flags and return addresses are stored on the stack, the program will find its way back, first to your handler, finally to the main program.

### 14.3.9 Interrupts in general

Our keyboard example illustrates the essence of interrupts – a spontaneous hardware request for attention by a peripheral, producing a program jump to a dedicated handler routine (usually resulting in some programmed I/O), followed by a return to the code that was interrupted. Other examples of interrupting devices are real-time clocks, in which a periodic interrupt (often 10 per second, but 18.2 per second in the original PC from which the PC104 bus derived) signals a timekeeping routine to advance the current time; another example is a printer port, which interrupts each time it is ready for a new character. By using interrupts, these peripherals let the computer interleave other tasks simultaneously; that's why you can be typing while your PC is printing a file (and, of course, keeping proper time throughout).

The PC104/ISA bus does not, however, illustrate the full generality of interrupts. As we saw, it has a set of six IRQ lines on the 8-bit bus (there are five more on the 16-bit extension), each one of which can be used only for a single interrupting device. The IRQ lines are numbered according to priority; in the event of multiple interrupts, the lowest-numbered interrupt is serviced first. And several of the IRQ lines are preassigned to essential peripherals, leaving precious few available. Furthermore, the interrupt is *edge-triggered*, which frustrates any reasonable possibility of using wired-OR to combine several peripherals on a single IRQ line. Evidently the original IBM PC designers, not anticipating the need for shared interrupts, designed a far-from-optimal interrupt scheme.

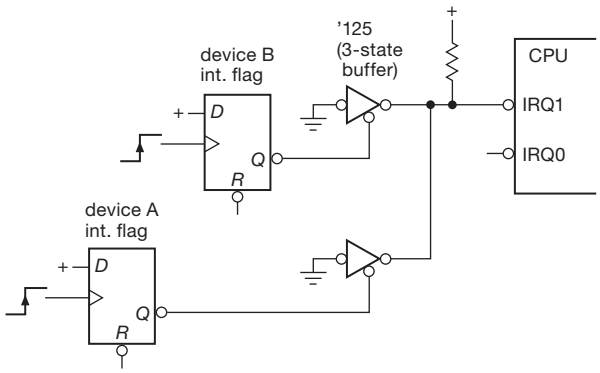



**Figure 14.17.** Sharing a level-sensitive interrupt line via wired-OR.

#### A. Shared interrupt lines

The usual interrupt protocol, as implemented on many microcomputers, circumvents these limitations. Look at Figure 14.17. There are several (prioritized) IRQ-type lines; these are active-LOW inputs to the CPU (or its immediate support circuitry). To request an interrupt, you pull one of the IRQ′ lines LOW, using an open-collector (or three-state) gate, as shown (note the trick for using a three-state gate to mimic an open-collector gate). The IRQ′ lines are shared, with a single resistive pullup, so you can put as many devices on each IRQ′ line as you want; in our example, two

ports share IRQ1. You would generally connect a latency-sensitive (impatient) device to a higher-priority IRQ′ line.

Because the IRQ′ lines are shared, there could always be another device interrupting on the same line at the same time. The CPU needs to know who interrupted so it can jump to the appropriate handler. There is a simple way, and a complicated way, to do this. The simple way is called *autovectored polling* and is used nearly universally (though not on the PC104 bus). Here's how it works.

### Autovectored polling

Some circuitry on the CPU board instructs the microprocessor that it is to use autovectoring, which works just like the PC104 – each level of interrupt forces a jump through a corresponding vectoring location in low memory. You put the addresses of the handlers in those locations, just as in our previous example.

Once in the handler, you know which level of interrupt you're servicing; you just don't know which particular device caused the interrupt. To find out, you simply check the status registers of each of the devices connected to that level of interrupt (a device never requests an interrupt without also indicating its need by setting one or more readable status bits). If a bit is set indicating that something needs to be done, you do it, including whatever it takes to cause the device to disassert its IRQ′: some devices (like our keyboard) clear their interrupt when read, whereas others may need a particular byte sent to some I/O port address.

If the device you serviced was the only one interrupting at that level, that IRQ′ will now be HIGH on returning to the interrupted program, and execution will continue. However, if there had been a second interrupting device at the same level, that IRQ′ line would still be held LOW (by the wired-OR action of the shared IRQ′ line) on return from the service routine, so the CPU would immediately autovector back to the same handler. This time the polling would find the other interrupting device, do its thing, and return. Note that the order in which you poll status registers effectively sets up a "software priority," in addition to the hardware priority of the multiple IRQ′ levels.

### Interrupt acknowledgment

We shouldn't leave the subject of interrupts without mentioning a more sophisticated procedure for identifying who interrupted – *interrupt acknowledgment*. In this method, the CPU doesn't need to poll the status registers of possible interrupters because the interrupting device *tells* the CPU its name, when asked. The interrupter does this by putting an "interrupt vector" (usually a unique 8-bit quantity) onto the DATA lines in response to an "interrupt-acknowledge" signal that the CPU generates during the interrupt processing.

Nearly every microprocessor generates the needed signals. The sequence of events goes like this.

(a) The CPU notices a pending interrupt.
(b) The CPU finishes the current instruction, then asserts (i) bus signals that announce an interrupt, (ii) the interrupt level being serviced (on the low-order ADDRESS lines), and (iii) READ-like strobes that invite the interrupting device to identify itself.
(c) The interrupting device responds to this bus activity by asserting its identity (interrupt vector) onto the DATA lines.
(d) The CPU reads the vector and jumps into the corresponding unique handler for the interrupting device.
(e) The handler software, as in our last example, reads flags, gets and sends data, etc., as needed; among its other duties, it must make sure the interrupting device disasserts its interrupt.
(f) Finally, the interrupt handler software returns control to the program that was interrupted.

Sharp-eyed readers may have noticed a flaw in the procedure just outlined. In particular, there has to be a protocol to ensure that only one device asserts its vector, since there may be several simultaneous interrupting devices at the same IRQ level. The usual way to handle this is to have a bus signal (call it INTP, "interrupt priority") that is unusual in not being shared by devices on the bus, but rather is passed along *through* each device's interface circuit, beginning as a HIGH level at the device closest to the CPU and threading along through each interface. That's called a "daisy chain" in the colorful language of electronics. The rule for INTP hardware logic is as follows: if you have not requested an interrupt at the level being acknowledged, pass INTP through to the next device unchanged; if you *have* interrupted at that level, hold your INTP output LOW. Now the rule for asserting your vector goes like this: put your vector number onto the data bus when requested by the CPU only if (a) you have an interrupt pending at the level being acknowledged, and (b) your input INTP is HIGH. This guarantees that only one device asserts its vector; it also establishes a "serial-priority" chain within each IRQ level, with devices electrically closest to the CPU getting serviced first.

There is a nice alternative to the serial daisy-chain method of interrupt acknowledgment: instead of threading a line through each possible interrupter, you bring individual lines back from each one to a priority encoder (§10.3.3E), which in turn acknowledges the interrupt by

asserting the identity of the highest-priority interrupting device. This scheme avoids the nuisance of daisy-chain jumpers.

In most microcomputer systems it isn't worth implementing the full-blown interrupt acknowledgment just described. After all, with 8-level autovectoring you can handle up to eight interrupting devices without polling, and several times that number with polling. Only in large computer systems, in which you demand fast response with dozens of interrupting devices present, might you succumb to the complexity of the interrupt-acknowledgment protocol. However, it is important to realize that even simple computers may be using vectored interrupt acknowledgment *internally*. For example, the simple 6-level autovector interrupt scheme of the PC104 is actually generated by an interrupt controller support chip that lives close to the CPU and generates the proper interrupt acknowledgment sequence just described (see below).

#### B. Interrupt masks

We put a flip-flop in our simple keyboard example so that its interrupts could be disabled, even though the interrupt controller chip itself lets you turn off ("mask") each level of interrupt individually. We did that so that some other device could then use IRQ3. For a bus with shared (*level*-sensitive) IRQ′ lines, it is especially important to make each interrupt source maskable, using an I/O output port bit. For example, a printer port normally interrupts each time its output buffer is empty ("give me more data"); when you've finished printing, though, you don't care. The obvious solution is to turn off printer interrupts. Since there might be other devices hooked to the same interrupt level, you must not mask that whole level; instead, you just send a bit to the printer port to disable its interrupts.

#### C. How the PC104 ISA interrupts got the way they are

The ISA bus was created by the IBM PC's designers, and adopted without change (but with a different connector) by the PC104 consortium. The 8086/8 microprocessor used in the original IBM PC actually implemented the full vectored interrupt-acknowledgment protocol. To keep things simple, however, the PC designers used an 8259 interrupt controller IC on the motherboard. The way it was used in the PC, it had a set of IRQ inputs from the I/O bus card slots (that's where you make your interrupt requests), and it connected to the microprocessor's data bus and signal lines. When it received a request on an IRQ line from a peripheral, it figured out priority and went through the whole business of asserting the corresponding vector onto the data bus. It had a mask register (accessible as I/O port 21h) so that you could disable any specified group of interrupts.

The 8259 let you select (through software) either *level*- or *edge*-triggered interrupts on its IRQ input lines, according to a byte sent to a control register (I/O port 20h). Unfortunately, the PC designers decided to use edge triggering, probably because that makes it a little easier to implement interrupts (for example, you can just connect the real-time clock's square-wave output directly to IRQ0). If they had selected level-sensitive interrupts instead, you would be able to hang multiple interrupting devices on each IRQ′ line, with software polling as described above.

There is a partial solution to this problem. As long as there is an IRQ line available, you *can* combine several interrupting devices on a single PC board, with logic to generate edge-triggered interrupts on that single IRQ line. But, since the interrupting devices have to know about each other, you can't use this scheme for independent plug-in peripherals. Furthermore, you still use up an IRQ line per card, so in a complicated system there will not be enough to go around.

#### D. Software interrupts

The Intel x86-series of CPUs have an instruction ("`INT n`," where `n` is 0–255) that allows you to produce the same kind of vectored jump as an actual hardware interrupt. In fact, among its 256 possible jump vectors are duplicates of the eight levels of IRQ-requested hardware interrupts (`INT 8` through `INT 15`, to be exact). Thus, you can make a "software interrupt" from a program statement.

Don't confuse this with the externally triggered hardware interrupts we've been talking about. Software interrupts turn out to be a handy way of implementing vectored jumps from user code into system software. But they are not real interrupts, in the sense of a hardware call for attention from an external autonomous device. On the contrary, you build these into your software, you know when they are coming (that's why you can pass arguments through registers), and they are merely the response (albeit identical with what follows a true interrupt) of the CPU to its own code. You might think of software interrupts as a clever way to extend the instruction set.

### 14.3.10 Direct memory access

There are situations in which data must be moved very rapidly to or from a device. The classic examples are fast mass-storage devices like magnetic and optical disk drives, and network connections. Interrupt-initiated programmed processing of each data transfer in these examples would

be both awkward and, worse, too slow. For example, data comes from a typical magnetic disk drive at sustained rates up to 500MB per second. With the bookkeeping involved in handling an interrupt, the situation would be hopeless, even if the disk were the only interrupting device in the system. Devices like disks and tapes (not to mention real-time signals and data) can't stop in midstream, so a method must be provided for reliably fast response and high overall byte transfer rates. Even with peripherals with low average data transfer rates, there are sometimes requirements for short *latency*, the time from initial request to actual movement of data.

The solution to these problems is direct memory access (DMA), a method for direct communication from peripheral to memory. In some microcomputers the communication is actually handled by the CPU hardware, but that doesn't really matter. The important point is that no programming is involved in the actual transfer of data; bytes are moved between memory and peripheral via the bus, without program intervention. The only effect on the executing program is some slowing down of execution time, because DMA activity "steals" bus cycles that would otherwise be used to access memory for program execution. DMA usually involves more hardware complexity in the interface itself, and is not generally used unless necessary. However, it is useful to know what can be done, so we describe briefly what you need to make a DMA interface. As with interrupts, the PC104/ISA bus uses a streamlined DMA protocol, with a "DMA controller" chip on the motherboard doing the hard work for you, thus making a DMA interface relatively straightforward. We first explain the more usual "bus mastership" method of DMA, then the PC104's simplified DMA protocol.

### A. A typical DMA protocol

In DMA transfers, the peripheral requests access to the bus via special "bus request" lines (prioritized like the IRQ lines) that are part of the bus. The CPU gives permission and releases control of address, data, and strobing lines. The peripheral then asserts memory addresses onto the bus and either sends or receives data, one byte at a time, according to the strobes it asserts; in other words, it takes over the bus (it becomes "bus master") and acts like a CPU, directing data transfers. The DMA bus master is responsible for generating addresses (usually a block of successive addresses, generated with a binary counter) and keeping track of the number of bytes moved. The usual way to do this is to have a byte counter and an address counter in the interface. These are initially loaded from the CPU, via programmed I/O, to set up the DMA transfer desired. On command from the CPU (via a command bit, written with programmed I/O), the interface makes its DMA request and begins to move its data. It may release the bus between each byte (allowing the CPU to sneak a few instructions in), or it may take the more antisocial approach of keeping the bus for a block of transfers. When all transfers are complete, it releases the bus for the last time and notifies the program that it is finished by setting a status bit and requesting an interrupt, whereupon the CPU can decide what to do next.

Getting data or programs from disk storage is a common example of DMA transfer: the executing program asks for some "file" by name; the operating system software translates this into a set of programmed data OUT commands to the disk interface's control (or "command") register, byte count register, and address register (specifying where to go on the disk, how many bytes to read, and where to put them in memory). Then the disk interface finds the right place on the disk, makes a DMA request, and begins moving blocks of data to the specified place in memory. When it's done, it sets bits in its status register to signify completion and then makes an interrupt. The CPU, which has meanwhile been executing other instructions (or possibly just waiting for data from the disk), responds to the interrupt, finds out from the status register of the disk interface that the data are now in memory, and then goes on to the next task. Thus, programmed I/O to the interface (the simplest kind of I/O) was used to set up the DMA transfer, DMA itself (stealing bus cycles from the CPU) was used for rapid transfer of data, and an interrupt was used to let the computer know the task was done. This sort of I/O hierarchy is extremely common, especially with mass-storage devices; you can expect maximum DMA transfer rates of hundreds of megabytes per second on a contemporary microcomputer bus like PCI express.

### B. DMA on the PC104/ISA bus

The PC104/ISA bus, the descendant of an earlier (and simpler) time, has a simpler DMA protocol. A support chip that accompanies the microprocessor includes a DMA controller (an 8237 on the original IBM PC), with built-in address and byte counters, along with the logic to disable the CPU and take over the bus. A peripheral that wants to do DMA, therefore, doesn't have to generate addresses and drive the bus. Instead, it signals the controller (via one of the three DRQ1–DRQ3 "DMA request" lines), which in turn responds by returning the corresponding DACK0–3′ ("DMA acknowledge"). The controller then controls the transfers, asserting address and strobing lines, with the peripheral asserting (or receiving) data to (or from)

memory. In this whole process the memory sees nothing unusual going on, since addresses and memory strobes (MEMW′ or MEMR′), normally supplied by the CPU, are supplied by the DMA controller, and if it's DMA *to* memory, data are supplied by the peripheral. The peripheral, on the other hand, knows something special is happening because it requested DMA access (and received confirmation via DACK′); so when the DMA controller asserts IOR′ (or IOW′), the peripheral supplies (or accepts) successive bytes. You might wonder why some innocent bystander peripheral doesn't get hurt in the DMA process, since both I/O strobes and addresses are being asserted, whereas the addresses are in fact the *memory* addresses that go with the memory strobes MEMW′ or MEMR′ asserted by the controller; they have nothing to do with I/O port addresses. The secret is our old friend AEN, specifically added to the bus just to solve this problem. AEN is asserted HIGH during DMA transfers, and all I/O port addressing must be qualified by ANDing with AEN LOW to prevent spurious responses to DMA memory addresses.

Even with the use of a separate controller chip, you still have to set up the starting address, byte count, and direction for the impending DMA transfer. These data go to the DMA controller, which is obliging, having a set of registers that you write (via programmed I/O) from the CPU. It's all pretty straightforward, except for some complexity created by a proliferation of "modes" (single transfer, block transfer, etc.). The PC104/ISA bus has a rather modest DMA capacity, about 2 $\mu$s per byte transferred; and, as with interrupts, the PC104 bus is sparse on DMA channels.

### 14.3.11 Summary of PC104/ISA 8-bit bus signals

Through our examples – programmed I/O, interrupts, and DMA – we've seen most of the PC104 bus signals, which make a "multidrop" tour through the stacked peripherals (Figure 14.18). Table 14.2 lists the full bus, with pin connections. For completeness we summarize them all here, beginning with the ones we've already met.

**A19–A0: Address lines** Three-state, out from bus master, active-HIGH. All 20 lines are used to address memory (with MEMR′ and MEMW′ as strobes, analogous to IOR′ and IOW′), but only the 16 least significant lines are used during I/O access (64K port addresses); I/O devices should qualify address with AEN LOW.

**D0–D7: Data lines** Three-state, bidirectional, active-HIGH. Asserted by CPU during memory or I/O write; asserted by memory during memory read or DMA from memory; asserted by I/O port during I/O read or DMA to memory.

**IOR′, IOW′, MEMR′, MEMW′: Data strobes** Three-state, output only, active-LOW. Asserted by bus master during read or write. On *writes*, data should be latched on trailing (rising) edge, qualified by address; on *reads*, data should be asserted during the strobe (and ready before the trailing edge), qualified by address.

**AEN: Address enable** Two-state, output only, active-HIGH. Asserted by CPU during DMA cycles. I/O ports must not respond with normal address decoding to IOR′ and IOW′; instead, I/O port that received DACK′ looks at IOR′ or IOW′ to put or take DMA data bytes to or from DATA lines.

**IRQ2–IRQ7: Interrupt request** Two-state, input only, rising-edge-triggered. Asserted by interrupting device. Prioritized, with IRQ2 highest, IRQ7 lowest. Maskable in the interrupt controller, via CPU write to port 21h. Each IRQ level can be used by only one device at a time.

**RESET: Reset driver** Two-state, output only, active-HIGH. Asserted by CPU during power-on. Used to initialize I/O devices to known startup state.

**DRQ1–DRQ3: DMA request** Two-state, input only, active-HIGH. Asserted by I/O device requesting DMA channel. Prioritized, with DRQ1 highest, DRQ3 lowest. Acknowledged by DACK1′-DACK3′.

**DACK0′–DACK3′: DMA acknowledge** Two-state, output only, active-LOW. Asserted by CPU (DMA controller) to indicate grant of corresponding DMA request.

**ALE: Address latch enable** Two-state, output only, active-HIGH. The 8088 used a multiplexed data/address bus, and this signal corresponds to the 8088's strobing signal, used by latches on the motherboard to latch the address. Can be used to signal beginning of a CPU cycle; usually ignored in I/O design.

**CLK: Clock** Two-state, output only. This is the CPU's clocking signal; it's asymmetrical, 1/3 HIGH and 2/3 LOW. The original PC used a 4.77 MHz clock, but higher speeds are common. CLK is used to synchronize wait-state requests (via IOCHRDY), in order to stretch an I/O cycle for slow devices.

**OSC: Oscillator** Two-state, output only. This is a 14.31818 MHz square wave, which can be used (when divided by 4) as a color-burst oscillator for color display.

**TC: Terminal count** Two-state, output only, active-

**Table 14.2 PC104/ISA 8-bit Bus Signals**

| Signal | Qty | Active | Type[a] | Direction CPU[b] ↔ I/O | Pin # | Function |
|---|---|---|---|---|---|---|
| **A[19..0]** | 20 | H | 3S | → | A12..21 | address (A15..0 for I/O) |
| **D[7..0]** | 8 | H | 3S | ↔ | A2..9 | data |
| **IOR#** | 1 | L | 3S | → | B14 | I/O read strobe |
| **IOW#** | 1 | L | 3S | → | B13 | I/O write strobe |
| **MEMR#** | 1 | L | 3S | → | B12 | memory read strobe |
| **MEMW#** | 1 | L | 3S | → | B11 | memory write strobe |
| **AEN** | 1 | H | 2S | → | A11 | DMA address signal |
| **IRQ[7..2]** | 6 | ↑ | 2S | ← | B21..25, B4 | interrupt request |
| **RESET** | 1 | H | 2S | → | B2 | power-on reset |
| **DRQ[3..1]** | 3 | H | 2S | ← | B16,6,18 | DMA request |
| **DACK[3..0]#** | 4 | L | 2S | → | B15,26,17,19 | DMA acknowledge |
| **ALE** | 1 | H | 2S | → | B28 | "address latch enable" |
| **CLK** | 1 | – | 2S | → | B20 | CPU clock |
| **IOCHCK#** | 1 | L | OC | ← | A1 | I/O error — makes NMI |
| **IOCHRDY** | 1 | H | OC | ← | A10 | pull LOW for wait states |
| **OSC** | 1 | – | 2S | → | B30 | 14.31818 MHz |
| **TC** | 1 | H | 2S | → | B27 | DMA terminal count |
| **GND** | 4 | | PS | | A32;B1,31,32 | signal & power gnd |
| **+5V** | 2 | | PS | | B3,29 | +5V supply |
| **+12V** | 1 | | PS | | B9 | +12V supply |
| **−5V** | 1 | | PS | | B5 | −5V supply |
| **−12V** | 1 | | PS | | B7 | −12V supply |

**Notes:** (a) OC = open-collector; PS = power supply; 2S = 2-state (active pullup); 3S = 3-state.
(b) address, data, and read/write lines can be driven by bus master (e.g., during DMA).

HIGH. Tells I/O port that a DMA block data transfer is complete. A DMA device must qualify it with DACK′ for the channel in use, since TC is asserted when any of the DMA channels finishes a block transfer.

**IOCHK′: I/O channel check** Open-collector, input only, active-LOW. Generates highest-priority interrupt (NMI, "nonmaskable interrupt"); used to signal error condition from some peripheral. CPU figures out who's in trouble by device polling (§14.3.9A); each peripheral that can assert IOCHCK′ must therefore have a status bit that can be read by the CPU.

**IOCHRDY: I/O channel ready** Open-collector, input only, active-HIGH. The processor generates "wait states" if requested by a slow peripheral (that pulls it LOW) before the second CLK rising edge of a processor cycle (normally four CLKs). Used to extend bus cycle for slow I/O or memory.

**GND, +5 Vdc, −5 Vdc, +12 Vdc, −12 Vdc: ground and dc supplies** Regulated dc voltages that are bused for use by peripheral interface cards. Check the specifications of the host processor for power limitations, which are machine dependent. Generally speaking, there should be enough power to run anything you can stack onto the PC104 bus.

### 14.3.12 The PC104 as an embedded single-board computer

The PC104 standardized bus has been implemented in numerous single-board computers (SBCs), with an impressive variety of compatible peripheral cards; these are created by more than 100 manufacturers. These little boards often wind up as *embedded systems*, that is, dropped into instruments as part of their intelligent design. Figure 14.19 shows a view looking down into the access hatch of a complex optical detector system on an astronomical telescope: there's a PC104 SBC, running embedded Linux from its piggyback flash-memory "disk." This particular SBC (from Diamond Systems) includes Ethernet and serial ports (and other stuff not used here). The box on the left is an Ethernet media converter that lets us use optical fiber back to the control room; that's a good idea when your observatory

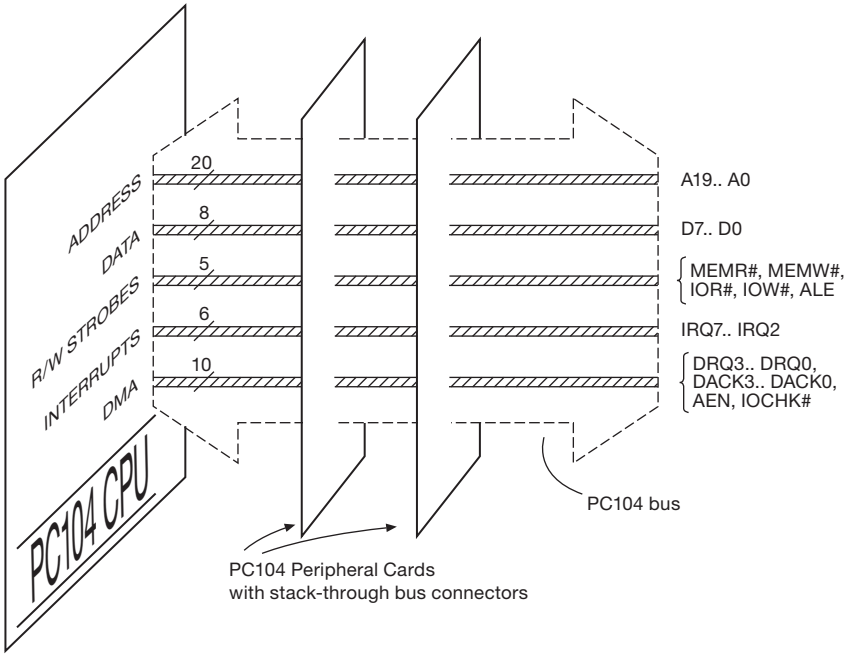



**Figure 14.18.** The PC104 parallel multidrop bus, making its grand tour.

is on a mountaintop (that's where people build them), because those thrilling lightning displays you get in the summertime can zap everything that is connected by wire cabling.

## *14.4 Memory types*

As we remarked back in §14.1.2, computers need fast "random-access" memory (i.e., memory in which you can get to any data directly, as compared with the sequential access of data stored on magnetic tape), which usually takes the form of *dynamic RAM modules*. These are the skinny circuit boards that plug into memory sockets on the computer motherboard, with typically 240 contacts along the mating edge. The most common variety currently is the "SODIMM" (small-outline dual-in-line memory module, $3\times13.3$ cm), on which is mounted an array of individual DRAM chips, with module capacities up to several gigabytes (organized as a 64-bit- or 72-bit-wide parallel data path). These are what you want if your task is simply to populate a commercial motherboard – just make sure you have chosen compatible memory modules from the bewildering variety[26] (check the computer motherboard specs and/or the online product finder from manufacturers like Corsair, Crucial, or Kingston).

But there's lots more to know if you want to understand what's going on, and if you want to be able to design your own systems that need memory. In this chapter section we'll discuss the various kinds of memory: static RAM, dynamic RAM, and non-volatile memory types like "flash RAM."

### 14.4.1 Volatile and non-volatile memory

For many applications there's no need to retain memory contents when power is turned off. A computer, for example, freshly loads its operating system, application programs, and data into working memory during the boot process, so it's OK if that memory is volatile. Those

[26] Characterized by *too many* parameters: form factor (e.g., SIMM, RDIMM, SODIMM), pin count (e.g. 200, 204, or 240 pins), density and data width, ECC or non-ECC, buffered or unbuffered, SDRAM generation (SDR, DDR, DDR2, DDR3, DDR4, DRDRAM), clock speed (e.g., PC3-10600) and data rate (e.g., 1333 MT/s), CAS latency (e.g., CL=4), voltage (e.g., 1.5 V), single or double rank, registered or unregistered, and parity or no parity. There are literally hundreds of choices, most of which will not work in a given motherboard. As a specific example, this footnote is being written into some portion of four banks of 1 GB density 64-bit wide DDR2 PC2-6400 CL=4 (5-5-5-15-2T) 1.9 V unregistered and unbuffered SDRAM with no parity and no ECC, on 240-pin SODIMMs.

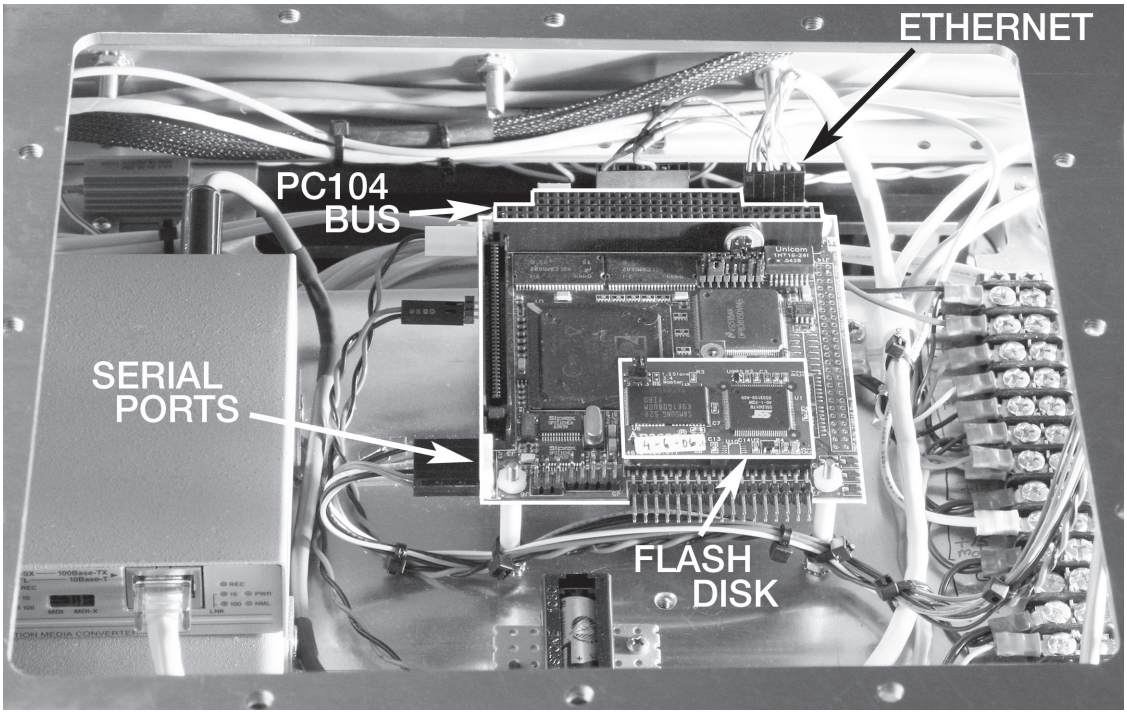



**Figure 14.19.** PC104 single-board computer embedded in an astronomical instrument.

programs and data must, of course, be retained somewhere when power is off, and that is the function of non-volatile mass memory, usually in the form of a hard disk (rotating magnetic storage) or a "solid-state disk" (a misnomer: it's an array of flash-memory chips, wrapped up in a disk-like enclosure for compatibility).[27] By accepting volatile memory you gain in speed, density, endurance (number of erase/write cycles before wearout), and price compared with those of currently available non-volatile technologies, as we'll see. Both static RAM (SRAM) and dynamic RAM (DRAM) are volatile,[28] whereas flash RAM and EEPROM (along with some interesting new technologies) are non-volatile.

### 14.4.2 Static versus dynamic RAM

Static RAM stores bits in an array of flip-flops, whereas dynamic RAM stores bits as charged capacitors. A bit once written in a SRAM stays there until rewritten or until the power is turned off. In a DRAM the data will disappear in less than a second, typically, unless "refreshed." In other words, a DRAM is always busy forgetting data, and it is rescued only by periodic clocking through the "rows" of the two-dimensional pattern of bits in the chip. For example, you have to access each of 8192 row addresses in a 1 Gb DRAM every 64 ms (an average rate of one row every 7.8 $\mu$s).

You might wonder what would possess anyone to choose a DRAM: by not using flip-flops, the DRAM saves space, giving you more data on a chip and at something like a tenth the cost.

Now you might wonder why anyone would choose *static* RAM (fickle, aren't you?). The major virtue of SRAM is its simplicity, with no refresh clocks or timing complexity to worry about (the refresh cycle competes with normal memory access cycles and must be properly synchronized). Thus, for a small system with only a few memory chips, the natural choice is SRAM. Furthermore, SRAM's zero quiescent current (compared with significant idling current in standard DRAM) makes it desirable for battery-powered devices. In fact, CMOS static RAM, backed up with a battery when main power is off,

[27] In a small system, for example an embedded processor of the kind we'll see in the next chapter, the "mass memory" more often takes the form of a single flash-memory chip, or (better) some flash memory included on the microprocessor ("micro*controller*") chip itself.

[28] The volatile memories have no wearout mechanism – you can read and write to your heart's content.

constitutes an alternative nonvolatile memory (with advantages in speed and endurance). A further advantage of SRAM is its availability in very high-speed versions (access time of 8 ns or less in asynchronous versions; clock rates of 400 MHz or more in synchronous versions). Finally, as we'll see, there is a class of memory known as *pseudostatic* RAM (PSRAM), which combines the low cost and high density of DRAM with the low power and simple interface of SRAM; it might well be described as "a DRAM in SRAM's clothing." Let's take a closer look at both SRAM and DRAM.

### 14.4.3 Static RAM

Static RAM stores each bit in a flip-flop cell (Figure 14.20): the flip-flop itself consists of a pair of cross-connected inverters (each made from a complementary pair of pMOS and nMOS switches), with two additional nMOS transmission gates to couple it to the outside. The latter are switched on to convey the flip-flop's state to a pair of complementary *bit lines* (which drive a differential latching *sense amplifier*) during a READ, and to assert (overwrite) the flip-flop state (according to the levels present on the bit lines) during a WRITE. This is known as a "6T" (six-transistor) SRAM cell.

As with most varieties of memory, SRAM bit cells on a single chip are organized into multibit "words" that are read or written as a group. Common widths range from 8 bits to 32 bits, often accompanied by additional parity bits (thus 9-, 18- or 36-bit widths). These words are further organized logically into an array of rows and columns, as shown in Figure 14.21, such that the selected word's bit cells are coupled to the respective bit-line pairs (and sense amps) according to the word's multibit address. The figure represents a 4 Mb SRAM (512 k 8-bit words[29]), organized as eight planes (one for each bit) of 1024 rows and 512 columns.

#### A. Asynchronous SRAM timing

Traditional SRAM is asynchronous, meaning that there is no clocking input; instead, you apply address, data, and control signals with proper timing, and data comes out (READ cycle) or is written (WRITE cycle) accordingly. Using asynchronous SRAM is as easy as falling off a log: to

[29] We follow common usage here, with "Mb" (megabit) signifying $2^{20}$ bits, about 5% more than a *decimal* million ($10^6$). Purists prefer to use the prefix "Mi" (pronounced mebi, contraction of "mega binary") to mean $2^{20}$, to keep pure the essence of mega as a good old-fashioned million. They would say that this memory is 4 Mib, or half a mebibyte; some would think they are speaking with food in their mouths.

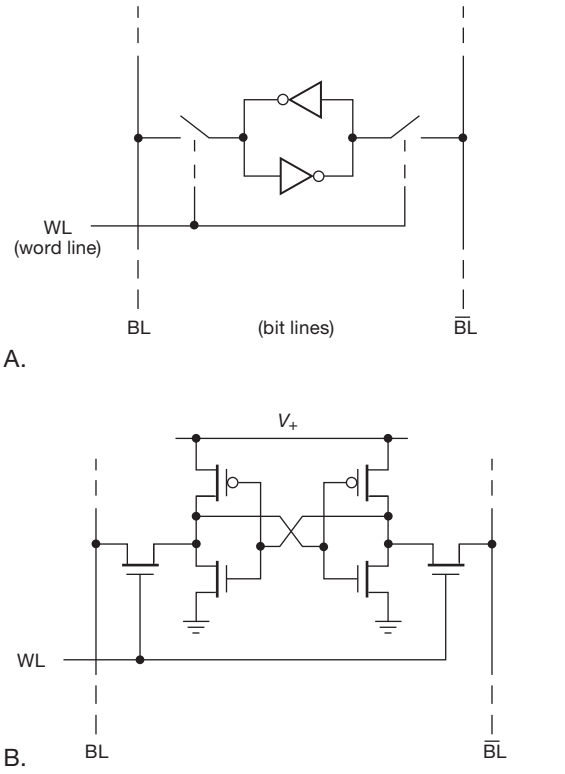



**Figure 14.20.** Static RAM holds each bit in a four-transistor CMOS flip-flop, read or written by a pair of one-transistor transmission gates. WL, word line; BL, bit line.

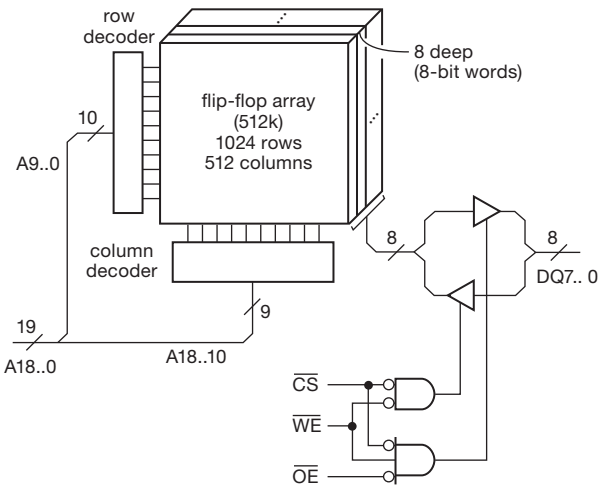



**Figure 14.21.** Asynchronous SRAM architecture: an array of 6T bit cells with an $n$-bit parallel address, and simple control by WE′, CS′, and OE′. Word widths of 8 bits and 16 bits are most common.

READ a word, you assert the address, chip enable (CE′), and output enable (OE′); the requested data obligingly appears on the three-state data lines a maximum of $t_{AA}$ (address-access time) later. To WRITE a word, you assert address, data, and CE′, then follow (after a minimum of an address setup time, $t_{AS}$) with a write enable (WE′) pulse; valid data are latched into memory at the end of the WE′ pulse. Figures 14.22 and 14.23 show the actual timing constraints for a fast static RAM ($t_{AA}$=8 ns, $t_{AS}$=0 ns), of the sort you might use in a switch or router, or as an external memory cache. The "speed" of the memory is set by the time from assertion of valid address to valid data (for READ) or to completion of the write cycle (for WRITE), assuming the other signals (CS′, WE′, and OE′) are asserted when needed.

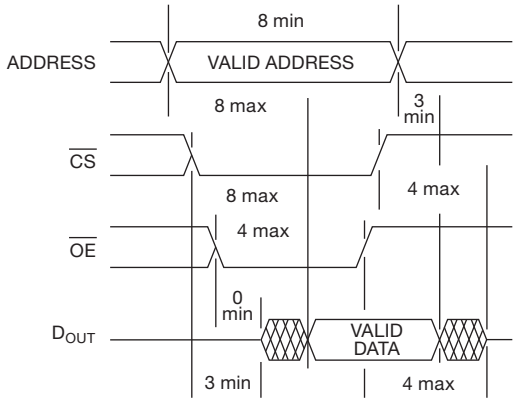



**Figure 14.22.** Asynchronous SRAM read cycle (during which WE′ is held HIGH). The numbers shown are guaranteed worst-case timings, in nanoseconds, for this 512 kB fast (8 ns) SRAM (Samsung K6R4008V1D-08).

Static RAM is currently available in sizes up to 16 Mb, with word widths of 1 to 32 bits. Variations include memories with separate input/output pins, memories with very low current drain ($\sim 1\mu$A standby, $\sim$1 mA when operating at 1 MHz), and memories with dual-port access (independent address lines, data lines, and control lines, sharing the same stored memory, with "semaphores" for software handshaking between ports).

For whatever it's worth, note that you don't have to hook up SRAM data lines to corresponding numbered data lines of a processor (or any other device needing memory) – you can scramble them any way you want, since they get unscrambled when you read back what you wrote.

### B. Pseudo-static RAM

Static RAM has the advantages of a simple control interface and very low power consumption. But its six-transistor

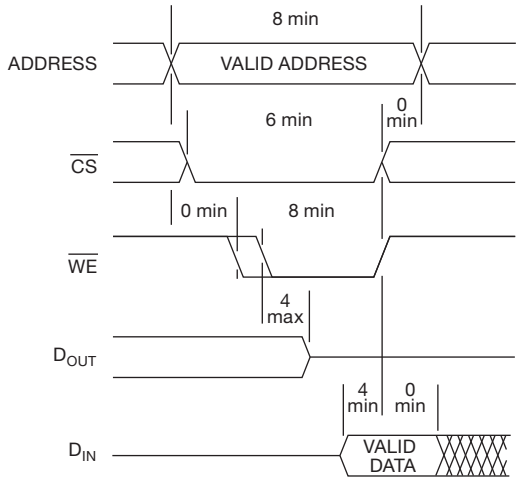



**Figure 14.23.** Write-cycle timing for the same memory as in Figure 14.22.

cell uses more area than the analogous one-transistor/one-capacitor DRAM cell (discussed below), so it costs more per bit and you can't put as much memory on a chip of a given size. However, the denser and less-expensive DRAM requires periodic refreshing, and, along with its multiplexed address scheme, it presents a more complicated interface.

Pseudo-static RAM combines their best features: it combines a DRAM core with hidden refresh logic, and wraps the combination in an interface that mimics the clean and simple asynchronous SRAM (Figure 14.24). It's a "DRAM in SRAM's clothing." Currently you can get

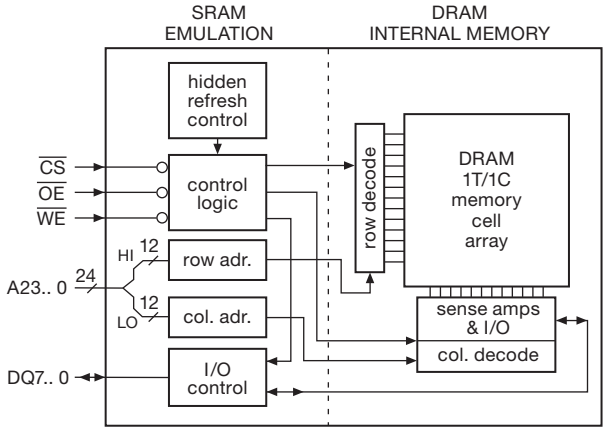



**Figure 14.24.** "Pseudostatic RAM" looks for all the world like a classic asynchronous SRAM. But its simple external interface conceals the truth: an efficient 1T/1C DRAM core, wrapped in an SRAM-emulating layer of logic (complete with hidden refresh).

PSRAM in densities up to 128 Mb; some are even drop-in compatible with asynchronous SRAM (i.e., identical pinout and function). They have access speeds of ∼50 ns, but faster (∼20 ns) in sequential "page-mode" accesses. (The refresh is hidden from the outside, and fast enough not to interfere at all with the speed or timing of asynchronous SRAM-like accesses. It is completely "invisible.") Because of the hidden refresh activity, the standby current is of order ∼100 μA. There's also a "deep power-down" mode in which the current can be as low as a few microamps; the bad news is that it's so deep you lose the refresh and therefore the stored data.

Although the standby current of PSRAM is significantly higher than the best you can do in micropower conventional SRAM, it's just fine for mobile applications like cellphones, with their typical battery capacity of ∼1000 mAh. The result is that pseudo-static RAM has largely replaced conventional asynchronous SRAM, except for applications like memory caches that require the blazing speed (≤10 ns) of fast SRAM.

#### C. Synchronous SRAM

In previous chapters we sang the praises of synchronous clocked logic, with its benefits in terms of noise (things have settled down before each clock), predictable performance, pipelined architectures, lack of metastability, and so on. So, why is it that SRAM is *asynchronous*?

It doesn't have to be. You can wrap a synchronous clocked state machine, complete with its data registers, around the intrinsically asynchronous memory-array core. Then you've got *synchronous* SRAM.[30] Because it is clocked, the speed of synchronous SRAM is given as a maximum *frequency*; currently available devices have speeds in the range of 100–400 MHz, and contain from 1 Mb to 72 Mb on a single chip, with word widths in factor-of-2 multiples of 9 bits (i.e., 9, 18, 36, and 72 bits, which include a parity bit with each byte).

Synchronous SRAM (and, even more so, synchronous DRAM, as we'll see) is a more complicated beast than its simple asynchronous sibling: these devices have complex modes, for example double data-rate transfers (using both clock edges, see Figure 14.25) and fancy methods of bursting and interleaving data; so you have to worry about mode bits and the like. And, of course, you've got a pretty fast clock (differential), with data going in and out in steps of just a few nanoseconds. Put another way, synchronous SRAM was devised for the highest speed and throughput, so the blessing of synchronous operation is mixed with the curse of fast clocks and tight timing margins.

### 14.4.4 Dynamic RAM

As we remarked earlier, you can save a lot of space on the chip by going to a one-transistor memory cell (with the state held on a small capacitor) if you're willing to carry out the periodic refresh of the capacitor's charge. That's dynamic RAM, with its "1T1C" memory cell (Figure 14.26). This is the workhorse of contemporary memory, with sizes of several gigabits per chip, typically supplied as plug-in memory modules with capacities currently up to 16 GB (128 Gb).[31]

As with SRAM, traditional DRAM memory was *asynchronous*, with timing a bit trickier than the simple SRAM, especially when refresh was added to the mix. And, as with SRAM, a synchronous variant (SDRAM) was created by wrapping a synchronous clocked state machine around the intrinsically asynchronous array of DRAM memory cells. One might say that "an SDRAM is an asynchronous DRAM in synchronous clothing." Asynchronous DRAM is history, now. But it's worth looking at, briefly, to understand what's going on inside the now-dominant synchronous DRAM.

#### A. Asynchronous DRAM

Figure 14.27 shows in simplified form a 16-word by 1-bit asynchronous DRAM, laughably small by any real-world standard, but just right for explaining how it works. The nMOS transistors each have a small capacitor (of order 30 femtofarads) from channel (call it the drain) to ground, and are arrayed as four rows and four columns. The row drivers use the latched row bits (high half of the address) to bring the corresponding nMOS gate terminals HIGH, turning on those transistors and therefore connecting their capacitors to the column lines, which in turn connect to a set of latching sense amplifiers (SA's). The output of one of the sense amps is selected by the latched column bits (low half of the address), passing through the bidirectional multiplexer to the output/input buffer. (Dynamic memory multiplexes the address bits, halving the number of address lines.)

Here's how it works. Let's take a READ cycle first, assuming that the various capacitors (one per bit) have been

[30] And the analogous thing is done with dynamic RAM, as we'll see, creating SDRAM in its many flavors: SDR (single data rate), DDR (double data rate), DDR3, DDR4, etc.

[31] *This* month, anyway; we've been riding Moore's law now for four decades, and we are certain that this will prove hopelessly quaint by the time the ink is dry.

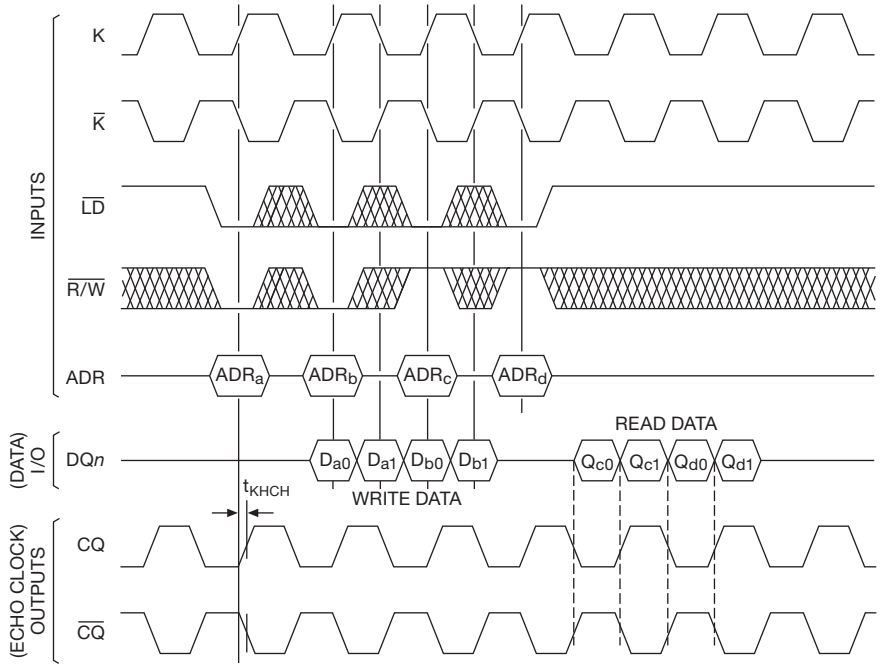



**Figure 14.25.** Synchronous SRAM timing, with double data-rate (DDR-2) clocking during both READ and WRITE. Data is clocked in two-word (successive addresses) bursts: for WRITE cycles the input data must be presented on both edges of the input clock $K/\overline{K}$, one cycle after the address is loaded; for READ cycles the data emerges 1.5 clock cycles after address loading, on both edges of the "echo clock" $CQ/\overline{CQ}$ (which regenerates the input clock $K/\overline{K}$, with a slight delay $t_{KHCH}$).

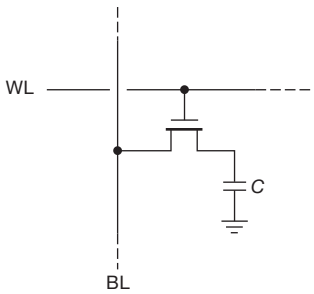



**Figure 14.26.** Dynamic-RAM "1T1C" bit cell. Each bit is held as a charged (∼1 V) or discharged (0 V) capacitor, whose state is read, written, or refreshed by a bit line (BL) when asserted by a controlling word line (WL). Typically $C \approx 30$ femtofarads (1 fF=$10^{-15}$ F).

previously written, therefore either positively charged (to ∼1 V), or uncharged. Look at Figure 14.28, where we've shown the basic (single-address) READ and WRITE cycles for standard 70 ns asynchronous DRAM.[32] The address lines are multiplexed, with the two high-order address bits (row address) asserted first, along with a row-address strobe (RAS′). Those bits are latched, causing the gates of the selected row to go HIGH, turning on the MOSFET transmission gates, and thus coupling the respective capacitors to the column sense amplifiers (SA). The sense amplifiers are latching devices, here drawn notionally as fedback noninverting amplifiers. (In practice they are implemented as flip-flops that begin the cycle in a balanced state and become unbalanced by the bit-capacitor charge that is switched into them.[33]) During this part of the DRAM cycle the sense amplifiers in each column do two things: they latch into the state of the bits that were held in the cells of

[32] There are additional modes, with names like "page mode" and "extended data out," for more efficient access to data from several consecutive addresses.

[33] In a further dose of reality, things are a bit more complicated: the sense amplifiers are *differential*, and the DRAM array is usually built in a "folded-bit" arrangement so that any given row line activates only the even or odd cells; the inactive (neutral) bit line floats at the "precharge" level ($V_{DD}/2$) and acts as a reference voltage by which the balanced sense amplifier compares the $\Delta V$ "bump" up or down from the capacitor's charge in the respective bit cell. The $\Delta V$ is quite a bit less than the $\pm 0.5$ V you might expect, owing to the additional ∼200 fF capacitive loading of the bit line and sense amplifier: in practice, memory designers aim for $\Delta V \geq 100$ mV, for a clean readout of 0 or 1 by the sense amplifier.

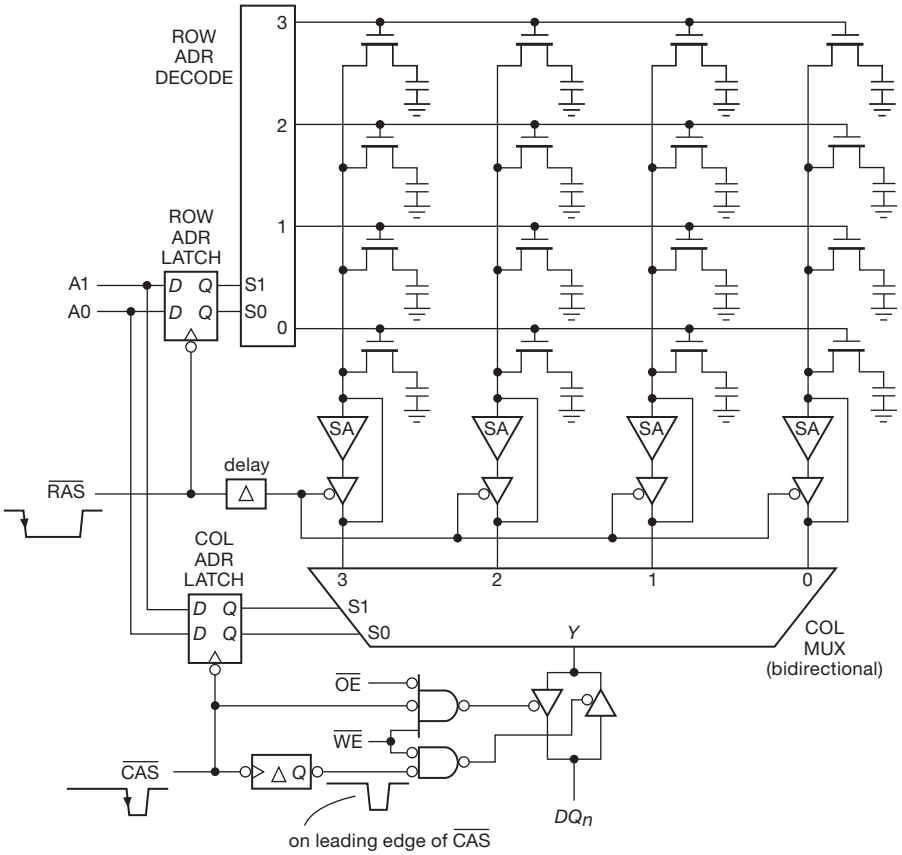



**Figure 14.27.** Asynchronous DRAM architecture, here illustrated with a tiny 4×4 array of 1-bit "words." The address lines are multiplexed, with row and column addresses registered internally. The latching sense amplifiers (SA's) read (and refresh) the state of the row-addressed bit lines during a READ cycle. During a WRITE cycle they are instead overdriven by the input data on the same shared input/output data line *DQ*0.

the selected row; and they "refresh" those cells by asserting their latched level back onto the respective cell capacitors.

In the second half of the DRAM cycle the address lines carry the two low-order address bits (column address), latched by the column-address strobe (CAS′). That latched address causes the column multiplexer to select the output from the selected sense amplifier, which (because WE′ is disasserted) is asserted onto the bidirectional (input/output) data line (the single DQ0 here, in this 1-bit wide memory; most DRAMs are wider, with word sizes of 4, 8, or 16 bits). The memory is *asynchronous*, meaning (as with asynchronous SRAM) that the valid data appears on the data line(s) with some guaranteed maximum delay time following assertion of the various addresses and strobes. In contrast to the now-universal *synchronous* DRAM (next subsection), there's no master clock.

The WRITE cycle is similar, but with input data and WE′ asserted around the leading edge of CAS′; the WE′ line causes the data line DQ0 to become an input, whereupon the asserted data level forces (overrides) the selected sense amplifier into the state of the data input. That latched state then charges (or discharges) the respective bit cell capacitor.

Look at Figure 14.27 again. Once a row has been activated (in the RAS part of the cycle) and a column has been selected for reading (or writing), there's no reason to repeat the same row address if you want to read (or write) other columns within that same row. That's the idea behind "page-mode" and "extended-data-out" cycles; and it's of great practical utility, because computer memory accesses usually involve bursts of multiple successive addresses (for sequential instructions, data arrays, and so on).

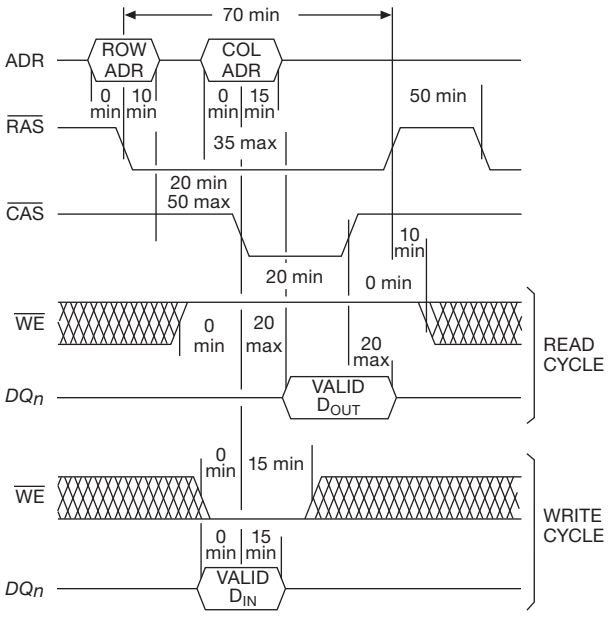



**Figure 14.28.** Asynchronous DRAM timing. RAS′ and CAS′ are the row and column address strobes for the multiplexed address. As with asynchronous SRAM, data in/out is not tied to a clocking edge; rather, it obeys worst-case timing specifications (here shown for standard 70 ns memory).

### B. Synchronous DRAM

As we remarked above, all DRAM currently in common use is of the synchronous variety ("SDRAM"), in which an externally supplied clock signal[34] controls a synchronous state machine that is wrapped around the intrinsically asynchronous DRAM memory core. The original single-data-rate (SDR) SDRAM has evolved through several generations, with names like DDR (double data-rate, i.e., clocking data on both clock edges), DDR2, DDR3, and DDR4. The clock transitions are used for the obvious things: to load the row and column addresses and to clock data in (WRITE) or out (READ). Synchronous DRAM is usually operated in "burst mode," with multiple data words from consecutive column addresses clocked out in sequence (see the MODE description, below).

Unlike asynchronous DRAM, SDRAM is a complicated beast, with a set of "commands" (also clocked) that determine what is happening. These commands are defined by three bits, which rather curiously use the legacy signal names RAS′, CAS′, and WE′ that originated with asynchronous DRAM. Here, however, these three input signal bits, set up before the next clocking edge, determine what happens on that clock edge. Look at Figure 14.29, where we've listed five basic commands. "Activate row" loads a row address, with subsequent "READ" and "WRITE" commands loading a column address and initiating the corresponding data transfer. During such transfers (which may be 4-, 8-, or 16-bits wide, according to the particular chip's architecture), the data moves out or in at the clock rate (SDR, as in Figure 14.29), or at double that rate (DDR), as timed by the clock.

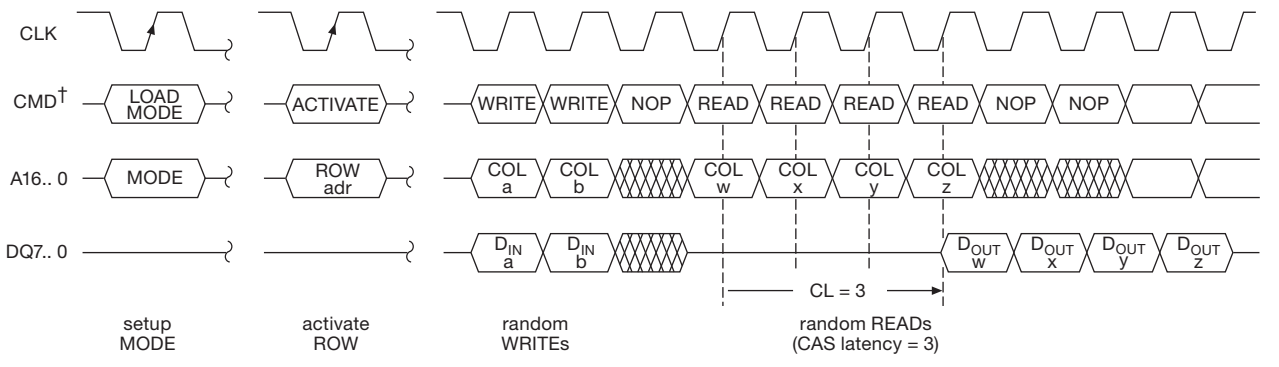
However, note particularly the timing of a READ: although data moves out at the full clock rate, there is a delay (*latency*) of several clock cycles from assertion of the column address to the corresponding data. This is the "CAS latency," illustrated in Figure 14.29 for the case of CL=3. A given memory chip will specify (through its part number) a minimum CAS latency for a specified clock frequency (for example, an MT47H128M8HQ-25E is a 128 MB 8-bit-wide DDR2 SDRAM in a 60-ball FBGA package with CL=5 at $t_{clk}$=2.5 ns). Because the rest of the system needs to know (and agree on) the actual CAS latency, you tell the chip what CAS latency to use by sending it a "LOAD MODE" command (i.e., a clock cycle, during which RAS′, CAS′, and WE′ are all held LOW), with the mode defined by the bits asserted on the address lines. The mode includes not only the CAS latency, but also bits defining whether it's single-address access or burst mode (for the latter, a choice of two, four, or eight consecutive words), and some other control options that you really don't want to hear about. In a further bit of complication, some varieties of SDRAM (e.g., DDR2) have a clock latency during WRITE cycles; logically enough, that's called "write latency" (WL), which you respect by asserting the data delayed by WL clocks after the column address is clocked in.

Because it is clocked, the speed of synchronous DRAM is given in terms of *frequency*, with labels like "DDR3-1600." That would describe a double-data rate SDRAM, conforming to the DDR3 standard, with data clocked on both edges of an 800 MHz clock. Currently available devices have data rates in the range of 400–1600 MT/s (megatransfers per second), and contain from 1–4 Gb on a single chip, with word widths of 4, 8, or 16 bits. The next generation (DDR4) raises the transfer rate to 1600–3200 MT/s, and raises single-chip densities to 16 Gb or more.

### 14.4.5 Nonvolatile memory

Nonvolatile memory (NVM) – data storage that is retained in the absence of dc power – is essential in many everyday applications, for example (a) startup ("boot") code, and

[34] At these high speeds it's invariably configured as low-voltage differential: *CK* and $\overline{CK}$.




†CMD is given by $\overline{\text{RAS}}$, $\overline{\text{CAS}}$, and $\overline{\text{WE}}$:

| | $\overline{\text{RAS}}$ | $\overline{\text{CAS}}$ | $\overline{\text{WE}}$ | ADR lines |
|---|---|---|---|---|
| ACTIVATE ROW | L | H | H | row address |
| READ | H | L | H | column address |
| WRITE | H | L | L | column address |
| NOP | H | H | H | — |
| LOAD MODE | L | L | L | mode bits |

**Figure 14.29.** Synchronous DRAM uses an external interface whose operations are synchronized by an externally applied clock. The simplest "single data rate" (SDR) SDRAM hijacks the RAS′, CAS′, and WE′ for use as a 3-bit command. Shown here are random (as opposed to *burst*) writes and reads for SDR SDRAM. Burst-mode operation is far more common, with data from several consecutive column addresses being clocked on successive clock edges (or on *both* edges, in the case of the popular DDR ("double-data rate") SDRAM.

various settings, in a general-purpose computer; (b) resident firmware for any device that uses an embedded processor; or (c) temporary storage on a USB "thumb drive" of documents, images, or other files. The semiconductor memories we've seen above are unsuitable, because they lose their stored data when dc power is removed.

One solution to the problem is to provide a battery, so power is never lost. That's "battery backed-up" memory (which is commonly used to store computer settings, where it's sometimes called, simply, "the CMOS"). The good news is that you can use any kind of memory with a micropower data-retaining standby mode (usually static or pseudostatic RAM), which means that you don't need to worry about limitations that afflict many forms of nonvolatile memory, for example the leisurely (∼ms-scale) program or erase times, or the endurance (limited number of write cycles, typically $\sim 10^5$–$10^6$). The bad news is that you've got to provide a battery and make sure it is not discharged.[35]

The other solution is some form of truly non-volatile storage. This is a very active area of semiconductor development, with most current devices using some form of charge storage on a MOSFET's "floating gate" (e.g., the ubiquitous "flash memory" in USB thumb sticks, in all manner of consumer electronics, and in computer solid-state drives known as SSDs). This is impressive technology, with single-chip storage capacities currently up to 1 Tb, data retention of at least 10 years, and retail prices as low as $0.50 or less per gigabyte. But, as we'll see, floating-gate (FG) storage has some drawbacks, notably a finite number of erase/write cycles (endurance) and a relatively slow write (and erase) access time (∼milliseconds); by contrast, standard SRAM and DRAM have unlimited endurance, and equally speedy read and write speeds (∼10s of nanoseconds). Some NV technologies in development may remedy these shortcomings; they go by names like ferroelectric RAM (FRAM), magnetoresistive RAM (MRAM), and phase-change RAM (PRAM).

In the following subsections we describe briefly some of the precursors to current non-volatile memory technology; then we'll look at the 800-pound gorilla in the room, namely floating-gate flash memory.

[35] Not a completely trivial task: we bought four motherboards from a major manufacturer, all of which lose their CMOS settings if incoming ac power is lost for more than a few hours. This is quite an inconvenience (there are roughly 25 settings that must be restored), probably caused by a batch of chips with out-of-spec leakage current or minimum retention voltage.

### A. Legacy non-volatile memory

#### Mask ROM

First there was "mask ROM," which consists of gates with a hardwired connection pattern, built-in from birth, that cannot be altered. It's a fixed lookup table. And it's very much "read-only" (hence "ROM"). This is still a useful method for putting a conversion table, for example, into a custom IC. It's certainly non-volatile; but it's not really what we imagine as useful NV storage.

#### PROM

Next came programmable read-only memory (PROM), which you can program, once, in the field. It's also called "fuse ROM," because it uses an array of tiny on-chip fuses (metallic or semiconductor), which are selectively blown out to leave the desired fixed memory pattern. The subsequent development of reprogrammable (erasable) NV memory rendered PROM obsolete.

#### EPROM

Erasable programmable memory came next. Here the bit pattern is held on tiny capacitors, in the form of buried "floating gates" that alter the threshold voltage of the associated MOSFETs (Figure 14.30); this is the same technology as that used in contemporary flash memory. The "control gate" is then used to read out the stored bit: when the floating gate is charged (negatively) with electrons, the threshold voltage is several volts positive, whereas an uncharged floating gate produces a negative threshold voltage (therefore conduction with the control gate at 0 V). Because there's no connection to the floating gate, you use tricks for writing and erasing: to write, negative charge is put onto the gate by a process called "channel hot-electron injection" (CHE), which involves running drain current with elevated drain voltage (12–20 V); to erase you expose the chip to UV light for several minutes, which erases the entire chip at once. The erase process requires a transparent quartz window to let the UV light in (hence the alternative name, UV EPROM); the associated hermetically sealed ceramic packaging makes these things expensive, and large. And, they had to be removed from the circuit for erasure and reprogramming. These were made obsolete by in-circuit *electrically* erasable non-volatile memory.

#### OTP EPROM

What a magnificently self-contradictory name: "One-Time-Programmable Erasable Programmable Read-Only Memory! (Engineers *do* have a sense of humor, though perhaps a bit twisted.) OTP EPROM was the answer to the problem of EPROM packaging cost, namely, to put a UV-erasable EPROM in an inexpensive plastic package. The only (!) drawback is that the package is opaque, so you can't erase the thing. What we did, in those days, was to buy one or two of the pricey UV EPROMs, use them to develop and test the program, then program the plastic parts with the fully debugged code. These devices, too, became obsolete with the development of electrically erasable non-volatile memory.

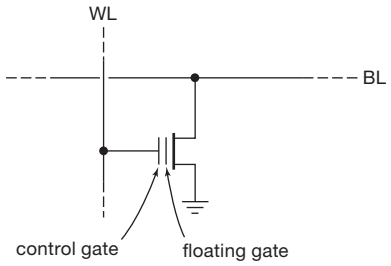



**Figure 14.30.** "Floating-gate" bit cell, used in non-volatile memories such as EEPROM and flash ROM. Data is written to the floating gate by tunneling or hot-electron injection; its charge alters the threshold voltage, as read out by the control gate and bit line. The leakage currents out of the floating gate are so low that data is retained for at least ten years, requiring no power or refresh.

### B. Electrically-erasable non-volatile memory

#### EEPROM

The modern era in non-volatile memory was ushered in with the electrically erasable programmable read-only memory[36] (EEPROM, or "E-squared"). No more UV sunbaths! No more removing and replacing the ICs – these puppies are *in-circuit* reprogrammable! The trick here is to use elevated voltages to discharge the floating gate, by a quantum-mechanical phenomenon known as *tunneling* (in which a particle, here an electron, with insufficient energy to overcome the potential barrier of the gate insulator can, under the right circumstances, just magically appear on the other side[37]). It's officially called Fowler–Nordheim tunneling, usually abbreviated F-N. EEPROMs include on-chip charge pumps to generate the elevated voltages needed to program like UV EPROMs and to erase via F-N

[36] Another name is EAROM – electrically alterable read-only memory.

[37] A macroscopic analogy: you need to get your oxcart over the Khyber Pass, but, because of poor planning, you lack an ox. Hoping for a miracle, however, you give the cart a shove. There is, according to quantum mechanics, a finite probability that the cart will vanish from your hands, and reappear on the other side of the pass. For oxcarts the probability is awfully small (and that's a real understatement!); but it works fine for electrons hoping to find their way to the buried gate.

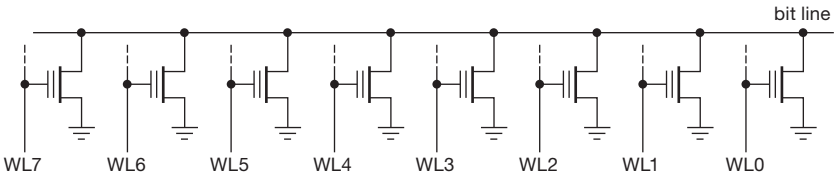



**Figure 14.31.** NOR flash memory, with its parallel arrangement of bit cells.

tunneling. Voilà – *in-circuit* programming (i.e., writing) and erasing.

EEPROMs use a memory-cell configuration (for example, a two-transistor bit cell) that permits bits or words to be erased and reprogrammed individually. That's a nice flexibility, but the circuitry that allows this flexibility takes up space that could be used for additional storage. The subsequent development of *flash* memory exploited a one-transistor bit cell, with its higher density (see flash memory, below); the price you pay is that erasure is carried out in much larger *blocks*, by way of their substrate contact.

EEPROMs, in common with flash memory, have a limited endurance, usually in the range of $10^5$–$10^6$ erase/write cycles. Erasure and writing are slow (∼10 ms), compared with reading (∼100 ns). They have been largely superseded by flash memory. However, the flexibility of single-byte (or even single-bit) rewritability makes EEPROMs well suited for retaining small amounts of data, for example calibration parameters (think of a bathroom scale's strain gauge), settings (that scale, again: pounds or kilograms), or lookup tables (still thinking of that scale... which needs correction for ambient temperature). EEPROM for those kinds of applications is usually included on the same chip as the microcontroller itself (see Chapter 15). You can get standalone EEPROMs, also; these usually use a serial protocol (SPI, $I^2C$, UNI/O, Microwire), so they fit in a tiny package (SC-70, DFN, etc.). There's a good selection from companies like Atmel and Microchip, with densities from 128 bits to 1 Mb. They're inexpensive, too – a 1 kb $I^2C$ EEPROM costs $0.17 in quantity, and twice that for a 64 kb part.

The EEPROM's intrinsic flexibility (erase/write individual bits or bytes) requires more silicon than the denser flash memory, with its *sector*-wide erase/write. Let's take a look.

### C. Flash memory

Flash memory discards the bit or byte writability of EEPROM, instead performing block-sized erase. The good news is that it's faster (it erases "in a flash," hence the name), if you want to erase many bytes. The bad news is that you cannot modify smaller amounts of data (though external logic, with some SRAM, can fake it so that you don't realize what's going on inside). But the big news is that the chip is denser, particularly in the form known as NAND flash, so you get a lot of memory on a single IC (up to 1 *tera*bit, currently; though the innards aren't an honest "chip," but rather a "stack" of thinned chips). Currently available flash memory comes in two flavors, called NOR and NAND.

#### NOR flash

Figure 14.31 is a simplified view of the bit-cell arrangement used in NOR flash memory, which was the original flash architecture. Because the storage transistors are in parallel, you read out by gating off all except the selected transistor. The floating gate cells are programmed via channel hot-electron injection (the same as with EPROM). However, as with all varieties of flash memory, erase must be done a block (or "sector") at a time, with the block's elevated substrate voltage causing F-N tunneling from the floating gates. Typical block sizes are 4–64 kB, often flexibly configured with several different block sizes on one chip. NOR flash uses a simple parallel SRAM-type interface (sometimes providing a choice of asynchronous or synchronous modes), and can be used directly to hold executable code. Note, however, that the limited endurance, as well as the sector-size erase, means that the executable code must treat the memory as read-only. Currently available NOR flash devices come in densities from about 1 Mb–1 Gb.

#### NAND flash

By contrast with NOR, the designers of NAND flash intended it as a dense mass-storage device, more akin to a magnetic hard drive. It's the memory that's used in USB thumb drives, compact flash (CF) cards, secure digital (SD) cards, solid-state drives (SSD), and the on-chip code memory used in microcontrollers ($\mu$C; see Chapter 15). The name NAND derives from the series connection of bit cells (Figure 14.32), which are read by holding HIGH all gates except the selected transistor; the selected transistor then determines the conduction of the series string, according

to the charge on its floating gate. Both erasing and writing use tunneling; erasing is done in sectors. What's not obvious is the greater density you get with a series connection: no space-hogging source or drain contacts are needed, except at the ends of each string. To keep the pin count down, NAND uses a serial command interface; and, like NOR flash, it erases in sectors. Note, however, that a device like a USB thumb stick includes a memory controller, so that the electrical and timing details of erasing and data read/write are invisible to the external interface. Likewise, the specification for SD flash cards requires that they include not only the native SD interface protocol, but also the standard SPI (serial peripheral interface) protocol that is included in most contemporary microcontrollers.[38] Memory controllers used for flash storage devices do more than merely obey the rules: they use tricks like "wear leveling" (cycling through different blocks of storage to minimize reuse) to circumvent the endurance-limiting effects of insulator damage caused by tunneling; and they detect bad and degraded cells, whose addresses they then write to locations in sector 0 of the memory chip so that future memory accesses are diverted to spare cells.

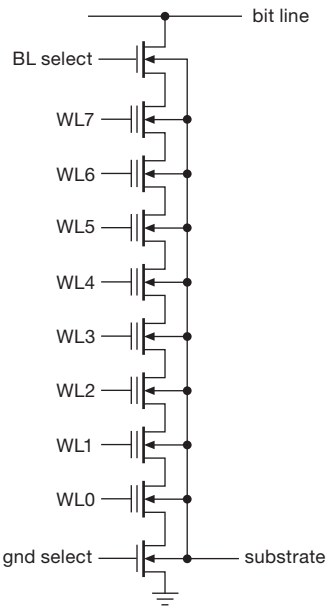



**Figure 14.32.** NAND flash memory, with its series arrangement of bit cells.

[38] The actual protocol that goes over the SD-card SPI interface isn't trivial, because it must support memory operations such as read, write, erase, check status, and so on. But it's straightforward, and you can get libraries for popular microcontrollers such as AVR and ARM (see Chapter 15).

In a daring ploy to multiply memory density, manufacturers of NAND flash are using what's politely called "multilevel cell" (MLC) or "triple-level cell" (TLC) storage. Put bluntly, this means putting partial levels of charge on the floating gate, which are then read out by measuring the approximate threshold voltage. Current implementations use four (MLC) or eight (TLC) levels, therefore 2 or 3 bits per transistor. (Should we call this a "$\frac{1}{2}$T" or "$\frac{1}{3}$T" cell?!) This is scary stuff – your precious data is held on partially charged tiny floating gate capacitors, whose level of charge must not change too much during the next few years. To get a sense of the scale, the roughly 0.3 femtofarad capacitance must not leak (or acquire) more than ∼3000 electrons in, say, $3 \times 10^8$ seconds (ten years). That's one electron per *day*! It's a miracle the stuff works at all.[39]

Currently available NAND flash comes in densities up to 1 Tb per IC; to get those densities, the manufacturers use 2-bit MLC storage, and multiple chip stacking within one IC package. To us, it is astonishing that a single IC can hold the equivalent of 16 bytes of information for every person on Earth.[40]

### D. Future non-volatile memories

Flash memory is great stuff. But it has finite endurance, and the erase/write process is slow. An ideal non-volatile memory would be "SRAM without the volatility." That is, it would have random read and write access at full speed, unlimited endurance, and long retention.

There are several technologies being explored; the following appear to be the most promising.

#### Ferroelectric RAM (FRAM, FeRAM, or F-RAM)

A ferroelectric material is the electric analog of a ferromagnetic material; that is, it holds the state of electric polarization. The idea is to make a 1T1C-style DRAM bit cell, but with the capacitor replaced with a thin film of ferroelectric material (e.g., a few atom layers of strontium bismuth titanate). You write a bit as with DRAM, applying a field across the film. Readout is different, however: you read "destructively," by sensing whether a write changed the state (producing a pulse of current); then you restore the state.[41] FRAM potentially can deliver random

[39] But it *does*! The several gigabytes of figures and text for this book edition made many journeys between office and home, in the form of partially charged femtofarad bits.

[40] We predict, with considerable confidence, that by the time you read this the bytes/IC will have increased far more than the people/Earth.

[41] This is analogous to the method used with the magnetic-core memory of yesteryear, in which the core's state was read by forcing it to a

read and write speeds of <50 ns, many-year retention, and very high endurance. Companies like Fujitsu and Cypress are currently manufacturing FRAM, offering serial interface devices (SPI and $I^2C$) to 2 Mb with claimed endurance of $10^{14}$ cycles, and parallel (asynchronous SRAM-emulating) devices to 4 Mb. An example of the latter is the MB85RE4M2T, with specified random read and write cycle times of 150 ns, ten-year retention at 85°C, and (drumroll) endurance of $10^{13}$ read and write cycles (that's 300 years, at 1000 read/writes per second.

#### Magnetoresistive RAM (MRAM)

Magnetoresistance is the change of electrical conductivity of a material under an applied magnetic field. Variations of the effect ("giant magnetoresistance," "colossal magnetoresistance," "tunnel magnetoresistance") are used in magnetic disk readout heads, an improvement over the traditional coil-based readout (which generates a voltage proportional to *changes* in flux, rather than to flux itself) that has led to an explosion in hard-disk capacity. It's possible to fabricate tiny on-chip magnetoresistive cells (complete with some ferromagnetic material), written by pulsing a current, and read out by its magnetoresistance.

MRAM has been "in development" for many years, with some recent production parts. For example, Everspin (spinoff from Freescale) offers 8-bit and 16-bit word size MRAMs, with densities to 16 Mb. Their MR2A16A, for example, is a 4 Mb 16-bit-wide non-volatile MRAM, with a standard asynchronous SRAM parallel interface. It has specified read and write cycle times of 35 ns, 20-year retention, and (big drumroll) claimed "unlimited" endurance; it costs $20 in unit quantity. Other companies involved in MRAM include Hitachi, Hynix, IBM/TDK, Infineon, Samsung, and Toshiba/NEC (where a 1 Gb MRAM is rumored). MRAM is not cost competitive with flash, by a long shot; it's still at the starting gate – but it's worth keeping an eye on.

#### Phase-change RAM (PRAM or PCM)

Certain metallic alloys ("chalcogenide glasses") exhibit a large difference of electrical resistance between their crystalline and amorphous (glass-like) states. This can be exploited to make memory cells, in which the phase-change alloy is heated in controlled ways to convert between the two states. Heating sounds like a slow and diffuse process, but at the small scales here (tens of nanometers) things happen locally, and in tens of nanoseconds. Phase-change memory is being pursued by companies like Samsung, Micron, IBM, and STMicroelectronics, and some prototype samples have been shipped (e.g., a 128 Mb, 90 nm PRAM from Numonyx). No claims of unlimited endurance, yet; but PRAM potentially offers high-density non-volatile memory with fast read and write cycle times.

known state, sensing if there was a change of state, then restoring (if needed) the original state.

### 14.4.6 Memory wrapup

Encapsulating the long-winded narrative above, we can summarize the current state of memory and its usage this way.

- Much of the time the problem has been solved for you.
  - Microcontrollers (Chapter 15) include on-chip memory, both flash (for non-volatile program storage), SRAM (for working memory), and, often, EEPROM (for parameters, tables, etc.).
  - Computer motherboards are configured to use fast SDRAM, as plug-in SODIMM (or other form-factor) cards; just follow instructions and you'll be fine.
  - Field-programmable gate arrays (FPGAs) without on-chip flash are configured to accept a configuration code from an attached serial flash memory, well specified in their datasheets.
- The simplest volatile memory is asynchronous SRAM, or its popular look-alike, pseudo-static RAM. Standard SRAM and PSRAM speeds are ∼50 ns, with "fast" SRAM speeds around 10 ns; the latter are good for external cache memories, while the former are still used in niche applications like high reliability medical electronics. Micropower SRAM can retain data at standby currents of ∼1 $\mu$A, and, for leisurely applications, run at currents of ∼1 mA.
- Synchronous SRAM has a more complicated interface, but runs at high clock speeds (to 400 MHz, combined with double data rate); good for external cache.
- Synchronous DRAM, in its double data rate varieties, is the favorite for fast computer working memory. Generally used as plug-in modules, but can be used as bare ICs, for example in conjunction with an embedded microprocessor (or FPGA with internal microprocessor core) in a stand-alone application like a wireless router, game console, display panel, or set-top box.
- Flash NAND is the current winner for dense non-volatile storage, and dominates in modules (USB thumbsticks, CF and SD cards) and as a hard-disk replacement (SSD), as well as for on-chip (microcontrollers, and application-specific ICs) and on-board (consumer electronics, video, network, etc.) applications. A particularly easy-to-use

form is provided by serial-interface flash, both as standalone ICs and, pleasantly, in the form of the SD card (which has an SPI-compatible interface); you can hook these serial memories directly to most microcontrollers, which include an integrated 3-wire SPI port.
- Flash NOR has the merit of a standard asynchronous SRAM interface, so you can substitute them for SRAM and execute read-only code directly.
- The byte rewritability of EEPROM is well suited to storage of parameters and tables, particularly where only a modest amount of data is involved.
- There is hope, in some recent technologies, for finding memory's holy grail: full-speed non-volatile memory with unlimited endurance. (We had that, a half-century ago, with magnetic-core memories. They're OK, if you're happy with cycle times measured in microseconds, densities measured in kilobytes, and prices in kilodollars. But, we've become jaded by the dazzling speed, density, and low cost of semiconductor memory; there's no going back. . . .)

### 14.5 Other buses and data links: overview

The PC104/ISA peripheral bus that we've seen in detail exemplifies a *parallel* multidrop bus architecture, with a group of shared DATA lines, ADDRESS lines, and some STROBE signals that signal direction and timing of data transfer (along with additional lines for interrupts and DMA). It originated more than 25 years ago, and has been superseded by faster and wider parallel buses, notably the PCI and PCIe[42] peripheral buses used in contemporary PCs.[43]

But the generality of a parallel data bus extends far beyond the innards of computer motherboards. Parallel data transfers, sharing the PC104's concepts of data–address–strobe, are used more generally in electronic devices for data transfer among devices such as liquid-crystal displays, video processing chips, and analog/digital converters, as we'll illustrate shortly. These connections between circuit devices often omit cumbersome addressing, and are better thought of as parallel data *links*. True parallel buses, however, can also extend to *external* data-hungry peripherals such as tape and disk storage, or test and measurement instrumentation, in the form of the SCSI or general-purpose interface buses (GPIBs).

And then there are the increasingly popular *serial* buses and data links, going from the simple (and slow) legacy RS-232 ("COM" port), to the fast USB and FireWire protocols. Serial links send their data bits sequentially, rather than as a byte-wide (or more) wholesale multibit transfer; so you might expect them to suffer in speed. Remarkably, however, fast serial protocols recover most of that speed by using low-voltage differential signaling at very high bit rates. A fine example is the evolution of the 16-bit ATA (also called IDE, or PATA) parallel disk link (a total of 40 wires, in the once-familiar ribbon cable) into a serial "SATA" link with just two pairs in a skinny cable. The SATA link is actually the *faster*: it can transfer up to 6 gigabits/second, whereas the now-obsolete "PATA" tops out at 1 gigabit/second.

More speed in a *serial* link – how can that be? Differential low-voltage signaling certainly helps, but there are two additional factors favoring serial links like SATA: (a) such links are "point-to-point" (one driver at the transmit end and one receiver at the other end), rather than "multidrop" (one driver somewhere on the bus and multiple receivers along the bus); and (b) unlike a multiwire parallel bus, a single serial line has no timing *skew* (the scatter of propagation times between the clock lines and the several data lines). These factors really matter at contemporary data rates of gigabits per second: (a) At such speeds the bus lines are electrically "long" (electrical signals travel $\sim$20 cm in a nanosecond in cables and on circuit boards), and so you've got to think of bus wires as *transmission lines* (Appendix H), on which multiple taps (called "stubs") act as impedance mismatches and generate a sequence of reflected signals; by contrast, a point-to-point connection has but one recipient, permitting good impedance matching. And, (b), at these speeds the unavoidable timing skew sets an upper limit on data clocking rates (as revealed by an "eye diagram" – Figure 14.33) in parallel buses, nicely circumvented in serial links in which the clock is recovered from the data stream itself. As it turns out, the advantages of controlled impedance and absence of timing skew outweigh the advantages of parallelism.

This move from parallel (and multidrop) to serial (and point-to-point) is seen broadly: in computer buses (PCI $\rightarrow$ PCIe), in disk interfaces (PATA $\rightarrow$ SATA; SCSI $\rightarrow$ SAS), in Ethernet cabling (coaxial multidrop $\rightarrow$ twisted-pair point-to-point), and in external buses (e.g., GPIB $\rightarrow$ USB). In addition to performance benefits, these serial

[42] PCIe is a hybrid: serial communication on a set of parallel "lanes."

[43] The latter are considerably more complex; we chose the PC104/ISA for its simplicity, and also because it is well supported in contemporary PC104 offerings, which will likely ensure its survival well beyond the lives of its forgotten contemporaries. The latter had names like Unibus, STD bus, EISA bus, MicroChannel, Q-bus, Multibus, VAX BI, NuBus, Futurebus, and Fastbus.

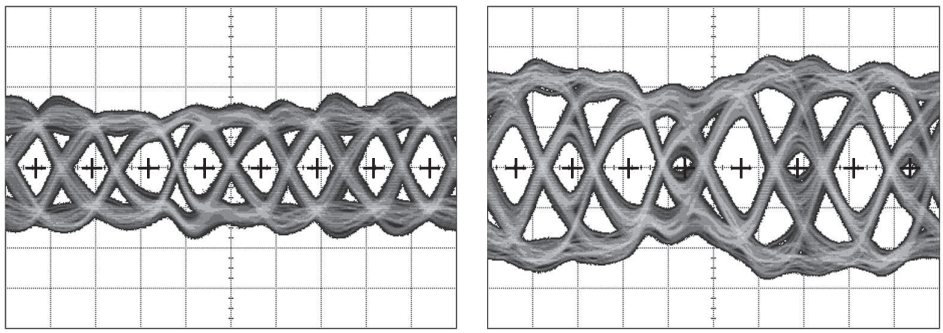


**Figure 14.33.** An *eye diagram* is formed from persistent 'scope captures of the voltage on a single data channel, triggered by the clock (whether recovered by the receiver or transmitted along with the data). This actual 'scope capture shows the signal at the far end of a 60 cm length of 0.085″ semi-rigid coax, driven by an 11.2 Gb/s (!) pseudorandom data stream. Left: with transmit equalization – the open "eyes" indicate adequate signal margin at the equally spaced clocking points (indicated with a "+" symbol). Right: without transmit equalization – a good example of a bad eye. Horizontal: 71.4 ps/div; vertical: 125 mV/div, differential. Oscilloscope bandwidth should extend to the 3rd (or even the 5th) harmonic of the clock frequency to display an accurate eye diagram. (Courtesy Hayun Chung.)

links are less expensive, with their small connectors and cabling.

Note that these serial links, though lacking address lines, can act logically like classical addressed buses, by streaming addressing bits along with the data. Examples are USB, FireWire, SAS (serial attached SCSI), and eSATA (external SATA).

Finally, serial protocols are commonly used for communication between ICs; these have names like SPI (Serial Peripheral Interface), $I^2C$ (Inter-IC), and JTAG (Joint Test Action Group).

Table 14.3 provides a helpful (we hope) organization and summary of the characteristics of commonly used data buses and links. In this section we'll look at these buses and data links, with examples to put some meat on the bones.

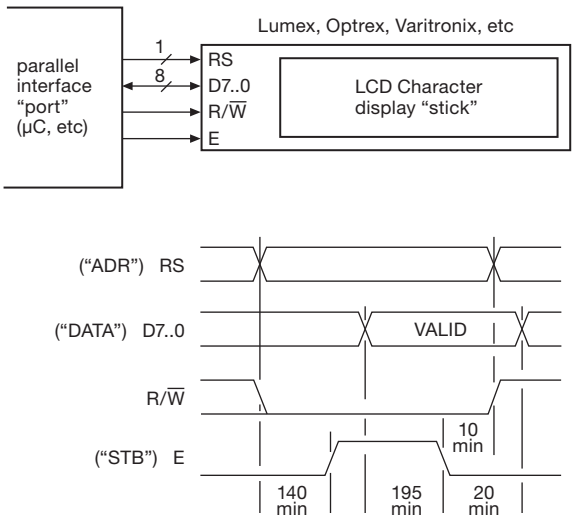



**Figure 14.34.** LCD module with parallel interface, with WRITE-cycle timings (in ns). The READ cycle is similar, but with R/W′ asserted HIGH (and D7..0 asserted by the LCD).

## *14.6 Parallel buses and data links*

### 14.6.1 Parallel chip "bus" interface – an example

The familiar LCD "character displays" that you see on many devices originated with a simple parallel bus, which has become a standard (they now come also in serial bus variants). It is very simple: 8 DATA lines (D7..0), 1 (!) ADDRESS line ("RS"), a R/W′ line, and a data STROBE ("E"). The DATA lines are bidirectional; the other three are one-way, as shown in Figure 14.34. The 1-bit RS (address) selects either the display's internal *instruction* register (adr=0) or its *data* register (adr=1). We saw these earlier, in Chapter 12 – see, for example, Figure 12.78.

The operation is simplicity itself: a character is written to the display by setting its target address to the data register (RS=1=HIGH), asserting R/W′ LOW, then putting its *character code byte* on the DATA lines and pulsing the strobe line E. The display is then "busy" (which you can find out by doing a READ from address 0: the BUSY flag is returned as bit D7); you can send the next character when BUSY becomes false.

That's how you send characters. You can also do things like clearing the display, advancing the cursor, or determining where successive characters are displayed, by writing *command bytes* to the instruction register (adr=0). For example, writing 01h clears the display; writing 10h moves

## Table 14.3 Common Buses and Data Links[a]

| | Par/Ser | PP/MD[b] | Mb/s (max)[d] | Devices per ch | Address bits | Length max (m) | Hot swap? | Used for | Comments |
|---|---|---|---|---|---|---|---|---|---|
| ***computer internal*** | | | | | | | | | |
| **PC104/ISA** | P8,16 | MD | | - | 20, 24 | - | N | peripheral cards | original IBM PC/XT (ISA), in PC104 format |
| **PCI** | P32,64 | MD | 1000, 2000 | - | 32 | - | N | peripheral cards | obsolescent |
| **PCIe** | P1,2,4,8,16 | PP | 4000 per "lane" | 1 | - | - | N | graphic & peripheral cards | multiple serial unidirectional lanes[e] |
| **IDE/PATA** | P16 | MD | 1000 | 2 | 2 | 0.45 | N | hard disk and optical drives | obsolescent |
| **SATA** | S | PP | 1500, 3000, 6000 | 1 | - | 1 | Y | hard disk, ssd, & optical drives | hot-swap; also external: eSATA |
| **SCSI** | P8/16 | MD | 320, 2500 | 8/16 | - | 12 | note f | hard disk and tape storage | ultra2/ultra-320; also external |
| **SAS** | S | PP | 3000, 6000 | 4 | - | 8 | Y | hard disk drives | "serial attached SCSI" |
| ***computer peripheral (external)*** | | | | | | | | | |
| **4-20mA** | S | PP | 110 baud | few | - | 10k | Y | slow devices, "legacy" | "current loop," robust in noisy environments |
| **RS-232C** | S | PP | 0.1 | 1 | - | ~30 | Y | instruments, computer port | "COM" ports; USB-to-serial adapters |
| **RS-485** | S | MD | 0.1, 10 | 32 | - | 1000, 10 k | Y | process control | see graph, end Chapter 12 |
| **parallel (printer)** | P8 | PP | 2.5 | 1 | - | 10 | Y | printers; obsolete | original printer port; USB-to-parallel adapters |
| **GPIB** | P8 | MD | 8 | 15 | - | 20 | Y | test/measurement insts | fat cable! |
| **SCSI** | P8,16 | MD | 320, 2500 | 8/16 | - | 12 | N | hard disk and tape storage | ultra2 / ultra-320; also internal |
| **eSATA** | S | PP | 3000 | 15 | - | 2 | Y | hard disk and optical drives | hot swap; external SATA |
| **Firewire** | S | MD | 480,800,(3200) | 63 | 64 | 10 | Y | disk storage, live video | full duplex, repeater topology, IEEE-1394 |
| **USB 1.1** | S | PP | 12 | 127 | 11 | 5 | Y | computer perhiperals | half-duplex, tiered star topology |
| **USB 2.0** | S | PP | 400 | 127 | 12 | 5 | Y | ...including hard drives, etc | |
| **USB 3.0** | S | PP | 4800 | 127 | - | 3 | Y | ...including solid-state drives | full-duplex on added SuperSpeed pairs |
| **Ethernet** | S | PP | 10, 100, 1000 | 1 | 48 | 100[h] | Y | computer network | original coax was multi-drop |
| **WiFi** | S | PP | 6 to 54, to 600[i] | 1 | 48 | 180 | Y | wireless ethernet | IEEE 802.11a, g, n |
| **Bluetooth** | S | PP | 1, 2, 3 | 8 | 48 | 10 | Y | wireless perhiperals | range: 1m at 1mW to 100m at 100mW |
| **Zigbee** | S | MD | 0.045, 0.25 | 240[j] | 64 | 30 | Y | small nearby wireless devices | 15-hop mesh network, IEEE 802.15.4 |
| **CAN** | S | MD | 1.0, 0.01 | 64 | note k | 1k, 40 | Y | auto, factory floor, lab | 8-byte payload per packet |
| **LIN** | S | MD | 0.02 | 16 | note l | 40 | Y | automobile | one wire, sub-network under CAN |
| ***inter-chip*** | | | | | | | | | |
| **parallel**[c] | P4,8,16 | MD | to 3000 | NA | - | - | - | general purpose, fast | |
| **LVDS** | S | PP | to 6000 | NA | - | - | - | IC to IC, backplane, fast | |
| **I²C / SMBus** | S | MD | 0.01,0.1,0.41,3.4 | 112 | 7 | - | - | IC to IC, addressable | 2-wire bidirectional, with handshake and adr |
| **SPI** | S | MD | >20[g] | 1 | - | - | - | simple IC to IC | 4-wire, full-duplex, no handshake, no address |
| **JTAG** | S | PP | ~100 | many | 7 | - | Y | diagnostic, program loading | IEEE 1149.1 |
| **1-wire** | S | MD | 0.1 | many | 64 | 100 | Y | sensors | Dallas; power and data on one line! |

**Notes:** (a) there are more! these are commonly used, and likely to endure. (b) PP= point-to-point, MD = multidrop. (c) generic, no standards. (d) assumes no overhead.
(e) PCIe uses multiple parallel "lanes" of unidirectional serial links; the number of lanes is written "Xn": X1, X2, X4, X8, X16. Each lane accommodates 4 or 6 Gb/s.
(f) yes for SCA-2 connectors. (g) chip dependent, no established standard. (h) 100base-Tx. (i) four 64-QAM spatial streams, 40 MHz channel width.
(j) up to 65534 is possible. (k) with DeviceNet protocol layer. (l) communication initiated by master.

the cursor one character to the left; and writing 06h specifies that each successive character is written to the right of the previous character. The same protocols are used in a compatible display variant that substitutes a handsome "vacuum-fluorescent" technology. See §12.5.3; or take a look at §15.10.4 to see a frivolous example.

### 14.6.2 Parallel chip data links – two high-speed examples

The LCD hardly needs the extra speed of a parallel bus; in fact, its interface permits byte transfers to be carried out as two successive half-byte (4-bit "nybbles") transfers. But sometimes you really need the speed, for example with a really fast digital-to-analog converter like the one shown in Figure 14.35. The 8-bit AD9748 DAC converts at rates up to 210 Msps (megasamples/sec), so you've got to feed it new bytes every 5 ns. The chip obliges by having a byte-wide input port, combined with a differential clock input that accepts high-speed LVDS (low-voltage differential signaling). Unlike the LCD, this is not a *bus* in any sense, because there's no bidirectional communication and no addressing (not even a chip select or enable pin, as with the LCD). Note the speedy timing, with setup and hold times of 2.0 ns and 1.5 ns.

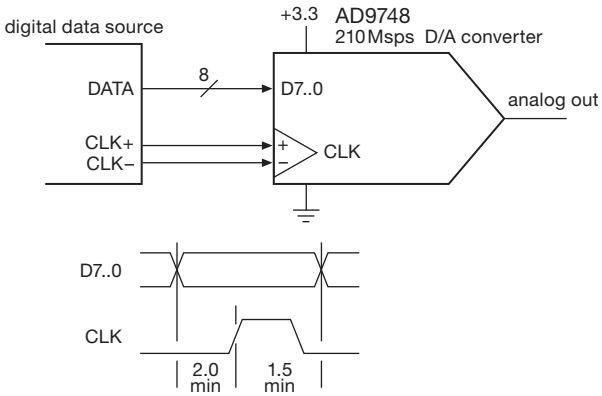



**Figure 14.35.** Fast parallel data link to a 210 Msps DAC.

Figure 14.36 shows another example with similar speed requirements – a video encoder chip that can handle the full bandwidth demands of high-definition video – but additionally with the need to specify literally hundreds of associated video parameters (such as video input and output formats, color and contrast corrections, test patterns, and so on). So, it has a 16-bit wide parallel input port, with frenetic nanosecond-scale setup and hold times. But it also has a leisurely serial port, which can accept either the $I^2C$ or SPI serial protocols and which provides access to the ∼250 internal byte-wide configuration registers for setup. This hybrid approach takes advantage of the flexibility and compactness of a serial bus, while at the same time using a dedicated parallel input port to accept a firehose of digital video data.

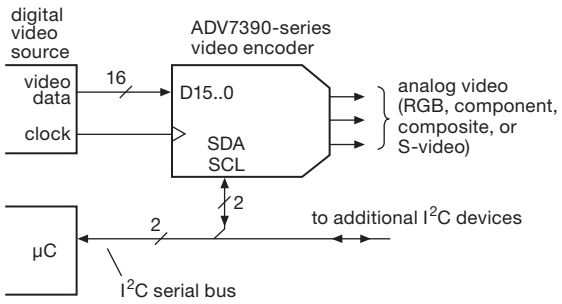



**Figure 14.36.** Video encoder with fast 16-bit parallel video-data input and 2-wire $I^2C$ serial bus configuration port.

We discuss serial buses and data links in §14.7, below.

### 14.6.3 Other parallel computer buses

As we remarked above, the buses within computers that are used to move data to peripherals and to memory have evolved, and continue to evolve. Contemporary buses like PCI and PCIe move data quickly (1 Gb/s to 32 Gb/s), with widths of 16 to 64 bits. PCIe ("PCI express") is actually a parallel/serial hybrid, and is emblematic of a shift toward *serial* data communication: the PCI that it replaces used a conventional bidirectional multidrop parallel data path (32 or 64 bits wide, with 32 address bits), whereas the newer PCIe uses 1 to 16 "lanes" of unidirectional point-to-point serial-link pairs, each of which runs at a speedy 2.5 Gb/s (PCIe v.1), 5 Gb/s (v.2), or 8 Gb/s (PCIe v.3). Timing is carried with the data, on each serial link, by coding the data so that the recipient can carry out "clock recovery" without separate clocking or strobing signals.[44] This is necessary for high bitrate communication; otherwise timing *skew* (the *difference* in propagation times of the clock and the several data lines) limits the maximum bit clocking rate.

Parallel data buses exist also within the silicon chips themselves: all processors contain internal data paths to send data among the registers and arithmetic units, to and from the on-chip memory caches, between the processor and external memory, and so on; you'll see names like

[44] PCIe uses "8b/10b" encoding, with each 8-bit byte sent as a 10-bit symbol, to generate a dc-balanced and "run-length-limited" bitstream in which there are never more than five consecutive 1s or 0s.

AMBA and Wishbone. As processors grow in speed, their external buses have followed; the current configuration is a fast and wide "Front Side" bus (FSB), which requires support chips ("Northbridge" and "Southbridge," likely to merge into a single unified support chip) to split off separate buses for memory, fast peripherals (PCIe bus), and slower peripherals (SATA, PCI, USB, Ethernet).

Another trend in contemporary computer designs is the integration of more functions, and more complexity, within the processor chip itself. Thus we have the delightful and inexpensive micro*controllers*, with their spectrum of on-chip "peripherals," ready to drop into a circuit or instrument with hardly a thought of buses or interfacing. Embedded microcontrollers are the subject of the next chapter.

Yet another approach dispenses with the dedicated processor altogether, exploiting the power of configurable logic (Chapter 11) in massive FPGA "system-on-a-chip" (SoC) devices. These permit flexible system design (including on-chip "soft" processor cores, internal buses, memory, and peripherals), with plentiful wide-bandwidth I/O pins. An example is the Xilinx Virtex series of FPGAs, topping out at several million flip-flops, tens of MB of on-chip RAM, and more than 500 LVDS differential ports. In additional to soft configurable SoCs, there are application-specific standard parts (ASSPs, see §11.4.2) for many consumer applications. For example, there's Broadcom's BCM7405 "Multiformat HD Digital Video/Audio SoC for Satellite, IP and Cable STBs with Picture-in-Picture" (the title says it all[45]).

### 14.6.4 Parallel peripheral buses and data links

Although the industry is moving from parallel to serial buses, there are still a few parallel buses in use.

#### A. PATA (IDE)

The 16-bit-wide PATA (parallel ATAPI, also called IDE) was for many years the standard connection to internal hard disk and optical (CD, DVD) drives. The wide 40-wire ribbon cables and connectors are somewhat cumbersome, and even after the introduction of 80-wire cables (adding interleaved ground lines to improve signal integrity) the data rates topped out at 133 MB/s (∼1 Gb/s). PATA is obsolete, replaced with the serial SATA format.

#### B. SCSI

The 8/16-bit parallel small computer system interface dates from the 1980s, with enhancements in speed and signaling format (originally 5 V single-ended, with low-voltage differential added subsequently[46]), and was until recently the preferred interface for high-performance disks and tape storage devices. Internal SCSI buses use 68-wire ribbons with dual-row socket headers; external SCSI uses shielded multipair cable with high density 68- or 80-pin connectors.[47] Although SCSI bandwidth improved with each generation, it has been overtaken in speed and convenience by the serial formats: SAS (serial-attached SCSI), and SATA.

#### C. GPIB

The IEEE-488 general purpose interface bus was originated by HP for control and readout of test and measurement equipment; many instruments (e.g., from Agilent, Keithley, National Instruments, and Stanford Research Systems) continue to support GPIB control. It uses a robust stackable 24-pin connector and permits lengths to 20 meters. You can get GPIB adapter cards for the PCI and PCIe computer buses; or you can get adapters such as Ethernet-to-GPIB and USB-to-GPIB.

#### D. Printer ("Centronics") port

Last (and least) is the parallel "printer port" (LPT port), which was standard for two decades, beginning with the original IBM PC, where its I/O address was 378h (and still is). It was also used for external modems and for software "dongles." The original parallel interface was unidirectional (data OUT, with a few handshake lines), but extensions of the standard (IEEE 1284: ECP, EPP) made it bidirectional, faster, and more bus-like. It is quite thoroughly

[45] But in case you want to hear more, the product page has this to say: "The BCM7405 is a high-performance, high definition (HD) satellite, cable, and IP set-top box DVR System-on-Chip solution. It combines a fast 1100-DMIPS MIPS32/MIPS16e-class CPU, high-speed graphics processing, including video scaling and motion adaptive deinterlacing, a very flexible data transport processor, an MPEG-4/VC-1/MPEG-2/AVS-compliant video decoder, a programmable audio decoder, six video DACs, stereo high-fidelity audio DACs, dual Fast Ethernet ports, one with integrated PHY, triple USB 2.0, a PCI 2.3/Expansion Bus, a high-speed 400-MHz DDR2 memory controller, and a peripheral control unit that provides a variety of set-top box control functions. Integrated in 65-nm technology, it offers one of the highest levels of single-chip system performance available for STB applications."

[46] Differential signaling requires twice as many wires, but it is superior in terms of low crosstalk, excellent noise immunity, and far smaller ground and supply noise (because of balanced transitions); these improved characteristics let you use relatively small voltage swings, with correspondingly smaller drive currents.

[47] The miniature high-density connectors that were developed for advanced forms of parallel SCSI have come to be very useful in other fields, such as I/O connectors for computer DAQ boards for laboratory use by National Instruments and others. These are made by HRS Hirose, Honda Connectors, and others.

obsolete: printers use USB or Ethernet connections. Computers now rarely include a parallel port; if you need such a connection, you can get USB-to-parallel adapters.

That said, however, the parallel port (emulated with a USB adapter, say) is the simplest way to wiggle some bits from a PC. If you're running some version of Windows, you'll need a driver[48] that lets you get at the hardware (these operating systems don't let you get at hardware ports directly, as you were able in the good ol' days of DOS). Then you can write really simple code, like this example of Python[49] running in terminal mode:

```
>>> import parallel
>>> p = parallel.Parallel() # open LPT1
>>> p.setData(0x55)
```

You can use a compiled language, instead; some folks we know are fond of PowerBASIC, in which you can use in-line assembly language in a BASIC subroutine to send a value out to an addressed port. It looks like this:

```
Sub PortOut(ByVal PortNum as word, Byval Value
as byte)
   ASM MOV AL, Value
   ASM MOV DX, PortNum
   ASM OUT DX, AL
End Sub
```

## *14.7 Serial buses and data links*

Serial buses and data links have several important advantages, two of which we saw earlier: (a) the convenience of fewer wires in the cable and connector (think of a skinny USB cable, compared with the elephant's trunk of GPIB or SCSI), as well as fewer pins on the driver and receiver chips; (b) high intrinsic bitrate, because of absence of timing skew (self-timed via clock recovery) and clean line termination (if point-to-point). In addition, (c) a one-wire serial link is easily conveyed by optical fiber or by wireless transmission. And, if you want parallel bits at either end, there are chips generically known as SERDES (serializer–deserializer, pronounced "ser′-deez"), which convert a serial stream to parallel, and vice versa (see also §12.8.4 and §12.10.3). As examples of the latter, FTDI Ltd. offers the popular FT245 and FT2232, which convert between a relatively low-speed USB and a simple byte-wide parallel port, with a built-in first-in, first-out (FIFO) buffer; high-speed examples include the DS92LV18 18-bit SERDES (with speeds to 1.2 Gbps), or the generic SERDES used in the PHY (physical layer – i.e., the driver–receiver–switch ICs) of gigabit ("1G") and 10 gigabit ("10G") Ethernet links.

In the subsections that follow we describe most of the important themes in serial links, with examples from those in common use. As with the parallel buses, we look first at internal serial protocols (chip-to-chip, and within an instrument), then at the external serial buses. Within these categories we've ordered them roughly by increasing complexity, e.g., going from the simplest (and slow) 4-wire clocked link (SPI) to the complex (and fast) 1-wire 8b/10b-coded clock-recovery links used in SATA and PCIe.

See the next chapter for specific chip suggestions to implement these protocols with a microcontroller.

### 14.7.1 SPI

The serial peripheral interface (SPI) was introduced by Motorola, and is widely used for communication *between ICs* (for which the other popular standard is $I^2C$, discussed below).[50] It is organized as a master–slave protocol (like the PC104 bus), but uses only 4 wires (Figure 14.37): one clock, two data lines (one in each direction), and a chip select. They are named SCLK, MOSI (master out, slave in), MISO (master in, slave out), and $\overline{\text{SS}}$ (slave select; active LOW).[51] There are several ways of hooking things up, but most commonly the scheme of Figure 14.38 is used: the clock and data lines are bused to all slave chips

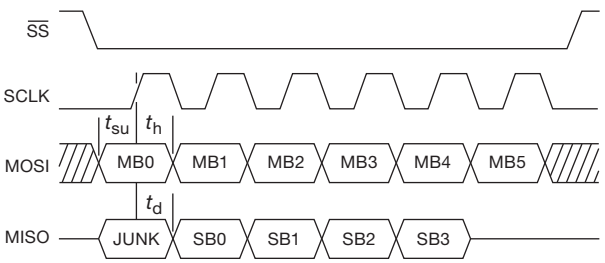



**Figure 14.37.** A typical SPI protocol, with bits in both directions clocked on the rising edge of SCLK. With SPI, the quantity and meaning of the bits are specific to the slave device, which here accepts 6 input bits from the master (MB5..0), and simultaneously outputs 4 other bits (SB3..0).

[48] Examples are DirectIO.exe, or InpOut32.dll.

[49] "setData(value)" is one of several bit-banging function calls in the pyParallel API; the documentation describes it as "Apply the given byte to the data pins of the parallel port."

[50] You see SPI and $I^2C$ on chips like sensors, converters, nonvolatile memory, analog switches, and digital potentiometers, to be controlled from a microcontroller, microprocessor, or other digital link.

[51] Sometimes with alternative naming: SDI, DI, or SI for data IN, and likewise for data OUT, with the signal names corresponding to the direction of data *at that IC*. For example, the MOSI pin on the master would connect to the DI pins on the slaves.

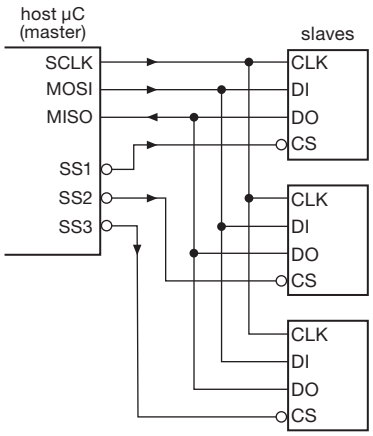



**Figure 14.38.** The common bused SPI configuration: clock and data lines are shared, with individual SS′ (slave-select) lines asserting the chip-select inputs of the corresponding slaves.

("multidrop"), with a separate dedicated select line to each slave.

The master controls all transfers, first asserting SS′ for the chosen slave IC (with the SCLK line in its resting state), then generating successive clock pulses, each of which enables a bitwise transfer of data on MOSI and MISO for that chip only. There is no fixed protocol for what the data represents, how many bits are to be sent, etc. What happens, instead, is that a particular chip specifies the meaning of the serial bits sent to it and of the bits it simultaneously sends back.

To give an example, the AD7927 is a 12-bit ADC of modest speed (200 ksps), with a built-in 8-input multiplexer, and with a SPI serial port (Figure 14.39); the latter both controls the conversions (e.g., selecting the input channel, voltage range, output coding, etc.) and also delivers the converted digital outputs. This particular chip loads the first 12 input bits (after SS′ assertion, which also initiates conversion) into its control register (ignoring later bits), and it simultaneously sends back the result of the previous conversion as a 16-bit string, as shown in Figure 14.39.[52] See Figure 15.21 for some examples of SPI peripheral chips that are well suited to microcontroller applications.

### A. Some comments

The SPI protocol is free form in its content (how many bits are sent, and what they mean), as this example illustrates. SPI doesn't have any intrinsic addressing to designate where data is going within the target chip, so the usual scheme is to send a string of bits that gets shifted into sequential internal bit locations, with the datasheet defining how they sort out internally (the popular $I^2C$ alternative, below, takes a different approach). Some chips may be write-only (e.g., an LCD with serial input), others read-only (e.g., the Maxim MAX6675 thermocouple-to-digital-converter chip, with SCLK, MISO, and SS′ only: you don't tell *it* what the temperature is, it tells *you*). Some chips may invert the clock polarity, and also which edge clocks the data (this produces four possibilities, known as SPI *modes*; the above illustration used mode-2). SPI has simple timing, and full-duplex data transfers (i.e., in both directions simultaneously and independently); it has no required "handshake": a master can send lots of data – to a nonexistent chip!

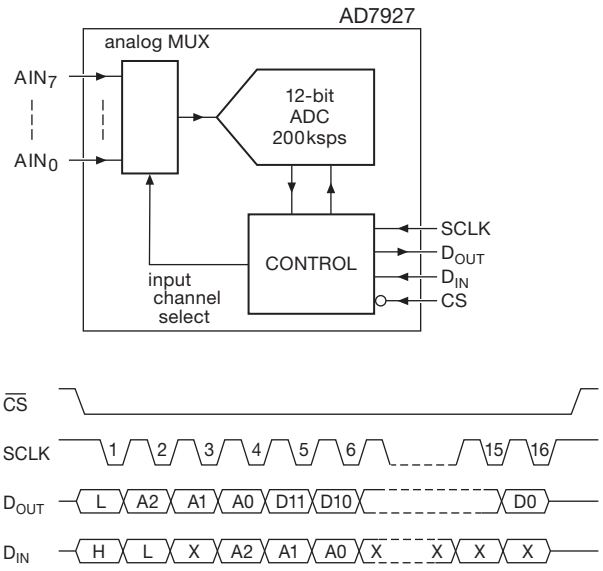



**Figure 14.39.** An 8-channel ADC with SPI control and readout. The protocol shown is the simplest of several allowed modes: the master's CS′ starts a conversion, whereupon the slave outputs both the channel address (3 bits) and the converted value (12 bits); the master simultaneously sends the address of the next input channel to be converted.

Because SPI (and SPI-like variants) do not conform to a well-defined standard, you have to read carefully the datasheet specifications for each interfaced chip. You'll discover, in addition to the polarity modes already mentioned, that the maximum (and minimum!) clock speeds can range from a few kilohertz to many megahertz. The AD7927, for example, specifies $f_{SCLK}$=10 kHz (min), 20 MHz (max). With several SPI chips in a system, you

[52] The AD7927 can do numerous additional tricks, such as cyclically converting a prescribed arbitrary sequence of input channels; you program that sequence by loading a "shadow register," which you access through two bits of the control register.

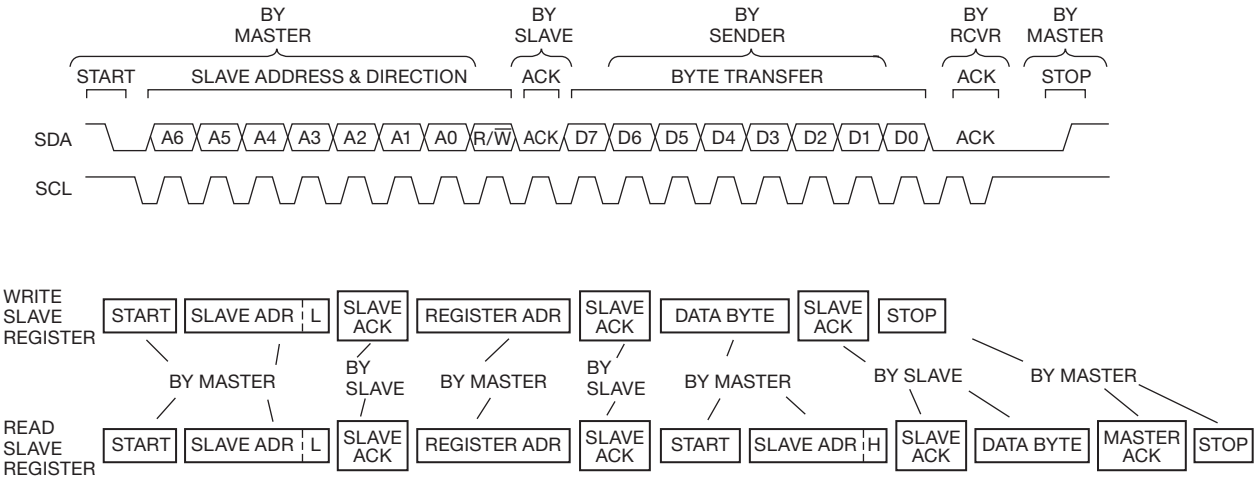



**Figure 14.40.** The I$^2$C 2-wire protocol (upper waveform pair): all transfers are in 8-bit groups, with a 1-bit acknowledgment (ACK). The first byte after START is always the master's assertion of the slave address (A6..0) and direction (R/W′); subsequent bytes flow from sender to receiver (depending on that first R/W′), asserted by sender and acknowledged by receiver. The master can access registers within a slave device by sending their internal address as a data byte, as shown in the lower block diagrams; both a WRITE and a READ are illustrated, the latter requiring a "repeated START" to create a READ after writing the register's address in the initial transaction.

may have incompatibilities among them that force you to write code to assert and disassert port bits "by hand" (this is called *bit-banging*, rather than using the microcontroller's built-in SPI interface).

A widely used alternative to SPI is the I$^2$C peripheral interface, discussed next.

### 14.7.2 I$^2$C 2-wire interface ("TWI")

The Inter-Integrated-Circuit (IIC, I2C, or I$^2$C) serial interface bus was originated by Philips (now NXP), for communication between chips.[53] It differs from SPI in several ways: (a) it uses only 2 wires, which are bused to all slave ICs (there is no separate chip select like the SS′ that is used in SPI); (b) addressing is sent (first) on the same line as data is sent or received; (c) the bus is "half-duplex" – that is, data can move in only one direction at a time (the direction is specified by a bit following the address); (d) although I$^2$C is a master–slave architecture (like SPI), any device on the bus can become master when the current master relinquishes control (by sending the stop bit that terminates its session with a particular slave).

Figure 14.40 shows the protocol. The 2-wire I$^2$C bus consists of a clock line (SCL) and a data line (SDA), both with resistive pullups to $V_+$. SCL is asserted by the master, whereas SDA is bidirectional: it is asserted by the master to specify the slave's address (7 bits) and the direction of transfer (1 bit); the slave then sends an acknowledge (ACK) bit, following which one or more data bytes move from master to slave, or from slave to master (always clocked by the master), depending on the direction-of-transfer bit that was specified initially with the slave's address. The session ends when the master sends a stop bit following the last byte transferred.[54] START and STOP commands are created by violating the normal convention of "data can change only during clock LOW."

To give an example, the AD7294 "12-bit Monitor and Control System with Multichannel ADC, DACs, Temperature Sensor, and Current Sense" is a do-everything chip for applications such as automobiles, industrial controls, and cellular base stations (Figure 14.41). It isn't the fastest kid on the block – a mere 300,000 conversions per second on its ADC – but it will keep an eye on the whole shop, reporting back to the mother ship via an I$^2$C port. The 44-page datasheet tells you how to communicate with its

[53] The closely related SMBus enforces tighter standards, both in protocol and in its electrical signaling.

[54] You can think of this whole process as the serial analog of a PC104/ISA data transfer: in the latter, the master asserts address on the A19..0 lines, and direction on the IOW′/IOR′ lines. If a WRITE, the master asserts the data on the bidirectional D7..0 lines; if a READ, the addressed slave asserts the data on those same line. In either case the transfer is clocked by the master's IOW′/IOR′ strobe. In I$^2$C the same steps take place, but in serial sequence on the single bidirectional SDA data–address–direction line, clocked by the single unidirectional SCL clock line.

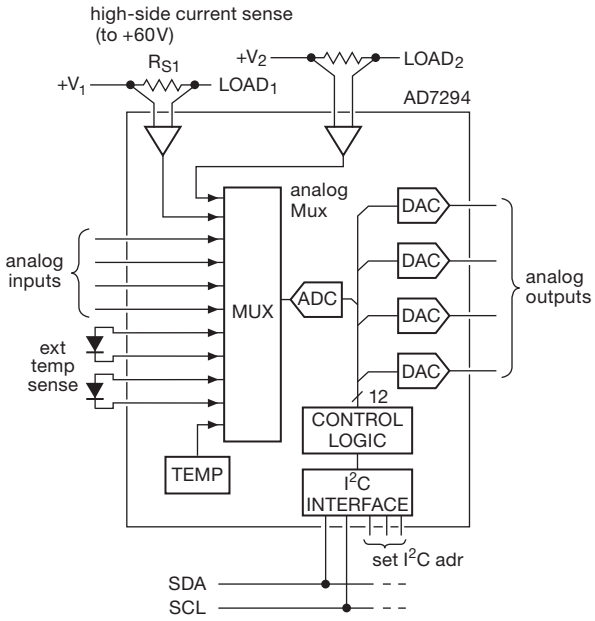



**Figure 14.41.** A multifunction monitor chip, rich with features but of modest speed only, with $I^2C$ control. The master can reach each of the 40 internal registers by sending the corresponding register address as the second byte of a transmission, as shown in Figure 14.40.

40 internal registers, with amusing names like "AlertRegisterA(R/W)" and "$DATA_{HIGH}RegisterT_{SENSE}INT$(R/W)." See Figure 15.22 for some examples of $I^2C$ peripheral chips that are well suited to microcontroller applications.

### A. Some comments

The 2-wire $I^2C$ protocol is well defined and is economical of wiring, particularly when you need to include many chips on the bus, because the 2 wires carry all data, addressing, and clocking. In addition, it allows the slave to slow down the master by doing "clock-stretching" (by holding SCL LOW; this is called *flow control*), and it allows multiple bus masters. It is particularly well suited to tasks in which you want to take aim at a particular register in a chip that is endowed with many; with the AD7294, for example, you would have a 3- or 4-byte transaction: the first byte is the chip's bus address, the second is the address of the chip's internal register, and the last byte (or two) is a write (or read) to (or from) that internal register.

### B. Comparison with SPI

When compared with SPI, however, $I^2C$ is a more complex protocol, and not as well suited to steady high-rate streaming of data. The flexibility of multiple bus mastership brings with it the problems of contention and arbitration. You have to give multiple devices unique addresses, which is commonly handled by including some dedicated pins to select among a built-in set (e.g., the AD7294 has 3 pins by which you can select any address from 61h to 7Bh), thereby defeating some of the advantage of low pin count. And the flexibility of addressing, bidirectional data, and bus mastership complicates debugging, as compared with the dead-simple SPI protocol.

Which to use? The choice is usually determined by the peripheral chip, which usually supports only one protocol or the other, whereas most microcontrollers include hardware support for both SPI and $I^2C$ (and if they don't, you can always do bit-banging in software).

## 14.7.3 Dallas–Maxim "1-wire" serial interface

The ultimate in reducing the number of wires is achieved in the 1-wire™ (plus ground) interface devised by Dallas (now merged with Maxim).[55] The single wire carries serial data and addresses, and also *power!* The way it does all this is by having data bits sent, bidirectionally, as brief pulses to ground, with each slave device having an on-chip capacitor to retain power. The goal is simplicity in interconnection to devices like temperature sensors, memory, converters, battery management, and so on (Figure 14.42). With only ground and data, the devices can be packaged in what Maxim calls iButton™, which looks just like a coin cell battery.

The protocol goes like this: multiple slave devices all bridge the common data line and ground, controlled by a master device (microcontroller or other digital interface). The line is pulled up to +5 V, which powers the slave devices and permits any device to assert a momentary LOW. The master initiates all transactions, asserting addresses and then either sending or receiving data. Data is encoded as pulse widths: a short pulse ($<15\,\mu$s) for 1, and a long pulse ($60\,\mu$s) for 0. If the master is sending, it simply asserts the bits with the corresponding pulse widths; if it is receiving, it sends the short pulse corresponding to a 1, and the addressed slave responds by either doing nothing (releasing the line to +5 V, therefore signaling a 1), or by holding the line low for $60\,\mu$s, creating a long pulse (therefore signaling a 0).

Each 1-wire device has a unique 64-bit address, given to it at the time of manufacture, which includes a byte indicating the type of device. The master resets the slaves

[55] Plenty of details in their Application Notes AN147, AN148, AN155, AN159, AN244, AN1796, and AN3358.

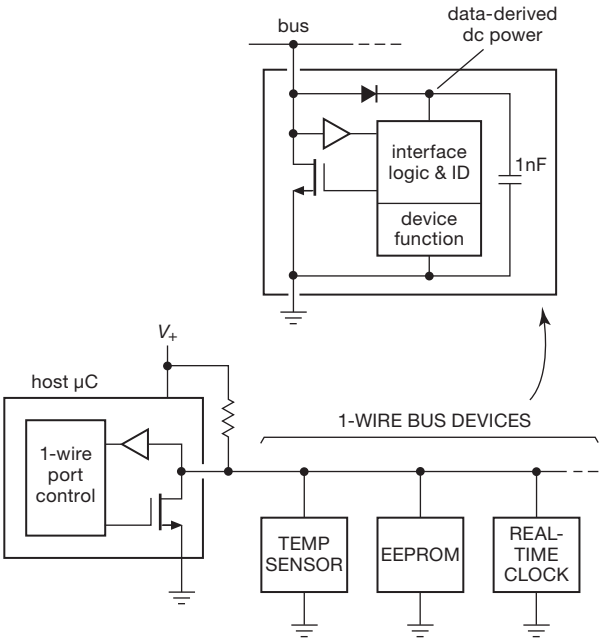



**Figure 14.42.** The 1-wire™data bus originated by Dallas–Maxim. An internal capacitor maintains slave power during the brief LOW data pulses.

with a long pulse (all "pulses" are LOW, i.e., to ground), then queries the bus to learn the attached addresses. The master can send broadcast messages to all attached devices, or it can carry out transactions with particular devices, according to their unique addresses. As with the $I^2C$ bus, that address is sent (by the master) on the data line, followed by the data to be sent or received.

The Dallas–Maxim 1-wire bus can be used for connections to sensors, etc., outside of an instrument. It's normally limited to a maximum network distance of 30 meters, but it can work up to 500 meters with an appropriate driver (see AN244); and extensions by Ethernet are possible.

### 14.7.4 JTAG

A useful chip interface, with an interesting history, is the JTAG (Joint Test Action Group) standard, also known as "boundary scan," or IEEE 1149.1. It was devised in the 1980s to deal with the thorny problem of testing for component or connection failures in circuit boards using the then-new surface-mount and multilayer technologies; these made it increasingly difficult to get at the IC pin connections, not to mention the innards of the chips themselves. As such, it provides a way to look at the registers and data paths within the chips, using a simple 4-wire serial bus; that lets you pinpoint what is broken without having to unsolder multiple parts to get at the problem.

JTAG turned out to have additional utility as a general-purpose serial interface, akin to SPI and $I^2C$, because of the way it accommodates an internal state machine in the target device for controlling data transfers. Among other applications, it is now widely used to program and debug microcontrollers in-circuit (e.g., ARM and AVR series), and to program cPLDs, FPGAs, and nonvolatile memory (flash, EEPROM), as well as other chips that include nonvolatile memory (e.g., the Maxim DS4550, where it coexists with an $I^2C$ port).

The basic scheme is shown in Figure 14.43. A clock line (TCK) and a mode-select line (TMS) are bused to all devices; the other two lines are daisy-chained through the chips, and are called data in (TDI) and data out (TDO).[56] TDI bits are clocked in on the rising edge of TCLK, and TDO bits are clocked out on the falling edge. The mode bits, which define what TDI and TDO will be doing, are clocked in on TCLK's rising edge. (There is also an optional bused reset line, not surprisingly called TRST.) The bus runs at speeds in the range of 1–100 Mb/s, depending on the particular target chip(s).

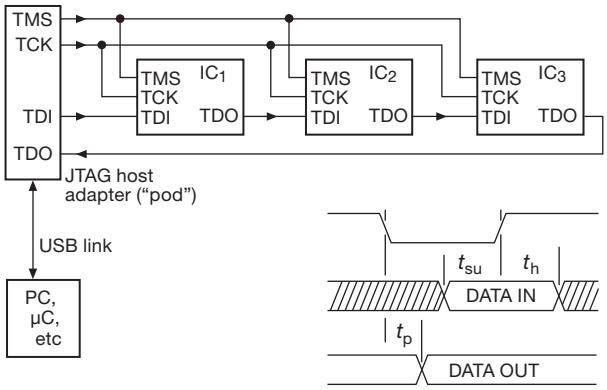



**Figure 14.43.** The JTAG "boundary scan" interface uses a pair of unidirectional bused lines (clock: TCK; mode select: TMS), and a daisy-chained data line passing through each slave (TDI, TDO).

Many PLDs and microcontrollers now use a JTAG port for programming; typically you get a programming "pod" that connects to a host computer via USB, and has a socket header to mate with a pin header on the target device's board. Sadly, the header configuration has not been standardized, although it's easy enough to make an adapter. However, the software that the chip manufacturers provide

[56] The letter "T" preceding each signal stands for "test," an indication of what the originators had in mind.

usually forces you to use their specific pod when talking to their chips. You use such vendor-supplied software, or open-source software, to load the compiled code into the target, and to carry out in-circuit real-time debugging, both via the same JTAG port.

### 14.7.5 Clock-be-gone: clock recovery

The SPI, $I^2C$, and JTAG serial links above, each use a separate clock line to time the data bits (SCLK, SCL and TCK, respectively). However, it is possible to do *clock recovery* from the data itself, if you arrange things correctly. This not only reduces the number of lines, it also permits higher ultimate data rates, because there is no issue of timing skew between clock and data. The remaining internal serial links (and, later, the external serial links) employ clock and data recovery (engineers' lingo: "CDR") from the transitions on the data line(s). The reader may wish to look ahead to §§14.7.9–14.7.12 for helpful information on how coding is used to make this work.

### 14.7.6 SATA, eSATA, and SAS

SATA is serial-ATA, and SAS is serial-attached SCSI. These are the fast serial buses for internal and external storage (disk, tape, and optical drives); they are replacements for the obsolete ATA (retroactively called PATA, or sometimes IDE) and SCSI, respectively. They share the same connector (though SAS offers additional connector types) and are hot-pluggable. Maximum data rate is currently 6 Gb/s, with upgrade paths to 12 Gb/s. SAS has some performance features not found in SATA, and is thus aimed at server (as opposed to consumer) applications; it basically continues the (parallel) SCSI protocol, but running on a serial connection interface.

These interfaces use low-voltage differential signaling, with 8b/10b encoding and clock recovery, and allow hot-swapping (although this feature requires operating system support to work correctly).

External SATA (eSATA) is an extension of the standard, for attaching external storage devices to a computer with SATA, using the same protocols. The physical connectors are different, however, being designed for greater ruggedness and signal integrity. External devices connected via eSATA currently require separate power, a nuisance likely to be remedied by the "Power Over eSATA" initiative.

### 14.7.7 PCI Express

PCI Express ("PCI-E," or "PCIe") was introduced in 2004, as a successor to the parallel PCI bus (and its brethren, PCI-X and AGP), for interfacing to peripheral cards on a computer motherboard. It replaced the wide (32- or 64-bit) multidrop parallel architecture with a set of point-to-point serial "lanes," each lane consisting of two LVDS differential pairs (one pair for 1-bit serial communication in each direction, see §12.10.3). With multiple lanes there is parallelism (in that sense it is a hybrid), but the communication is basically serial, with 8b/10b coding and clock recovery, etc.

PCIe has blazing speed: currently 4 Gb/s *per lane* (PCIe v2.0, double that of the original version 1.1), with 1 to 16 lanes. Thus you could have up to 64 Gb/s to an x16 slot, if the hardware at both ends could handle it.

Let us take a brief aside, to wonder how we've come to this elaborate scheme, given the elegance of the traditional parallel bus with its economical sharing of data lines (as in the PC104/ISA bus). Roughly, it goes like this: in the beginning (say the 1970s) integrated circuits were not dense, and the DIP packaging limited the pin count to 16 pins (often) or 40 pins (occasionally). They also were not terribly fast, running at clock speeds of 10 MHz or so. These circumstances favored the shared parallel bus – less wiring, fewer driver–receiver pins, and still plenty of bus speed.

As chips became faster, and humans demanded more speed, the parallel buses were enhanced, with more width (the 8-bit ISA bus was broadened to 16 bits, then expanded to EISA: 32 bits), and, at the same time, with more speed. This was successful, up to a point. That point was near when the performance of the parallel PCI bus (the successor to the ISA family) had grown from its original 32 bits at 33 MHz (thus 133 MB/s) to 64 bits at 133 MHz ("PCI-X," 1064 Mb/s). A further enhancement ("PCI-X 2.0") quadrupled the speed, but was not widely adopted. Problems of timing skew and of reflections from the multidrop stubs prevented further gains from the bus. By this time (2003) computer designers saw a better way, namely the serial point-to-point PCIe architecture.

Of course, PCIe has its challenges: consider a motherboard with a couple of x16 slots (a common configuration, even in inexpensive computers). Each slot requires 32 differential pairs (64 wires), so you've got to connect to 128 wires, and deal with 4 Gb/s (actually, 5 Gb/s raw, because of the coding overhead) on each of the 64 pairs. That function (along with the memory bus) is carried out in the Northbridge chip, which has hundreds of pins and the necessary speed and complexity for the task: it even has its own heatsink. And one must not ignore the fabrication technology needed to connect all these pins: even inexpensive motherboards routinely have a half-dozen wiring layers, with traces 0.12 mm (5 mils) wide.

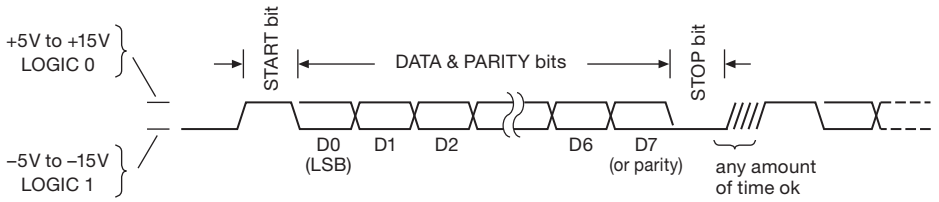



**Figure 14.44.** RS-232 serial data-byte protocol, which uses signal levels of both polarities. The two states are sometimes called *mark* (negative, logic 1) and *space* (positive, logic 0). You sometimes hear the descriptive phrases "LSB first" and "inverted levels."

In a sentence, then, the demand for greater bus performance coincided with the ability to proliferate point-to-point connections, to bring about the evolution from the elegant shared parallel bus. . . to the elegant coded point-to-point serial links!

This is clearly an area where casual circuit designers dare not tread – better just to buy some smarter person's finished design!

### 14.7.8 Asynchronous serial (RS-232, RS-485)

Turning now to *external* serial buses, we look first at the original (and long-lived) *asynchronous* serial link – used in the now-obsolete RS-232 serial port. We discussed RS-232 (and also RS-422 and RS-485) back in Chapter 12 (§12.10.3A), with emphasis on the physical layer (waveforms, noise immunity, and the like). Here we revisit RS-232, in the context of computer communication. These serial ports were popular for connection to external modems and to alphanumeric terminals like the legendary VT-100 (dismissively called *dumb* terminals). They have largely disappeared from computers, but you can get USB-to-RS-232 adapters; and you can get cards that plug into the internal bus and create a gaggle of RS-232 ports. The RS-232 designation ("Recommended Standard #232") refers to the *electrical* signaling scheme, which inconveniently uses voltages of both polarities to signal 1s and 0s (Figure 14.44; see also §12.10.4). However, the physical link need not be RS-232 – it can be the differential (and unipolarity) RS-422 or RS-485, or it can be an optical fiber (for galvanic isolation and immunity from environmental transients), or it can even be a 20 mA *current loop* (which is most often used for *analog* signaling).[57]

Whatever the physical transport medium, the signaling is straightforward: the transmitter rests in the *mark* state (logical 1), and springs to action with a START bit (logical 0, or *space*), followed (usually) by 8 data bits (which optionally may be 7 data bits plus a parity bit), followed by one or two trailing STOP bits (logical 1). The transmitter and receiver must agree on the bit rate and parity (if any); a common protocol, for example, is 9600 baud and "8N1," which means 8 bits, no parity, and one stop bit, transmitted at a rate of 9600 bits/s.[58] (Because of the overhead of the framing START and STOP bits, each byte during 8N1 transmission requires 10 bits, for a maximum net payload rate of one tenth of the baud rate, or 960 bytes/s.)

With this simple asynchronous serial coding, the receiver (whose clock runs at several times the baud rate) is triggered by the transition at the beginning of the START bit, waits for half a bit cell to be sure the START pulse is still present, then examines the bit value at the middle of each data cell (using the agreed upon baud rate interval); the STOP bit terminates the character, and is the resting state if no new characters are sent immediately. By resynchronizing on the START bit of each character, the receiver doesn't require a highly accurate clock; it need be only accurate and stable enough for the transmitter and receiver to stay synchronized to a fraction of a bit period over the duration of one character, i.e., an accuracy of a few percent.

There is one slight logical flaw in this pretty scheme, namely that a receiver may not be able to synchronize properly (that is, identify START/STOP) in an uninterrupted stream of data bytes. The best proof of this conjecture is a long sequence of the letter "U," which has the unfortunate distinction of being encoded as 01010101 (55h): plug that into Figure 14.44 (with the usual 8N1 setting) and you'll get. . . a *square wave*! A more serious flaw is the lack of standardization at the physical (electrical) level of RS-232: connector gender, hardware handshaking signals, and flavor of device ("DCE" and "DTE"; see Table 14.4 for official signal names and pin assignments). This is an eternal source of confusion because, too often, two RS-232 devices when connected together, won't work. We've all struggled with this, and readers have even complained to *us* (hey, give us a break, we didn't design RS-232; but we

[57] These alternatives permit longer cable runs (up to ∼1 km) and, in the case of RS-422/485, a multidrop bus topology.

[58] The serial asynchronous standard permits a wider range: 5 to 8 data bits, with optional parity; so it would be legal to specify 8E1, for example. In practice you rarely see anything but 8N1.

**Table 14.4 RS-232 Signals**

| Name | Pin number 25-pin | 9-pin | Direction (DTE ↔ DCE) | Function (as seen by DTE) | |
|---|---|---|---|---|---|
| TD | 2 | 3 | → | transmitted data | } data pair |
| RD | 3 | 2 | ← | received data | |
| RTS | 4 | 7 | → | request to send (= DTE ready) | } handshake pair |
| CTS | 5 | 8 | ← | clear to send (= DCE ready) | |
| DTR | 20 | 4 | → | data terminal ready | } handshake pair |
| DSR | 6 | 6 | ← | data set ready | |
| DCD | 8 | 1 | ← | data carrier detect | } enable DTE input |
| RI | 22 | 9 | ← | ring indicator | |
| FG | 1 | - | | frame ground (= chassis) | |
| SG | 7 | 5 | | signal ground | |

did at least provide a guide to "serial cables that actually work" – see Figure 10.17 in the previous edition of this book.). With finicky USB-to-RS232 emulators replacing the disappearing COM ports on contemporary computers, the problem has only gotten worse.

Figure 14.45 shows some 'scope captures of RS-232 waveforms: four single-byte captures from a stream of random[59] bytes, and a multibyte capture showing the invariant START and STOP bits.

The most common use of asynchronous serial links has been for alphanumeric data, in the standardized 7-bit ASCII (American Standard Code for Information Interchange) code of printable characters that is the norm for alphanumeric representation (Table 14.5).[60] However, any binary data can be so conveyed; you just won't be able to read it or print it directly. And serial communication via RS-232 is quite alive and well: many bench instruments include an RS-232 port for control and data transfer; and the serial format provides an easy way to communicate with microcontrollers (Chapter 15) with their (nearly universal) built-in serial ports (UARTs, universal asynchronous receiver–transmitters). See §12.10 for a discussion (with waveforms) of driver and receiver ICs.

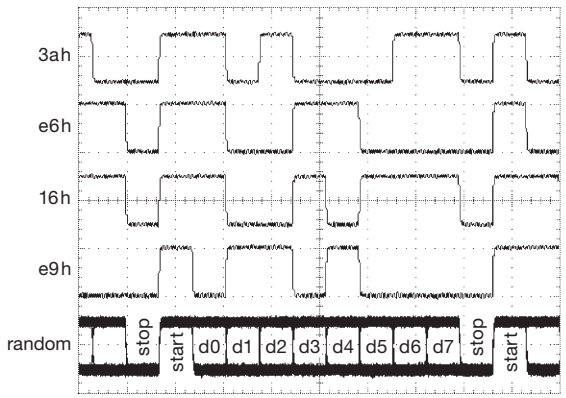



**Figure 14.45.** Captured RS-232 waveforms from a random byte generator, 8N1 at 14.4 kbaud into a 2.2 nF load. The bottom waveform is an accumulation of multiple bytes, whereas the top four are single bytes (with indicated values). Horizontal: 100 $\mu$s/div; vertical: 10 V/div.

### 14.7.9 Manchester coding

You don't need to frame data bytes with synchronizing START and STOP pulses, however, as long as you arrange

[59] *Truly* random: we fed analog noise to a comparator; see §13.14.7, and the "Museum" section on the *Numerical Recipes* Source Code CDROM (or www.nr.com).

[60] Standard ASCII has 128 characters, including the 32 "nonprinting" characters in the first column of Table 14.5. There are "extended ASCII" 8-bit character encodings that double the number of characters to include alphabetic characters from other languages, for example ë (decimal code 235), as well as symbols such as ° and ± (codes 176 and 177) and £ (code 163). A warning: there are multiple such extensions, for accommodating different language groups (some are gathered into Std. ISO-8859; others are proprietary); so they should be used with caution. Most software will accept these encodings, which you enter by holding the `alt` key while entering the 3-digit decimal code with a leading zero (thus `alt`-235 for e-umlaut). You can use this method also to enter (rather than act upon) any of the standard ASCII characters, including the nonprinting characters; thus, to enter a CR (carriage return) you would type `alt`-013. And good programming editors (such as Notepad++, UltraEdit, or Emacs) accept the insertion of control characters, in addition to displaying what's there. Try it!

**Table 14.5 ASCII Codes**

| non-printing | | | | | printing | | | printing | | | printing | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Name | Control char | Char | Hex | Dec | Char | Hex | Dec | Char | Hex | Dec | Char | Hex | Dec |
| null | ctrl-@ | **NUL** | 00 | 00 | **SP** | 20 | 32 | **@** | 40 | 64 | **‘** | 60 | 96 |
| start of heading | ctrl-A | **SOH** | 01 | 01 | **!** | 21 | 33 | **A** | 41 | 65 | **a** | 61 | 97 |
| start of text | ctrl-B | **STX** | 02 | 02 | **"** | 22 | 34 | **B** | 42 | 66 | **b** | 62 | 98 |
| end of text | ctrl-C | **ETX** | 03 | 03 | **#** | 23 | 35 | **C** | 43 | 67 | **c** | 63 | 99 |
| end of xmit | ctrl-D | **EOT** | 04 | 04 | **$** | 24 | 36 | **D** | 44 | 68 | **d** | 64 | 100 |
| enquiry | ctrl-E | **ENQ** | 05 | 05 | **%** | 25 | 37 | **E** | 45 | 69 | **e** | 65 | 101 |
| acknowledge | ctrl-F | **ACK** | 06 | 06 | **&** | 26 | 38 | **F** | 46 | 70 | **f** | 66 | 102 |
| bell | ctrl-G | **BEL** | 07 | 07 | **’** | 27 | 39 | **G** | 47 | 71 | **g** | 67 | 103 |
| backspace | ctrl-H | **BS** | 08 | 08 | **(** | 28 | 40 | **H** | 48 | 72 | **h** | 68 | 104 |
| horizontal tab | ctrl-I | **HT** | 09 | 09 | **)** | 29 | 41 | **I** | 49 | 73 | **I** | 69 | 105 |
| line feed | ctrl-J | **LF** | 0A | 10 | ***** | 2A | 42 | **J** | 4A | 74 | **j** | 6A | 106 |
| vertical tab | ctrl-K | **VT** | 0B | 11 | **+** | 2B | 43 | **K** | 4B | 75 | **k** | 6B | 107 |
| form feed | ctrl-L | **FF** | 0C | 12 | **,** | 2C | 44 | **L** | 4C | 76 | **l** | 6C | 108 |
| carriage return | ctrl-M | **CR** | 0D | 13 | **-** | 2D | 45 | **M** | 4D | 77 | **m** | 6D | 109 |
| shift out | ctrl-N | **SO** | 0E | 14 | **.** | 2E | 46 | **N** | 4E | 78 | **n** | 6E | 110 |
| shift in | ctrl-O | **SI** | 0F | 15 | **/** | 2F | 47 | **O** | 4F | 79 | **o** | 6F | 111 |
| data line escape | ctrl-P | **DLE** | 10 | 16 | **0** | 30 | 48 | **P** | 50 | 80 | **p** | 70 | 112 |
| device control 1 | ctrl-Q | **DC1** | 11 | 17 | **1** | 31 | 49 | **Q** | 51 | 81 | **q** | 71 | 113 |
| device control 2 | ctrl-R | **DC2** | 12 | 18 | **2** | 32 | 50 | **R** | 52 | 82 | **r** | 72 | 114 |
| device control 3 | ctrl-S | **DC3** | 13 | 19 | **3** | 33 | 51 | **S** | 53 | 83 | **s** | 73 | 115 |
| device control 4 | ctrl-T | **DC4** | 14 | 20 | **4** | 34 | 52 | **T** | 54 | 84 | **t** | 74 | 116 |
| neg acknowledge | ctrl-U | **NAK** | 15 | 21 | **5** | 35 | 53 | **U** | 55 | 85 | **u** | 75 | 117 |
| synchronous idle | ctrl-V | **SYN** | 16 | 22 | **6** | 36 | 54 | **V** | 56 | 86 | **v** | 76 | 118 |
| end of xmit block | ctrl-W | **ETB** | 17 | 23 | **7** | 37 | 55 | **W** | 57 | 87 | **w** | 77 | 119 |
| cancel | ctrl-X | **CAN** | 18 | 24 | **8** | 38 | 56 | **X** | 58 | 88 | **x** | 78 | 120 |
| end of medium | ctrl-Y | **EM** | 19 | 25 | **9** | 39 | 57 | **Y** | 59 | 89 | **y** | 79 | 121 |
| substitute | ctrl-Z | **SUB** | 1A | 26 | **:** | 3A | 58 | **Z** | 5A | 90 | **z** | 7A | 122 |
| escape | ctrl-[ | **ESC** | 1B | 27 | **;** | 3B | 59 | **[** | 5B | 91 | **{** | 7B | 123 |
| file separator | ctrl-\ | **FS** | 1C | 28 | **<** | 3C | 60 | **** | 5C | 92 | **\|** | 7C | 124 |
| group separator | ctrl-] | **GS** | 1D | 29 | **=** | 3D | 61 | **]** | 5D | 93 | **}** | 7D | 125 |
| record separator | ctrl-^ | **RS** | 1E | 30 | **>** | 3E | 62 | **^** | 5E | 94 | **~** | 7E | 126 |
| unit separator | ctrl-_ | **US** | 1F | 31 | **?** | 3F | 63 | **_** | 5F | 95 | **DEL** | 7F | 127 |

things so that there are enough transitions in a serial data stream so that the receiver can recover a clocking signal. A simple (though not terribly efficient) example is Manchester coding (Figure 14.46).

The successive bits are transmitted at fixed rate, with a required transition in the middle of each cell: a “1” is LOW-to-HIGH, and vice versa. There may or may not be a transition at the beginning of a cell, as required by the data. The guaranteed transitions at the bit rate facilitate clock recovery (with a phase-locked loop, or delay-locked loop), and the signal is dc balanced, so it can be transformer coupled.

Receiver synchronization with Manchester coding is not completely trivial, since, for example, a continuous sequence of 1s creates a simple square wave. If the receiver chooses the wrong phase, it will interpret the stream as 0s. However, the presence of mixed 1s and 0s disambiguates the clock recovery, because the wrong phase creates violations of the required mid-cell transitions.

Manchester coding is used in low-speed Ethernet (10base-T), which carries 10 Mb/s in one direction on a cat-5 twisted pair (two pairs are used, for full duplex).[61]

[61] When faster Ethernet standards were created, the bandwidth-inefficient Manchester coding (100% overhead, compared with the stream of data bits themselves) was replaced with 4b/5b encoding (a 25% bandwidth overhead), conveyed with 3-level voltage signaling, still on a single twisted pair. The next step – to 1 Gb/s – required 5-level signaling and the simultaneous use of four pairs (with a “hybrid” coupler, permitting each pair to carry signals simultaneously in both directions).

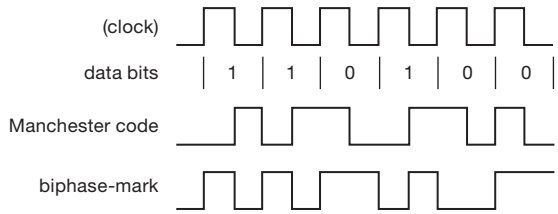



**Figure 14.46.** Biphase encodings. For Manchester ("biphase-level") coding, every bit cell has a transition in the middle, in a direction set by the bit's value; you can think of Manchester code as the exclusive-OR of a clock (not transmitted) and the data. For biphase-mark coding, the bit value is encoded as presence or absence of a mid-cell transition, following a mandatory transition at the beginning.

Because it is wasteful of bandwidth (by a factor of 2), the better "8b/10b" coding (see below) is used instead for high-performance self-clocking serial links such as SATA, HDMI (high-definition multimedia interface), PCIe, and gigabit Ethernet.

### 14.7.10 Biphase coding

Manchester is a special case of what is called *biphase coding*, and in fact is sometimes called *biphase-level*, to distinguish it from biphase-mark, biphase-space, and differential biphase codes. These all share the common feature of having a transition at every bit cell to facilitate receiving clock recovery. However, the latter three codes, which are closely similar to each other, have an important advantage over the vanilla Manchester code, namely that they are insensitive to an inversion of polarity (as can happen with transformer-coupled signals). Let's see how this works.

Look at the popular biphase-mark code in Figure 14.46: there is a required transition at the beginning of each cell, and a transition in mid-cell for a transmitted 1 (but no transition for a 0). It is the encoding of bits as the presence or absence of *transitions* (rather than as defined polarities, as in Manchester coding) that makes the biphase-mark code unambiguous, even with a polarity inversion of the transmitted levels.

Biphase-mark coding is used in digital audio links such as AES3, S/PDIF and Toslink, and in some magnetic stripe codes. It goes by the alternative names of *Aiken Biphase* or *F2F* coding.

### 14.7.11 RLL binary: bit stuffing

A simple serial scheme with reduced bandwidth is to transmit the data bits directly, adding occasional bits to ensure transitions by which the receiver can synchronize its clock oscillator. "Sending the bits directly" has the acronym NRZ (non-return to zero), with numerous variants, notably NRZI (non-return to zero inverted) in which the encoded stream changes state for an input 0, but stays put for an input 1.[62]

The trouble with either scheme is that there can be long runs of data that cause no transitions in the encoded stream: a string of 1s (or of 0s) for NRZ, or a string of 1s for NRZI. This is bad not only for clock recovery, but also because it extends the transmission band to low frequencies, complicating transformer coupling. The way to fix this is to modify the data (either before or after coding) in order to limit the length of successive output states with no transitions. This is called *run-length limiting* (RLL).

There are various RLL coding schemes in use, for example 8b/10b encoding (coming next) in which each 8-bit string of data is encoded into a well-chosen 10-bit stream to be transmitted; or the 8-to-14 ETF code used for optical compact discs (in which the run length is bounded at both ends: $2 \le RL \le 10$). The simplest RLL scheme, though, is probably simple "bit stuffing," as used, for example, in the USB serial link. USB uses NRZI with bit stuffing to limit encoded runs to six consecutive 1s or 0s. This is done by inserting ("bit-stuffing") a 0 into the raw binary data stream after a run of six 1s, forcing a transition in the NRZI-encoded stream. The receiver, of course, knows to ignore a 0 that is preceded by six 1s in the decoded stream. Although the worst-case overhead could be 16% (for a data stream of all 1s), in fact the overhead is less than 1% with random input data.

### 14.7.12 RLL coding: 8b/10b and others

A more sophisticated way of generating run-length limited serial streams involves encoding the binary data stream in blocks, according to relatively complex algorithms that bound the encoded runs, while additionally controlling the spectral shape, and perhaps adding robustness against errors. For example, DVD optical discs use a scheme called EFMPlus, in which 8-bit groups of data are encoded into 16 serial bits, the latter run-length limited to $2 \le RL \le 10$, while simultaneously shaping the spectrum.

The 8b/10b scheme is a popular code in the serial bus and data link world, being used in FireWire, SATA/SAS,

[62] These history-encrusted terms are somewhat confusing (suggesting, for example, that NRZI is related to NRZ by inversion of the code or the data). We prefer the terminology used by Sklar: NRZ-L ("NRZ-level," for NRZ), and NRZ-M ("NRZ-mark," for NRZI).

gigabit Ethernet, DVI and HDMI, and in the multiple "lanes" of the internal PCIe (PCI Express) versions 1 and 2.. It encodes 8-bit input data groups into 10-bit serial stream bits, using the flexibility (of multiple possible 10-bit codes for each 8-bit input group) to balance the number of 1s and 0s: it keeps a running tally of bit inequality, and chooses accordingly. The resulting serial stream is guaranteed to have no more than five sequential 1s or 0s; and the number of 1s and 0s in any string of 20 or more serial bits is guaranteed to differ by no more than 2. An analogous 4b/5b code is used in 100 Mbps Ethernet.

8b/10b coding is used also in some serializer-deserializer chipsets, for example the CY7C924. However, the simpler asynchronous NRZ scheme, framed by START and STOP pulses, is used in other high-performance SERDES chips, for example the DS92LV18 series from TI. They claim that it's better, because the receiver will lock to random data without interrupting traffic with PLL training patterns and without a loss-of-lock feedback path from receiver to transmitter. The legacy 9600-baud asynchronous serial communication is thus reborn as a *gigabaud* serial link.

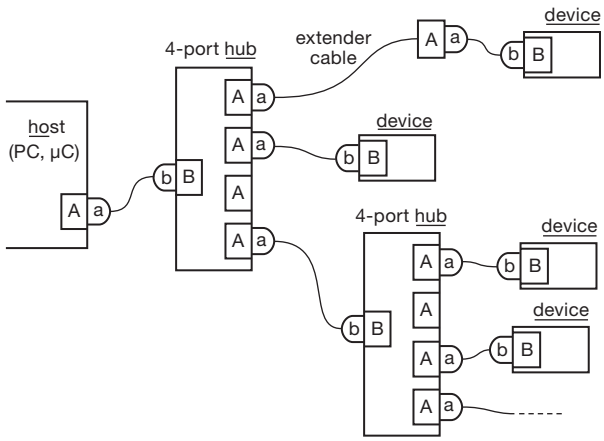



**Figure 14.47.** USB is a master–slave serial interface, arranged in a star topology (as opposed to a serial repeater topology like FireWire), with up to five repeater "hubs." The individual links are asymmetrical cables, with *A* at the master end and *B* at the slave end (a full-size pair is shown in Figure 1.123); here *a* and *b* represent plugs, *A* and *B* sockets. Individual links are limited to 5 m length (including passive extender cables), but an active hub resets the "ruler." An interesting variant (not shown) is the OTG (on-the-go) USB port, a chameleon that can masquerade as either an A or a B port.

### 14.7.13 USB

The Universal Serial Bus (USB) was introduced in 1995, with the goal of simplifying connections between computers and peripherals by having a unified serial connection. It initially supported "low-speed" and "full-speed" devices (1.5 Mb/s and 12 Mb/s, respectively) in USB version 1, adequate for devices such as keyboard and mouse, and relatively modest speed transfers to external memory devices such as flash "thumb drives." Version 2 added a "hi-speed" category (480 Mb/s), suitable for serious data transfer to external hard drives, optical storage, and the like. Version 3 adds two "SuperSpeed" shielded twisted pairs (and a piggyback pair of connector pins), delivering both full-duplex communication and another factor of ten in speed (up to 4.8 Gb/s). And the latest revision – USB 3.1 – doubles the speed (to 10 Gb/s), increases the dc power options (to 5 V at 2 A, or optionally as much as 5 A at 12 V or 20 V), and promotes a new and greatly improved "type-C" connector.

USB versions 1 and 2 are organized as half-duplex master–slave, with point-to-point electrical connections via a 4-wire cable that carries power and ground, and one differential pair of data. The cable is unsymmetrical, with an *A* end (host, or master) and a *B* end (slave); each type (*A*, *B*) comes in a full-size and a mini-size version, the latter for connection to small equipment such as cameras and PDAs. A USB network is a star topology, with multiple devices connected via USB hubs, which replicate the host (*A*) sockets, allowing multiple slaves (up to a total of 127 devices from a single host controller port); see Figure 14.47. The supplied power is rather modest – a maximum of +5 V at either 100 mA ("low-power") or 500 mA ("high-power")[63] – and a single link can be no longer than 5 m (but can be extended with hubs, to a total of 20 m). USB version 3.0 introduced full-duplex (and higher speeds), along with somewhat greater (but not enough, in our opinion) dc power – up to 900 mA; happily, version 3.1 addresses that shortcoming, permitting as much as 10 W at 5 V, and an astounding 100 W at 20 V. USB devices are "hot-pluggable," with ground and power connecting before the signal lines.

### 14.7.14 FireWire

FireWire, known officially as IEEE 1394, was introduced also in 1995, and was designed as a high-speed serial bus for audio, video (including high-definition), and disk storage. It emerged from the starting gate with a 400 Mb/s full-duplex transfer rate (compared with USB's 12 Mb/s half-duplex), and with significant dc powering through its cable

[63] A USB device is allowed only low power, when first connected, and it must negotiate for high-power status. If successful, the controller turns on a 500 mA-capable power source. Table 12.4 lists a few protected power-switch ICs popular for this purpose.

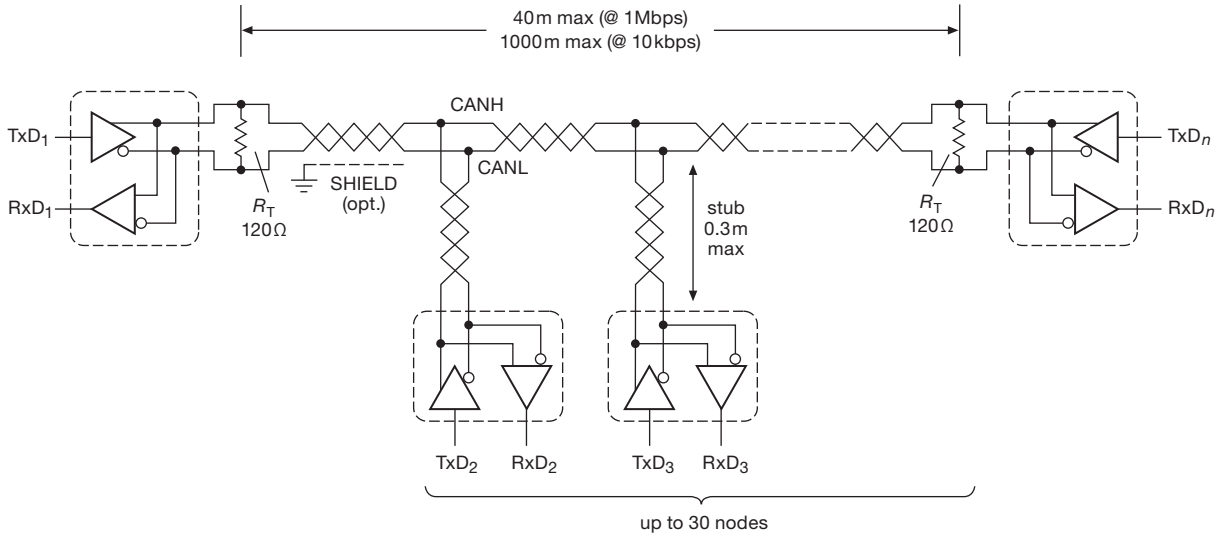



**Figure 14.48.** The Controller Area Network (CAN) is a multimaster, multidrop architecture that is used widely in automotive and factory-floor applications. It is optimized for short "broadcast" messages, and operates well in noisy environments.

(up to 45 W and 30 V, compared with USB's paltry 2.5 W). Subsequent revisions increased the data rate to full-duplex 800 Mb/s (FireWire 800), with a factor of four (to 3.2 Gb/s) promised in the next revision.

FireWire allows multiple hosts and peer-to-peer communication; its cables are symmetrical, and connect devices in a bused (repeater) configuration. Individual links can be no longer than 4.5 m (for FireWire 400), but can chain through repeaters to a total of 72 m. FireWire 800 permits longer links through copper, Cat-5e network cable, or optical fiber. Like USB, FireWire connections are hot-pluggable; sadly, however, FireWire has been largely overtaken by recent revisions of USB.

FireWire provides plenty of dc power, and works well with streaming media such as high-definition video. We have found FireWire both faster and more stable than USB. It has a nice robust connector design[64] that can be plugged in "blind," owing to its asymmetric shape. In spite of FireWire's technical advantages, the USB standard is becoming dominant, probably because of the complexity, hardware cost, and royalty burdens of FireWire.[65]

### 14.7.15 Controller Area Network (CAN)

Everyone knows about USB and FireWire; but who has heard about the *CAN* bus? This bus, sometimes simply called "CAN," was originated by Bosch in the 1980s for automotive use and is now standardized as ISO 11898. It's all over the place. Several protocols have been layered on top of the CAN physical layer, notably DeviceNet™, which is in common use on factory floors.[66]

Unlike the other buses in this chapter, CAN can operate over distances to a kilometer (helpful for the factory application). It's not a point-to-point electrical link (like PCIe, Ethernet, or USB). Rather it's an egalitarian "multi-master" bus, accommodating up to 30 transceiver nodes along its length; that is, it is a "multidrop" bus (see Figure 14.48). The maximum bitrate is 1 Mbps for distances up to 40 m, dropping gradually to 10 kbps at the maximum specified bus length of 1000 m.

CAN is optimized for short "broadcast"-like transmissions, limited to 8 user data bytes per packet (plus some signaling overhead), directed to no one in particular. As the helpful Kvaser website (www.kvaser.com/can/) explains, a CAN message conveys the following sentiment: *"Hello everyone, here's some data labeled X, hope you like it!"*[67] This is just what you want in certain data acquisition and control networks: for example, a sensor generates a stream of temperature values and wants to tell the world about

[64] Said to be inspired by the kid-tested connector used in the Nintendo GameBoy.

[65] Which, however, managed to ship its billionth node in 2008!

[66] You'll find CAN in places other than the car and factory; one of our colleagues bought a laser scanning microscope (Zeiss Duoscan), and found. . . CAN bus!

[67] That's the sentiment; but these messages are *short!* – you'd run out of bytes partway into the second word.

them, but doesn't particularly care who is listening.[68] And it has the pleasant property of being completely quiet electrically when there's nothing to send – this in contrast to chatty buses like USB, FireWire, and Ethernet, which like to mumble incessantly to themselves (and to the rest of the world).

One way to think of this is that the CAN bus (and its extensions into the microprocessors on the bus) represents a "parameter space" that is constantly and automatically updated by the members of the bus. Thus, whoever needs to know the engine temperature (including a computer that you just clipped onto the bus for diagnosing a bug) just looks at the value that has been written into its engine temperature "mailbox."

Electrically, you can think of CAN as a differential version of an open-collector bus: the signal pair (called CANH and CANL) takes the form of a twisted pair (usually shielded – "STP"), terminated at both ends in its characteristic impedance (usually 120 Ω). In the quiescent state both lines rest at ∼2.5 V, and that's where they stay unless some node is talking. When that happens, data is driven onto the line pair in a curious asymmetric signaling scheme (Figure 14.49): a logic "0" is asserted by taking CANH ∼1 V higher (to ∼3.5 V) and CANL lower (to ∼1.5 V), generating a ∼2 V differential signal. But to assert logic "1," the talker does not reverse the sense of drive current, but rather *releases* drive current, which lets both lines of the pair go to their ∼2.5 V resting state. These two signaling states are called *dominant* (bus driven, logic 0) and *recessive* (bus released, logic 1 or inactive).

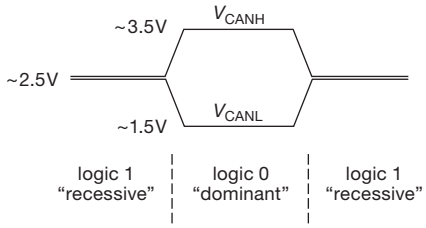



**Figure 14.49.** The CAN bus is differential, with an asymmetric signaling mode that simplifies arbitration.

This curious scheme was devised to simplify bus arbitration: there's no master controller in charge; instead any node can initiate a transmission, as long as it has seen the bus resting (recessive) for some minimum time. It then asserts its message bits sequentially, monitoring at the same time the state of the bus (every node must include a receiver). Of course, it's possible that another node has also initiated transmission. Such a "collision" is detected easily enough, because one of the transmitters will see a dominant (asserted, logic 0) state when it intended to send a recessive (disasserted, logic 1) state. It is then required to back off and try retransmission later.

The result is that, if there's a collision, the sender that generates the longest string of initial 0s gets the bus. This prioritizes the senders, because the initial bits of any message contain the sender's 11- or 29-bit message identifier, with lower numbers taking priority. And note especially the nice feature of the dominant–recessive scheme: the priority sender's message is not damaged by the collision (figure out why). In the acronym-studded language of networks, this is a "CSMA/CD+AMP" protocol.[69]

The CAN standard specifies good tolerance to common-mode noise, requiring correct operation at least from −2 V to +7 V, but extended by many transceiver ICs to ranges of −7 V to +12 V, or −12 V to +12 V. And ISO 7637 prescribes a torture test consisting of a train of ±150 V nanosecond-scale pulses that a transceiver must survive. In addition, you can use an inexpensive diode and zener protective device like the NUP2105 or NUP2202 (see ON Semi's App. Note AND8169; see also §12.1.5). For long bus runs, however, it's best to provide true galvanic isolation (avoiding ground loops mandates that there be only one ground point).

In keeping with its original intended automotive use, the CAN bus includes robust error-detection mechanisms: at the *bit* level there's bit monitoring (with error flagging if a disagreement), and detection of a "bit-stuffing" error (an opposite bit must be inserted after 5 consecutive bits of the same level). And at the *message* level there is a CRC (cyclic redundancy checksum), along with checking of an ACK (acknowledge) field and of a set of certain designated message bits (a "form check") that must be recessive. By this mechanism a transmitter can know if its message was corrupted and needs to be resent.

You can get plug-in CAN interface cards for standard computer buses such as PCI/PCIe, PC/104, and PCMCIA, and adapters from USB. And many microcontrollers (e.g., Atmel's AT90CAN series) include an on-chip CAN protocol controller. At the component level there are many manufacturers of CAN transceiver chips (most are powered from a single +5 V, but some work at +3.3 V). You

[68] And, conversely, it's inefficient for transmitting large blocks of data (for example, digitized audio or video) between two nodes. You also would not use it to link two processors in the same box.

[69] Well, at least it's shorter than the tongue-twister "carrier-sense multiple-access with collision detection and arbitration on message priority."

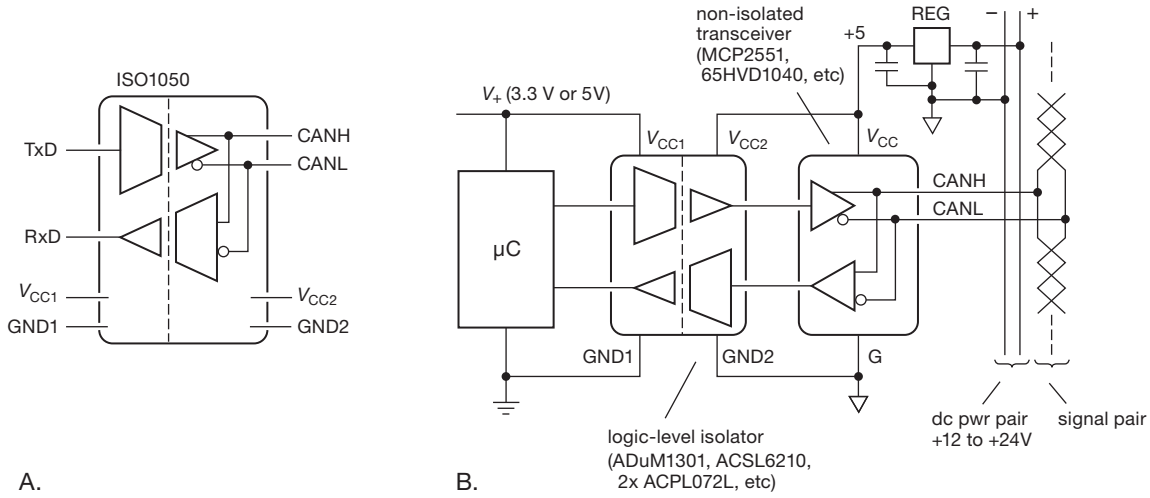



**Figure 14.50.** A. You can get single-chip galvanically isolated CAN transceivers, like TI's ISO1050, a good idea for long runs or noisy environments. B. Standard CAN cables include a second wire pair for isolated power, which you can use to power the bus side of the transceiver's isolation barrier. Here we've used a logic-level isolator between a microcontroller and the non-isolated CAN transceiver, the latter powered (along with the bus side of the isolator) from regulated +5 V derived from the bus dc power pair.

can also get galvanically isolated transceiver chips (e.g., the TI ISO1050, Figure 14.50), a good idea with long buses or noisy environments: you can power the bus side with an independent +5 V supply, or you can take advantage of the bused power pairs that often are included (for just such remote powering) along with the shielded and controlled-impedance signal pairs in CAN bus cables (e.g., Belden 3082/84 or Alpha 6451/52).

There are a couple of simplified CAN variants in use: single-wire CAN dispenses with CANL, and is limited to 40 kbps. And the closely related single-wire LIN (local interconnect network) bus dispenses with the regulated 5 V supply altogether, using a pullup to the +12 V battery and open-collector closure to ground; it's limited to 20 kbps. These inexpensive and simplified variants are sometimes used as a sub-bus in a CAN system; they both employ slew-rate limiting for reduced susceptibility to noise.

There is no defined CAN connector, but there are several common implementations, including a 9-pin D-subminiature, a 10-pin header, and a 4-pin open connector.[70]

[70] See `www.interfacebus.com/Design_Connector_CAN.html` and `www.interfacebus.com/Can_Bus_Connector_Pinout.html`. Additional information about the CAN bus can be found at www.kvaser.com/can/, www.can-cia.de, TI's App. Note SLOA101A, and Analog Devices' AN-770A.

### 14.7.16 Ethernet

Ethernet[71] is ubiquitous – those colorful telephone-like "RJ-45" network jacks – we all love it, we've mentioned it numerous times (for example in §12.10.3B, and in §14.7.9 in connection with Manchester coding), and we'll be mentioning it again in the next chapter. It was developed back in the 1970s at Xerox's famed Palo Alto Research Center ("Xerox PARC"), and at the physical layer (PHY) consisted originally of a shared coax cable (first "thick wire," officially called 10Base5; later "thin wire," or 10Base2), terminated at both ends, and tapped into by each node. Every node used transformer coupling, connecting right at the coax (no long stubs allowed), and (as with the CAN bus) there was a protocol for sensing collisions and backing off before retransmitting. To work properly in the face of inevitable (and data-destructive) collisions, there was established a minimum packet length (now standardized by IEEE Std. 802.3 as 74 bytes), and a maximum cable length (about 200 m for thin wire).

In contemporary practice the shared coaxial cable has been replaced with a point-to-point unshielded twisted-pair cable (UTP; Cat-5e or Cat-6), one end plugging into

[71] The whimsical name was chosen to convey the spirit of an all-present data-conveying medium, analogous to the "luminiferous ether" through which it was (incorrectly) thought that light traveled, before it was disproven by Michelson and Morley's famous experiment of 1887. With characteristic computer-nerd humor, the first two networked computers were named Michelson and Morley.

a computer's Ethernet NIC (network interface card), say, and the other end into a multiport "switch."[72] The latter buffers and forwards valid packets onward, without bothering uninvolved nodes. Ethernet, so configured, is collisionless.[73] Contemporary Ethernet is carried over twisted pair or optical fiber; standard speeds are 10 Mbps, 100 Mbps ("Fast Ethernet"), and 1 Gbps ("Gigabit Ethernet"), with evolution to 10 Gbps and 100 Gbps. And there is talk of *tera*bit Ethernet. As we remarked in §14.7.9, the evolution of higher speeds over twisted pair required some ingenuity: the slow stuff (called 10Base-T) uses Manchester coding and 2-level signaling, with one pair in each direction. To go to 100 Mbps (100Base-TX), 4b/5b coding is used, with 3-level signaling, again with one pair in each direction. Gigabit Ethernet (1000Base-T) uses 8b/10b coding, with all four pairs used in *both* directions (via a "hybrid"). These are descriptions of the *physical* layer; higher levels of the 7-layer OSI (open systems interconnection) network hierarchy don't know (and don't care) what's going on down there, so you can upgrade the hardware to your heart's content.

Twisted-pair Ethernet links are limited in length to ∼100 m owing to signal degradation and attenuation. Fiber does much better – to a kilometer or so with multimode fiber, and tens of kilometers with single-mode fiber.[74] You can get "media converters" to change between copper and fiber, or copper and wireless; some of these incorporate rate conversion as well. Look at the Ethernet offerings from Allied Telesis, TRENDnet, StarTech, or IMC Networks; or the broad range of converters and extenders (including products for USB and serial, in addition to Ethernet) from B&B Electronics. See also Chapter 15 for interface component suggestions, such as the Lantronix XPort or Silicon Labs 28-pin CP2201 chip (to which you can simply add an RJ-45 jack with an integrated transformer and indicator LEDs).

Ethernet is so widely supported that it has become the dominant communications medium between computers and within local-area networks (LANs). Instrument manufacturers took notice, and it's rare to find a contemporary electronic instrument without an Ethernet port, both for instrument control and for data readout. (Nearly all of the 'scope waveforms in this volume were pulled in through our lab's LAN, from a Tektronix TDS3000-series 'scope, by pointing a browser at the instrument's IP address.) And, in a pleasant development, a standard is emerging in instrument control (the LXI standard: LAN eXtensions for Instrumentation), making communicating with contemporary instruments quite simple, especially when compared with the proliferating varieties of USB drivers. Ethernet is making its way into the industrial arena, with "industrial" variants that add features catering to the needs of real-time control.[75]

## *14.8 Number formats*

We wind up this chapter with a short riff on number formats, meaning the way that numbers are represented internally during computation, or exchanged via digital media or communications ports. The scene is summarized in Figure 14.51, with some explanation in the following paragraphs.

### 14.8.1 Integers

*Signed integers* are always represented in 2's complement, using either 1, 2, or 4 bytes, or occasionally 8 bytes, as shown. The most-significant bit (MSB) tells the sign, even though 2's complement is not the same as sign–magnitude representation (e.g., −1 is 11111111, not 10000001; see §10.1.3). You can think of 2's complement as offset binary with inverted MSB; alternatively, you can think of it as an integer with the bit values as shown in Figure 14.51. Programming languages let you declare variables as *unsigned* integers, in addition to 2's complement signed integers. A 2-byte unsigned integer can have values from 0 to 65535. Quite apart from the number format itself, there's the hardware interface issue about how you pack integer data into a computer's larger word. For example, the binary integer

[72] This is a *star* topology, compared with the *bus* topology of coax Ethernet.

[73] Before switches were widespread and inexpensive, people used "hubs," which simply rebroadcast each packet to all connected nodes. With hubs you get collisions; with switches you don't.

[74] But twisted-pair Ethernet has a nice feature: it lets you send dc power ("Power over Ethernet," PoE), which is handy when connecting to remote devices like wireless access points, IP phones, or surveillance cameras. It uses the same signal pairs, applying the ∼48 V dc as common-mode "phantom power" between the two pairs, picked off between the transformer center taps at the remote end. Audio engineers have been using the same trick, for ages, to power their microphones.

[75] For example, implementations of the IEEE Std. 1588 "Precision Time Protocol" (PTP), which allows time synchronization to ∼100 ns via Ethernet (with dedicated hardware, for example the NSC DP83640 MAC/PHY chip, or included in embedded microcontrollers like TI's Luminary Stellaris ARM Cortex M3). PTP is currently being implemented by the LXI standard for interoperability between LAN-based instruments, as well as by several Ethernet-based Fieldbuses such as Profinet and CIP (Common Industrial Protocol).

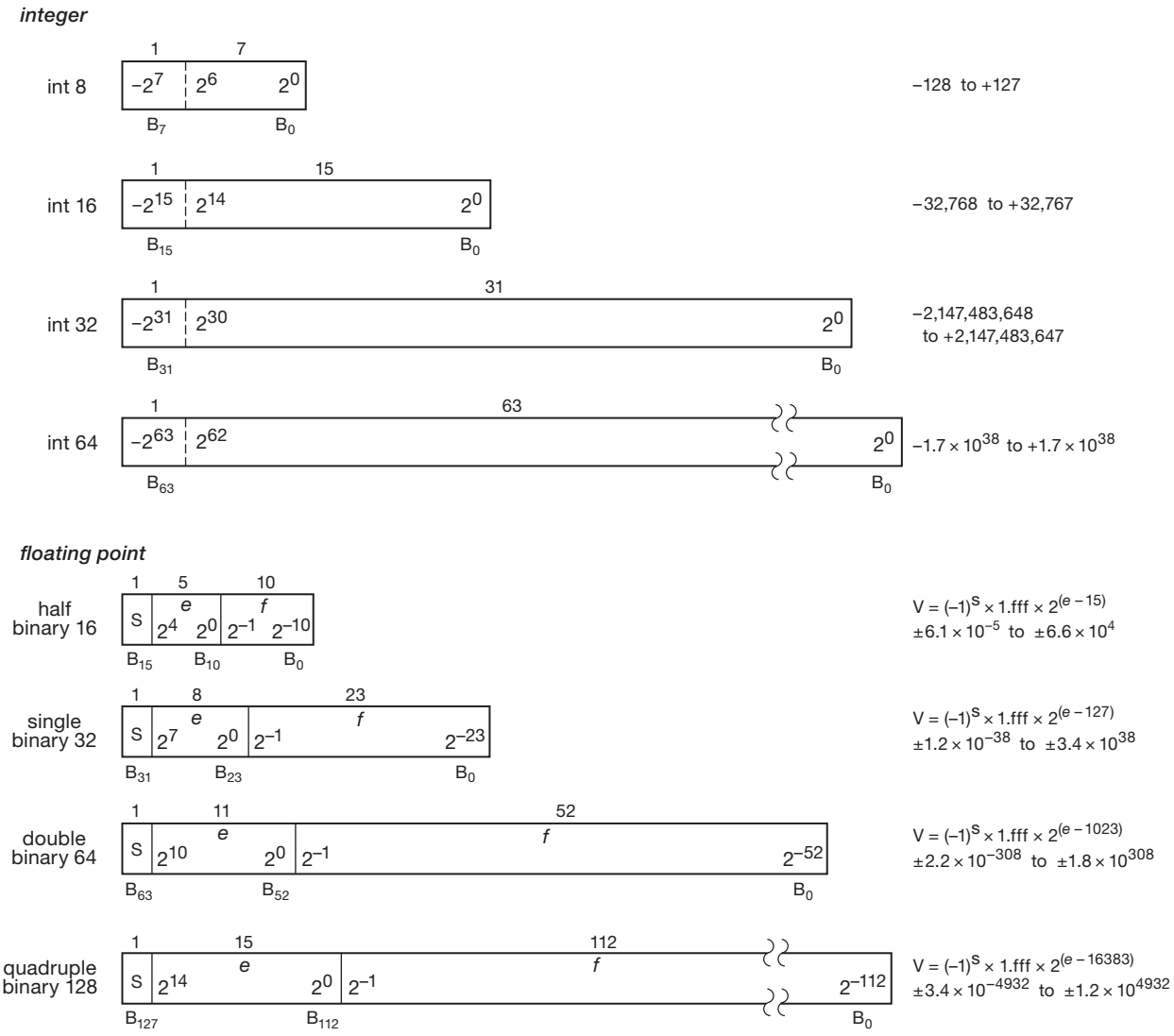



**Figure 14.51.** Commonly used number formats. The symbol "*e*" is the unsigned binary-integer value of the exponent field, used as shown to determine the value *V* of the various floating-point formats.

output from an ADC could be right-justified, so numbers go from zero up to the converter's full scale (0 to 4095, for a 12-bit ADC). But it's arguably better to left-justify the data, and think of the stored quantity as a *fraction*. This has a nice benefit, namely, if the ADC resolution is subsequently improved, it simply adds additional low-order fractional bits (rather than increasing the full-scale value).

### 14.8.2 Floating-point numbers

*Floating-point* numbers (sometimes called *real* numbers) are most commonly represented and stored as 32-bit ("single-precision") or 64-bit ("double-precision") quantities.[76] The bad news is that there are several incompatible representations in use. The good news is that the floating-point standard blessed by the IEEE (Std. 754-2008) has been implemented by nearly all processor families and is in nearly universal use.

Figure 14.51 shows the IEEE floating-point formats in detail. To see how it goes, look at the 32-bit single-precision format: it has 1 sign bit, 8 exponent bits, and

[76] Augmented by giant 128-bit ("quadruple precision"), and diminutive 16-bit (you guessed it – "half precision") formats.

23 bits of fraction. The exponent tells the power-of-2 that the fraction (see below) should be multiplied by. The exponent is "biased" by adding 127, so that the exponent field 01111111 corresponds to an exponent of 0; exponents thus go from $-127$ to $+128$. The fraction itself uses an interesting trick, originated by DEC in their floating-point format. A floating-point number in binary can always be written in the form $\mathrm{f.fff}\times 2^e$, where f.fff is the (base-2) mantissa ("significand") and $e$ is the (power-of-2) exponent. To maximize the precision you get with a given number of mantissa bits, you "normalize" it by shifting the mantissa left (and decrementing the exponent) until the leading bit is nonzero, thus casting it in the form $1.\mathrm{fff}\times 2^e$. Now, here's the "hidden-bit" trick: because the resulting normalized significand always has a nonzero MSB, it would be redundant to display it; i.e., you don't store 1fff in the number, just the fff, with the leading 1 assumed. The resulting number gains one bit of precision, and has a range of $\pm 1.2\times 10^{-38}$ to $\pm 3.4\times 10^{38}$.

**Exercise 14.3.** Show that the range of normalized floating-point numbers is as claimed, by constructing the smallest and largest numbers.

The IEEE double-precision format is similar, but with the significand precision more than doubled (by attaching 29 more bits) and with the exponent fortified by an additional 3 bits, giving the range of numbers shown in the figure. If you like double precision, you'll love the additional dynamic range and accuracy of the whopping 128-bit quadruple-precision format, with its dazzling 113 bits of fraction, aided and abetted by a 15-bit exponent.

At the other end, a newly popular format is the 16-bit "half-precision" format (which some people call a "minifloat"). It squeezes sign, exponent, and fraction into a 2-byte word. The largest values in this format are $\pm 65504$. This isn't a whole lot more than you can do with a 16-bit signed integer ($\pm 32767$), so it might seem pointless. Not so: the smallest value of IEEE minifloat is $\pm 6.1\times 10^{-5}$. So, you get a large *dynamic range*, albeit without great precision.[77]

This last point is worth a bit more explanation. The IEEE half-precision floating-point format (officially called "binary16") represents numbers spanning nine orders of magnitude, with an approximately uniform *fractional* step size of $\sim$0.06%. Unlike an integer representation, the fractional changes do not get coarser as you go to small values (figure out why). And this is a good characteristic for quantities that are perceived logarithmically, such as illumination or sound intensity.

The IEEE format allows non-normalized numbers also, to give some additional range at the small end, at the expense of precision (these have the exponent bits set to all zeros, which changes the interpretation of the fraction to 0.fff). For single-precision (binary32) floating, the range of these "denormalized" numbers then goes down to $\pm 1.4\times 10^{-45}$; but the steps become fractionally larger as you hit bottom. The standard also defines zero (both $e$=0 and fff=0; thus there are two zeros, $+0$ and $-0$), infinity ($e$ = all 1s, fff=0; therefore both signs), and a curious class of reserved quantities known officially as NANs ("not a number").

### A. Number storage in memory

Microprocessor designers like to express their individuality by storing numbers in memory in peculiar orders. Intel-originated processors store multibyte integers beginning with the least-significant byte in the lowest-numbered memory byte; Motorola (Freescale) processors do it the other way around.[78] Some processors (e.g., the popular ARM core) are ambidextrous (therefore "bi-endian"). And, to add a bit more confusion, some processors use one byte order for integers and the other for floating-point numbers. Endian-ness is of more than academic interest; it can matter, for example, when sending data to a peripheral via SPI or $I^2C$. Lots of luck!

[77] Characteristics that are particularly useful in applications such as video imaging. In fact, it was devised by Industrial Light & Magic for just such uses. The half-float format is supported in graphics processor chips, which are becoming popular for large-scale computation. These chips, with hundreds of fast processing cores, are challenging traditional CPUs; throw a few dozen of them in a box, and you've got yourself a pretty impressive supercomputer.

[78] Engineers get their jollies by calling these "little-endian" and "big-endian," respectively.

### *Review of Chapter 14*

An A-to-J summary of what we have learned in Chapter 14. This summary reviews basic principles, facts, and application advice in Chapter 14.

¶A. Processors and Data Buses.
This first of two processor-oriented chapters dealt with the subject of computer architecture and the interfaces (buses) over which data is exchanged, whereas the subsequent chapter is devoted to the use of microcontrollers as "embedded" components within a circuit or instrument. Given the omnipresence of computers in contemporary life, the reader is likely to be familiar with much of the material in this chapter.

¶B. Bus-oriented Computer Architecture.
In a classic computer architecture (§14.1) the processor unit (CPU or MPU) executes instructions fetched from memory, moving data on a set of lines called a *bus* (Figure 14.2). The instructions encode what is to be done, and the processor does it, as interpreted by its *instruction decoder*. Within the processor is an *arithmetic logic unit* (ALU), responsible for arithmetic and logical operations (e.g., ADD or COMPARE) performed on data held in *registers*; a *program counter*, which holds the memory address of the current instruction; a set of *flags* that are set according to the result of the last operation and which are tested for conditional branching; a *stack pointer* to address memory sequentially for temporary storage (e.g., of return address during interrupts and function calls); and, often, a *cache memory* to hold recent data and instructions for quicker access.

¶C. Peripherals.
A computation-oriented microprocessor (as would be used in a server or desktop computer) communicates with off-chip peripherals through one or more buses, whereas a processor aimed at embedded applications (a micro*controller*, Chapter 15) trades off some computational finesse by integrating instead a set of on-chip peripherals and memory. Typical peripherals include nonvolatile mass storage such as hard disk (hdd) and solid-state disk (ssd), random access memory (RAM), video graphics, network interface (Ethernet), data acquisition and control (ADC, DAC, digital I/O), and serial ports such as USB and SATA.

¶D. Instruction Set and "Machine Language."
A given processor is designed to interpret certain groups of bytes, taken together, as *instructions*, and to carry out the corresponding tasks (§14.2). Such *machine language* instructions can be of various lengths; for the Intel x86 32-bit processors, for example, instructions range from one to fifteen bytes in length. Programming directly in machine language is too tedious for humans; instead each instruction can be represented in a readable text-formatted *assembly language*, with understandable mnemonics (e.g., `ADD AX,BX`, which adds to the quantity held in register AX the quantity held in register BX). A program called an *assembler* then converts the assembly-language program (a text file created by the programmer, or by a *compiler* from a program written in a high-level language like C) into the processor's native machine language.

To illustrate processor instructions we chose the Intel x86, displaying a pared-down instruction set (§14.2.2 and Table 14.1). There are *arithmetic* instructions (for example `ADD`), *conditional branch* instructions (for example `JNZ label`, jump on non-zero), stack instructions (`PUSH` and `POP`), and I/O instructions (for example `OUT port,AX`). Along with the instructions, you've got to understand *addressing*, the various ways that locations in memory or registers can be designated. In the x86 language the basic possibilities are *direct* (the address itself), *indirect* (an address pointing to the place in memory where the address itself is held), *indexed* (a numerical offset from a base address, useful for "moving pointer" accesses to sequential addresses), and *immediate* (a numerical quantity contained within the multibyte instruction itself).

¶E. PC104/ISA: a Parallel Bus.
Bus interfacing can be a complicated business, with elaborate negotiation protocols and the like. To keep things simple we chose the legacy PC104/ISA 8-bit parallel bus (§14.3), an outgrowth of the original IBM PC bus that has been adapted to the industrial PC104 embedded system standard. It is in widespread use, and it illustrates well the basics of a multidrop bus (and parallel data transfer in general); for reference, the full set of bus signals is listed in §14.3.11.

WRITE cycle. To move some data out to a peripheral (a "write cycle"), the bus master (here the CPU) asserts the data onto a set of 3-state DATA lines (D0–D7) and the target address onto the ADDRESS lines (A0–A15), then it pulses a "write" line (IOW′), see Figure 14.8. The peripheral's corresponding hardware (Figure 14.9) responds by latching the data (on D0–D7) by clocking with IOW′ if it sees its unique address (on A0–A15). This transaction takes place automatically when the CPU executes an `OUT` instruction to the peripheral's address.

READ cycle. To fetch data from a peripheral (a "read

cycle"), the bus master asserts the peripheral's address, but does not assert the (3-state) data lines; instead it pulses a "read" line (IOR′) and latches the data it finds on D0–D7 (asserted by the addresses peripheral) at the end of IOR′; see Figure 14.13. The peripheral's corresponding hardware (Figure 14.14) responds by asserting its data onto D0–D7 by enabling 3-state drivers with IOR′ if it sees its unique address (on A0–A15). This transaction takes place automatically when the CPU executes an IN instruction from the peripheral's address.

Command and status bits. The "data" that the CPU fetches during a read cycle can be ordinary data (e.g., a byte from an ADC); but it can instead indicate a status (e.g., new ADC data is available to be fetched). Such status data is essential in most cases; see, for example, the simple keyboard interface in Figure 14.15, where a flip-flop is set when a new character is struck and cleared when a character is read; its status is readable on bit D7 of its peripheral address (KBFLAG). The corresponding program code (§14.3.5A) repeatedly reads that *status byte*, looping until it finds bit 7 set, whereupon it fetches a data byte from the keyboard's data address (KBDATA) and appends it to data stored in a string buffer in memory. In analogous manner, bits that are sent *to* a peripheral can indicate an action to be initiated by that peripheral; these are *command* bits, for example to tell and ADC to begin conversion, or a serial port to send a data byte. Multiple status (or command) bits can be grouped into a status (or command) *register*, each bit of which can have its own function.

Interrupts. The use of status bits lets the CPU find out if action needs to be taken, but it has to ask (by reading the status register). Such *polling* is one way for a peripheral to signal the CPU; the alternative is the *interrupt*, in which the peripheral asserts one of several dedicated bus lines (called IRQ*n* on the PC104 bus). This signals the CPU's hardware, which (if interrupts are enabled) causes program execution to break off from what it's doing and jump to an "interrupt handler" routine; see §§14.3.7–14.3.9 for hardware and software examples.

Direct memory access. Most efficient of all for rapid transfer of multiple bytes (e.g., reading a whole file from disk) is DMA, in which the successive bus addresses and control pulses are generated by hardware, and thus do not require the processor cycles needed for the READ or WRITE cycles of *programmed I/O*; see §14.3.10.

### ¶F. Other Parallel Buses.

The PC104/ISA internal multidrop computer bus is slow (less than 10 MB/sec), and was superseded by the **PCI** (peripheral component interconnect) multidrop bus, which reached 2 Gb/s in its 64-bit-wide incarnation. A PCI interface is considerably more complex than the simple PC104 described in ¶E, requiring some negotiation with the PCI controller, etc. PCI is now obsolete, having been replaced by the point-to-point serial PCIe, (see ¶H). For connection to disks, the legacy parallel interfaces are **IDE** (also called ATAPI or PATA) and **SCSI**; both are obsolete, having been replaced by serial buses: SATA and SAS. The original printer interface was parallel (a 25-pin D-connector or 36-pin "Centronics" micro-ribbon connector), replaced by serial buses (USB or Ethernet). And for connection to laboratory instruments the **GPIB** (general-purpose interface bus, originated by HP as the HPIB) is still alive and, well, surviving; it's a bit awkward, though, with its thick cables and hunky connectors. Most lab instruments now support Ethernet and USB. These and other buses are listed in Table 14.3.

### ¶G. Parallel Data Links.

At the much simpler level of data transfer between ICs on a circuit board (say between an embedded microcontroller and an ADC) the parallel bus's data–address–strobe protocol becomes very simple: for a point-to-point connection (i.e., a single peripheral chip) all you need are a byte-wide (or 4-bit-wide) data path plus direction and strobe; see for example Figure 14.34. This is hardly a *bus* – it's really just a data link.

Apart from some IC interfaces of this kind, and the wide and speedy CPU "Front-Side bus," nearly all parallel buses and data links have fallen by the wayside, yielding their dominance to *serial* buses and links (¶¶ H, I); ironically, a single serial link is usually faster than the byte-wide parallel link it replaces, with a lot less wiring and (for external interfaces) permitting much skinnier cabling.

### ¶H. Serial Buses.

There are many advantages to a *serial* data connection (§14.7): fewer wires, fewer pins on the driver and receiver chips, clean termination (in a point-to-point connection, i.e., a single driver and a single receiver), lack of timing skew (self-timed via clock recovery), and the flexibility of using an optical fiber or wireless channel. Counterintuitively, contemporary serial links deliver higher data throughput than their parallel forebears. For example, the last of the parallel disk interfaces topped out at 1–2 Gbps (PATA and SCSI, 16 bits wide), whereas their serial successors (SATA and SAS, a single differential pair in each direction) deliver 6 Gbps.

In the computer world the common *internal* serial buses are the **SATA** (serial ATA), **eSATA** (external SATA, a short

extension of SATA), and **SAS** (serial-attached SCSI) disk buses, and the **PCIe** (PCI express) motherboard bus (see §§14.7.6 and 14.7.7). The latter is a hybrid of sorts – unlike the earlier parallel multidrop PCI bus, PCIe is point-to-point and serial (a differential pair in each direction), but it runs multiple such "lanes" to each motherboard slot, labeled with "×" (commonly ×1, ×4, and ×16, but also ×8 and ×32). Each lane can deliver up to 8 Gbps, with separate clock recovery at the receiver.

For communication *external* to the computer the common serial buses are USB, FireWire, Ethernet, and CANbus. In its current revision (3.0) **USB** (§14.7.13) is full duplex (a differential pair in each direction) with speeds to 3.2 Gbps; this improves on the previous version (2.0) which is half duplex with speeds 480 Mbps. FireWire (IEEE 1394, §14.7.14) is full duplex, with versions supporting 400 and 800 Mbps (and with rates to 3200 Mbps included in the standard).

**Ethernet** (§14.7.16) is the bus of choice for computer networks; the original coaxial multidrop 10 Mbps versions (10base2, thinnet, and 10base5, thicknet) have long since been replaced by full-duplex point-to-point twisted pair connections (10base-T, 100base-TX, 1000base-T), which both circumvent the problem of collisions and offer data transfer speeds to 1 Gbps.

**CANbus** (§14.7.15) is an industrial-strength multidrop and multimaster bus used in automotive and factory environments; it's robust, and permits links up to 1 km, though at greatly reduced data rate (10 kbps versus its 1 Mbps for links less than 40 m long).

¶I. Serial Data Links.

Largely superseding the parallel data transfer between ICs on a circuit board (¶G) are the popular serial data links: SPI, $I^2C$, and JTAG. **SPI** (§14.7.1) is drop-dead simple, with a master–slave architecture comprising three wires shared among multiple slave chips: one data wire in each direction (MOSI, MISO) and a clock (SCLK); there's a separate slave-select line (SS′) needed for each slave device; see Figure 14.38. A selected device receives data (MOSI) and sends data (MISO) on each clock, see Figure 14.37. There's no universal SPI standard, so you have to conform to each chip's protocols according to its datasheet. Take a look at Figure 15.21 in the next chapter to see some SPI peripheral chip examples.

By contrast, **$I^2C$** (§14.7.2) is a more sophisticated multidrop serial link, with two wires only (SDA, data; SCL, clock); there's a well-defined protocol by which the master (the microcontroller) communicates both target address and data direction to the various slave devices. The minimal wiring and communication flexibility of $I^2C$ is offset by the need to give each device a unique address, and by the problems of contention and arbitration on the shared lines. Take a look at Figure 15.22 in the next chapter to see some $I^2C$ peripheral chip examples.

The **JTAG** bus (§14.7.4), originally intended for in-circuit testing and debugging, has become a popular interface for loading and debugging microcontroller code, and for programming nonvolatile on-board memory. It buses a shared clock (TCK) and mode line (TMS), and daisy-chains a data line (TDI in to each device, TDO out); see Figure 14.43. You connect to the JTAG bus with a programming "pod" that attaches to a USB port on your host PC.

And don't forget the good ol' **RS-232** "serial COM port" link (§14.7.8), still widely supported by microcontrollers. You can talk to them with a PC running a terminal emulator, connected with a USB-to-RS232 adapter like FTDI's TTL-232R-3V3 or TTL-232R-5V (3.3 V and 5 V, respectively).

¶J. Memory.

Any talk of computers is incomplete without a discussion of *memory*, set forth in §14.4. The ideal memory would have fast random-access, persistent retention without power (i.e., nonvolatile), and infinite endurance (number of read–write cycles); ideally it should also be low power, inexpensive, and compact. Real-world memory has not yet achieved these goals, at least in a single memory type.

The current popular favorite in *nonvolatile* memory (§14.4.5) is **flash NAND** (the stuff of USB memory sticks, camera memory cards, and solid-state drives). It is limited in speed and endurance, and must be written or erased in sectors; but it's outrageously inexpensive (∼$0.50/GB in single quantities). For retention of small quantities of data there's **EEPROM**, whose 2-transistor cell permits single-bit or -byte erasability. Some new technologies that may deliver near-infinite endurance are **FRAM** (ferroelectric RAM), **MRAM** (magnetoresistive RAM), and **PCM** (phase-change memory, also called PRAM).

Contemporary memory types with fast access and infinite endurance are all *volatile*: SRAM (static RAM) and DRAM (dynamic RAM). The basic **SRAM** is an array of addressable 6-transistor ("6T") flip-flops (Figures 14.20 and 14.21), configured for asynchronous or synchronous (i.e., clocked) operation; see §14.4.3. Static RAM has a simple interface, fast access, and zero quiescent current, but its 6T cell takes up lots of space, so its density is

considerably lower than that of dynamic RAM. For this reason a class of pseudo-static (**PSRAM**) memory has become popular: it exploits a hidden refresh mechanism to bring the density of DRAM to the simple interface of SRAM; it's a DRAM in SRAM's clothing. SRAM and PSRAM memories are useful in small or low-power systems where only a modest amount of memory is needed.

The other volatile memory type is **DRAM** (§14.4.4), an array of tiny (∼30 femtofarads) capacitors whose charge state holds one bit, addressed with an array of MOSFETs such that only one transistor is needed per bit, both for readout and for charge retention ("refresh"); this is a "1T1C cell" (Figures 14.26 and 14.27). DRAMs require periodic refreshing, and, having originated as asynchronous memory chips, they are now used in their synchronous (**SDRAM**) varieties: **SDR** (single data-rate), and evolving **DDR** (double data-rate) versions (**DDR2**, **DDR3**, and **DDR4**). DRAM has infinite endurance, fast access, high density, and low cost, and it is used for the main memory in PCs, laptops, and all variety of consumer electronics (e.g., cable and satellite boxes). It's what you get when you buy a DIMM module for your computer; these things are currently costing about $5–$10/GB.